

OPA: One-shot Private Aggregation with Single Client Interaction and its Applications to Federated Learning

Harish Karthikeyan 

harish.karthikeyan@jpmchase.com

Antigoni Polychroniadou 

antigoni.polychroniadou@jpmorgan.com

JPMorgan AI Research, JPMorgan AlgoCRYPT CoE

Abstract

Our work minimizes interaction in secure computation, addressing the high cost of communication rounds, especially with many clients. We introduce One-shot Private Aggregation **OPA**, enabling clients to communicate only once per aggregation evaluation in a single-server setting. This simplifies dropout management and dynamic participation, contrasting with multi-round protocols like Bonawitz et al. (CCS'17) (and subsequent works) and avoiding complex committee selection akin to YOSO. **OPA**'s communication behavior *closely* mimics learning-in-the-clear where each client party speaks only once.

OPA, built on LWR, LWE, class groups, and DCR, ensures single-round communication for all clients while also achieving sub-linear overhead in the number of clients, making it asymptotically efficient and practical. We achieve malicious security with abort and input validation to defend against poisoning attacks, which are particularly relevant in Federated Learning, where adversaries attempt to manipulate the gradients to degrade model performance or introduce biases.

We build two flavors of **OPA** (1) from (threshold) key homomorphic PRF and (2) from seed homomorphic PRG and secret sharing. The threshold Key homomorphic PRF addresses shortcomings observed in previous works that relied on DDH and LWR in the work of Boneh *et al.* (CRYPTO, 2013), marking it as an independent contribution to our work. Our other contributions include new constructions of (threshold) key-homomorphic PRFs and seed-homomorphic PRGs that are secure under the LWE, DCR Assumption, and other Class Groups of Unknown Order.

1 Introduction

Minimizing interaction in Multiparty Computation (MPC) is a highly sought-after objective in secure computation. This is primarily because each communication round is costly, and ensuring the liveness of participants, particularly in scenarios involving a large number of parties, poses significant challenges. Unlike throughput, latency is now primarily constrained by physical limitations, making it exceedingly difficult to reduce the time required for a communication round substantially. Furthermore, non-interactive primitives offer increased versatility and are better suited as foundational building blocks. However, any non-interactive protocol, which operates with a single communication round, becomes susceptible to a vulnerability referred to as the “residual attack” [56] where the server can collude with some clients and evaluate the function on as many inputs as they wish revealing the inputs of the honest parties.

We explore a natural “hybrid” model between the 2-round and 1-round settings. Our model allows for private aggregation, aided by an ephemeral committee of members, where the clients and committee members only speak once. This approach brings us closer to achieving non-interactive protocols while preserving traditional security guarantees. Our specific focus is within the domain of secure aggregation protocols, where a group of n clients P_i for $i \in [n]$ hold a private value x_i , wish to learn the sum $\sum_i x_i$ without leaking any information about the individual x_i . Furthermore, we support multiple sessions or iterations of the secure aggregation, where a different random set of clients is selected in each session, each with a different

input. In this model, per aggregation session clients release encoded versions of their confidential inputs x_i to a designated committee of ephemeral members and they go offline, they only speak once. Later, any subset of the ephemeral members can compute these encodings by transmitting a single public message to an unchanging, stateless evaluator or server. This message conveys solely the outcome of the secure aggregation and nothing else. The ephemeral members are stateless, speak only once, and can change (or not) per aggregation session. With that in mind, the committee members can be regarded as another subset of clients who abstain from contributing input when selected to serve on the committee during a current aggregation session. Each client/committee member communicates once per aggregation, eliminating the complexity of handling dropouts commonly encountered in multi-round secure aggregation protocols. The security guarantee ensures that adversaries corrupting some clients and committee members learn only the sum of honest clients’ outputs, with no additional information. We provide a standard simulation-based proof against malicious adversaries with abort.

It is crucial to highlight the distinction between our single-server setting and multi-server protocols [50, 39, 3, 84, 102]. In the multi-server model, clients securely distribute their inputs across a committee of multiple servers, which then engage in interactive protocols to achieve secure aggregation. However, these servers must be stateful, retaining data across aggregation iterations, and require significant computational resources. In contrast, our approach leverages ephemeral committee members who are stateless and operate with lightweight computation, eliminating the need for persistent data storage or extensive computation. A key design goal of our model is to ensure that committee members remain computationally light, making it feasible even for resource-constrained client devices to be committee members.

Our main application is Federated Learning (FL) in which a server trains a model using data from multiple clients. This process unfolds in iterations where a randomly chosen subset of clients (or a set of clients based on the history of their availability) receives the model’s current weights. These clients update the model using their local data and return the updated weights. The server then computes the “average” of these weights (in the naive setting this is known as FedAvg [75] where the global model is simply the average of the client model weights), repeating this cycle until model convergence is achieved. Intuitively, this was supposed to guarantee the privacy of the client-held data as the server only sees the final weights. Unfortunately, prior works, such as [91], have shown that the final weights can be successfully used to recover client-held data. This motivates the need for a secure aggregation tool. Unlike previous multi-round secure aggregation schemes with or without an offline setup [16, 12, 53, 64, 73], we introduce the first protocol that minimizes client involvement, ensuring that both clients and committee members can be computationally lightweight devices while speaking only once per aggregation iteration. Furthermore, our protocol does not require offline setup and guarantees completion, even if selected clients or committee members drop out and remain silent.

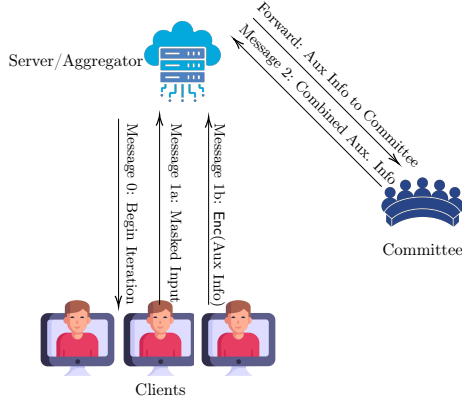
2 Our Contributions

We introduce OPA designed to achieve maximal flexibility by granting parties the choice to speak once or remain silent, fostering dynamic participation from one aggregation to the next. We present the communication model in Figure 1a. This diverges from prior approaches [16, 12, 64, 73, 54], which necessitate multiple interaction rounds and the management of dropout parties to handle communication delays or lost connections in FL. At its core, every iteration of OPA employs a random committee of size m as intermediate helper parties. We build OPA from new leakage-resilient, seed-homomorphic PRGs with simulatable leakage.¹ OPA relies on a mechanism such as PKI or authenticated channels.

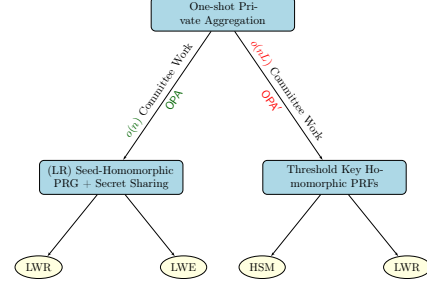
Cryptographic Assumptions: We construct OPA protocols providing a suite of six distinct versions based on a diverse spectrum of assumptions:

- Learning With Rounding (LWR) Assumption.
- Learning with Errors (LWE) Assumption.
- Hidden Subgroup Membership (HSM_M) assumption where M is a prime integer.

¹We can also build it from a (length-extended) key-homomorphic pseudorandom function (KHPRF) in the random oracle model with the same asymptotic performance. See Section C for the instantiations.



(a) The OPA system model operates in iterations. Each iteration begins with the server sending a message to initiate the process (Message 0). In response, clients train the model on their local data, obtain updates, and mask the input. (Message 1): masked input is sent to the server, while auxiliary information is transmitted (via encryption) to the committee via the server. Upon receiving the forwarded information, the committee members combine these into a single value. Finally, this consolidated data is sent to the server (Message 2), concluding the iteration. Importantly, the encryption information sent to the committee includes context information such as the current model and iteration. This prevents a malicious server from asking the committee for help with the same ciphertext. This thwarts residual attacks.



(b) Our Cryptographic Assumptions. Here, n is the number of clients, and L is the length of the input vector. Finally, “LR” refers to leakage-resilient. Note that the HSM assumption is parametrized by M and covers the case where M is a prime integer, $M = p^k$ for some prime p and integer k , $M = 2^k$, and $M = N$ where N is an RSA modulus (i.e., the DCR assumption).

Figure 1: OPA communication model and summarized contributions.

- HSM_M assumption where $M = p^k$ for some prime p and integer k .
- HSM_M assumption where $M = 2^k$.
- HSM_M assumption where $M = N$ where N is an RSA modulus (i.e., the DCR assumption).

OPA does not require any trusted setup for keys, and for M being either a prime or an exponent of prime, or the LWR assumption, we do not require any trusted setup of parameters either. The contributions are summarized in Figure 1b.

Threat Model: We assume a static, malicious adversary that can corrupt up to an $\eta < N$ fraction of them, where N is the number of clients in the universe of clients. For the committee of size m , the adversary can corrupt up to an η_C fraction. Additionally, up to a δ fraction of input-providing clients and a δ_C fraction of committee members may drop out per iteration. For OPA to function, we require $\delta_C + \eta_C < 1/3$ per iteration. In each iteration, a malicious adversary only learns the sum of the inputs of at least $(1 - \delta - \eta)|\mathcal{C}|$ where $\mathcal{C}$ is the number of online clients in that iteration. We ensure security against a malicious adversary, achieving overall malicious security with abort, a feature not present in prior works such as [16, 12, 73, 54, 64]. Specifically, when all parties adhere to the protocol, the server successfully computes the sum of the online clients’ contributions for a given iteration, provided a sufficient number of parties remain active during that iteration. Notably, we introduce a novel alternative approach instead of relying on signatures², as done in prior work to secure against a malicious server.

Other contributions beyond Secure Aggregation :

- New Key Homomorphic PRF Constructions: We develop the first Key Homomorphic PRF based on the HSM_M assumption and based on the LWE assumption.
- New Threshold Key Homomorphic PRF Constructions: We extend the HSM_M construction to a distributed key homomorphic setting using a modified Shamir’s Secret Sharing scheme over integers. We also

²See Remark 2 for more details.

extend the almost Key Homomorphic PRF based on the LWR assumption [18, 46] using Shamir Secret Sharing over prime-order fields. In doing so, we fix gaps in the prior Distributed Key Homomorphic PRF based on LWR, as proposed by Boneh *et al.* [18].

- **Packed Secret Sharing over Integers:** We also extend Shamir Secret Sharing over Integers to a packed version, which enables packing more secrets in one succinct representation.
- **New Seed Homomorphic PRG Construction:** Of independent interest, we also build a seed homomorphic PRG from the HSM_M assumption.³

Applications in Federated Learning: Our motivating application is Federated Learning (FL). The dynamic participation feature of OPA, crucial for federated learning, facilitates secure federated learning, where participants speak only once, streamlining the process significantly. In contrast, prior works [16, 12] involve eight rounds, and the work of [73] requires seven rounds, including the setup. Our advantages extend beyond just round complexity. See Section 2.1 and Tables 1 and 1 for a detailed comparison of asymptotics. We introduce another variant, OPA' , derived from a threshold key-homomorphic PRF, eliminating simulatable leakage. In OPA' , the committee’s workload scales with the length of the input vector L . Given its linear performance scaling with L , this remains a theoretical result aimed at achieving secure aggregation from the HSM_M assumption and for small L . In Section G, we show how to achieve security of honest user’s inputs in OPA' against a corrupt committee (but an honest server), for OPA' instantiated with HSM_M assumption (called OPA_{CL}).

Implementation and Benchmarks: We implement OPA as a secure aggregation protocol and benchmark with several state-of-the-art solutions [16, 12, 54, 73]. We benchmark OPA_{LWR} , based on the LWR-based seed homomorphic PRG, offering competitive performance over prior works. Concretely, at 1000 clients, OPA_{LWR} has a server computation time of 0.31s, orders of magnitude smaller than other protocols. Similar orders of magnitude improvement are observed in client performance over other protocols’ running time.

To further demonstrate the feasibility of our protocol, we train a binary classification model using logistic regression in a federated manner for two datasets. Our protocol carefully handles floating point values (using two different methods of quantization - multiplying by a global multiplier vs representing floating point numbers as a vector of integer values), and the resulting model is shown to offer performance close to simply learning in the clear. We also trained a neural network-based MLP classifier over popular machine learning datasets, including MNIST, CIFAR-10, and CIFAR-100. More details can be found in Section 9.

Resilience against Data Heterogeneity and Malicious Clients. While OPA is a tool that achieves properties expected of standard secure aggregation, we show how to combine OPA with existing solutions from the machine learning community to handle the heterogeneity of data distribution, i.e., when the clients’ datasets are non-iid distributed and when there are poisoning attacks from the clients to their contributed inputs. This is detailed in Section F, where we are the first to work on secure aggregation to replace FedAvg with FedOPT [86], which performs better accuracy and convergence when data is non-iid. We also show how to combine OPA with Byzantine Robust Stochastic Aggregation [65], which is secure against poisoning attacks of clients. Meanwhile, we also describe how to use lattice-based zero-knowledge proofs to ensure that a client can prove their honest behavior. This is illustrated in Section 8.

2.1 Detailed Contributions in Federated Learning

Next, we compare our protocol with all efficient summation protocols in terms of high-level features listed in Table 2, focusing on those that accommodate dynamic participation, a key feature shared by all federated learning methodologies. Regarding performance in Table 1, we list the communication complexity, computational complexity, and round complexity per participant. Notably, our protocols are setup-free, eliminating

³It is important to note that one cannot build OPA from this seed-homomorphic PRG. The main reason is that the seed space is in a group of an unknown order. Therefore, one requires an integer secret-sharing scheme. Unfortunately, because the server reconstructs an integer value, one cannot prove that this construction is leakage resilient, where the leakage is defined as $\text{sd} + r$. This is not indistinguishable from sd' , where they are all integers.

any need for elaborate initialization procedures. Furthermore, they are characterized by a streamlined communication process, demanding just a single round of interaction from the participants.

Asymptotic Comparison. More concretely, based on Table 1, our approach stands out by significantly reducing the round complexity, ensuring that each participant’s involvement is limited to a single communication round, i.e., each participant speaks only once. That is, users speak once and committee members speak once too. On the contrary, previous works[16, 12]⁴ require 8 rounds and the work of [73] requires 7 rounds in total, including the setup. This reduction in round complexity serves as a significant efficiency advantage.

Despite our advantage in the round complexity, our advantages extend beyond just round complexity (see Table 1). Notably, our protocol excels in computational complexity as the number of participants (n) grows. While previous solutions exhibit complexities that are quadratic [16, 68] or linearithmic [73] in n , our approach maintains a logarithmic complexity for the users, which is noteworthy when considering our protocol’s concurrent reduction in the number of communication rounds. Furthermore, our committee framework demonstrates a sublinear relationship with n for the committee members, a notable improvement compared to the linearithmic complexity and setup requirement in the case of [73] which considers a stateful set of decryptors (committee), as opposed to our stateless committee.⁵

When it comes to user communication and message sizes, previous solutions entail user complexities that either scale linearly [16, 68] or linearithmically [73] with the number of participants (n) according to Table 1. However, in our case, user communication complexity is reduced to a logarithmic level ($m \approx \log n$). Furthermore, as the number of users n increases, the communication load placed on the server is also effectively reduced compared to other existing protocols. That said, the above advantages underline the scalability and efficiency of our protocols in the federated learning context, which typically requires a very large number of n and L where L is the input length per client. It is important to stress that the number of committee members a client speaks to (m) purely depends on η , the fraction of malicious clients in the entire population of size N . We discuss sampling these clients in Section 7.2. Jumping ahead, we sample $m \log n$ clients for the committee and ensure that we assign clients to committee members, such that each client speaks with at most m clients while each committee member receives communication from at most $\log n$ clients (via the server). This reduces the computation and the communication for the committee member to be $O(\log n)$.

The works of [16, 12, 73] address a malicious adversary that can provide false information regarding which users are online on behalf of the server by requiring users to verify the signatures of the claimed online set. This approach introduces an additional two rounds into each protocol, resulting in 10 rounds in [16, 12] and 5 rounds in [73] with 10 rounds of setup. The setup communication complexity of [73] also increases to $O(k \log^2 n)$. Prior work often required a two-thirds honest majority for the malicious setting. One can leverage this to introduce a gap between the reconstruction threshold, r , and the corruption threshold, t . Specifically, if $r > (m + t)/2$, then a malicious server can only reconstruct for a unique set, and any other information is purely random and unhelpful to the server. Furthermore, the usage of signatures typically adds a round of communication between the committee and the server. The combined technique of signatures and the reconstruction-corruption threshold gap ensures that the committee speaks once, while guaranteeing security against residual attacks. See Remark 2 for more details. This is the added communication and computation cost:

- Client: It sends an additional $O(m)$ elements and incurs a computation cost of $O(m)$ to perform one additional secret-sharing.
- Committee: Each committee member receives an additional $o(n)$ elements from the clients, via the server. It decrypts and forwards them, as is, to the server. There is no additional computation cost.
- Server: The server receives an additional $o(n)$ communication from each of the m committee members and incurs a computation cost of $O(nm \log m)$.

⁴[12] offer a weaker security definition from the other works: for some parameter α between $[0, 1]$, honest inputs are guaranteed to be aggregated at most once with at least α fraction of other inputs from honest users.

⁵Flamingo [73] employed *decryptors*, a random subset of clients the server chose to interact with it to remove masks from masked data sent by the more extensive set of clients. LERNA[64] also requires a fixed, stateful committee (like Flamingo) to secret share client keys, whereas we support dynamic, stateless committees that can change in every round.

Table 1: Total asymptotic computation and communication costs for all rounds per aggregation with semi-honest security. n denotes the total number of users per iteration, with committee size m and L is the length of the input vector. k is the security parameter, and ℓ is the bit length of each element. The “Rounds” column indicates the number of rounds in the setup phase (on the left, if applicable) and each aggregation iteration (on the right). A “round of communication” refers to a discrete step within a protocol during which a participant or a group of participants send messages to another participant or group of participants, and participants from the latter group must receive these messages before they can send their own messages in a subsequent round. “fwd” means that the server only forwards the messages from the users. The second column in the User Aggregation phase refers to the cost of the committee members.

Protocol	Rounds		Computation Cost				Communication Cost						
			Server		User		Server		User				
	Setup	Agg.	Setup	Agg.	Setup	Agg.	Setup	Agg.	Setup	Agg.			
BIK+17[16]	-	8	-	$O(n^2L)$	-	$O(n^2 + nL)$	-	$O(nL\ell + n^2k)$	-	$O(L\ell + nk)$			
BBG+20[12]	-	8	-	$O(n \log^2 n + nL \log n)$	-	$O(\log^2 n + L \log n)$	-	$O(nL\ell + nk \log^2 n)$	-	$O(L\ell + k \log n)$			
Flamingo[73]	4	3	fwd	$O(nL + n \log^2 n)$	$O(\log^2 n)$	$O(L + n \log n)$	$O(m^2 + n)$	$O(k \log n)$	$O(nL\ell + nk \log^2 n)$	$O(k \log n)$	$O(L\ell + nk \log n)$	$O(m^2 + n \log n)$	
LERNA [64]	1	1	1	fwd	$O((\kappa + n)L + \kappa^2)$	$O(\kappa^2)$	$O(L)$	$O(L + n)$	-	$O(Lnk + m^2 \cdot k)$	$O(\kappa^2 k)$	$O(\kappa L + L \log n + k)$	$O(L\kappa + L \log n + k)$
SASH[68]	-	10	-	$O(L + n^2)$	-	$O(L + n^2)$	-	$O(nL\ell + \kappa^2 k)$	-	$O(L\ell + nk)$			
OPA	-	1	1	-	$O(nL + m \log m)$	-	$O(L + m)$	$O(\log n)$	-	$O(Ln + m \cdot n \log n)$	-	$O(kL + m)$	$O(\log n)$
OPA'	-	1	1	-	$O(nL + L(m \log m))$	-	$O(L + mL)$	$O(nL)$	-	$O(kL(n + m))$	-	$O(k(L + mL))$	$O(k(nL + L))$

To guarantee security against malicious clients, we propose zero-knowledge proofs based on the work of Lyubashevsky *et al.* [70] and is described in Section 8. Critically, our proofs are designed such that the server performs the bulk of the verification with the committee only checking if the share it decrypts is a valid opening to the commitment for the underlying commit-and-prove proof system.

Comparison with Willow [13]. The concurrent work, Willow, employs a static, stateful committee of members, with non-committee members (clients) communicating only once, offering improvements over previous approaches like LERNA. While Willow supports client-to-committee communication that remains independent of committee size—compared to LERNA and OPA—this advantage comes with trade-offs:

1. Each committee members undergoes a two-round setup procedure. While Willow focuses on the single-iteration setting, it is unclear how Willow can be extended to multi-iteration setting without requiring a two-round setup procedure, at the beginning of each iteration. Specifically, to achieve constant server to committee communication, the committee members always provide the secret key corresponding to the public key encryption (PKE) scheme. This PKE is used to encrypt the second mask that is used for malicious privacy. Further, even in the semi-honest setting, the secret key of the committee members who have dropped out is revealed to the server.
2. Each committee member also participates in two-rounds during the decryption phase: one for threshold decryption of the aggregated ciphertext and another to supply key shares for any missing members—similar to LERNA, where committee members also communicate multiple times
3. They additionally require parties called verifiers who are tasked with validating the behavior of a possibly untrusted server, though these verifiers can be committee members.

In contrast, OPA employs stateless committee members that can dynamically change across iterations. All client parties, including committee members, communicate only once per aggregation round, eliminating the need for a setup procedure. Furthermore, OPA requires no additional verifier committee. Last but not least, unlike Willow, OPA offers input validation and malicious client security with abort.

3 Related Work

First, we compare with other communication models that bear similarities with OPA in Section 3.1. Second, we give an overview and compare OPA with other tools employed for aggregation, with privacy, under various models in Section 3.2, while summarizing our comparison in Table 2.

3.1 OPA vs Other Communication/Computation Models

Shuffle Model. Note that our model bears similarities to the shuffle model, in which clients dispatch input encodings to a shuffler or a committee of servers responsible for securely shuffling before the data reaches the server for aggregation, as in the recent work of Halevi et al. [55]. Nonetheless, it is important to note that such protocols typically entail multiple rounds among the committee servers to facilitate an efficient and secure shuffle protocol.

Multi-Server Secure Aggregation Protocols. It’s worth emphasizing that multi-server protocols, as documented in [50, 39, 3, 84, 102], have progressed to a point where their potential standardization by the IETF, as mentioned in [81], is indeed noteworthy. In the multi-server scenario, parties can share their inputs securely among a set of servers, which then collaborate to achieve secure aggregation. Some of the works in this domain include two-server solutions Elsa [85] and SuperFL [102] or the generic multi-server solution Flag [9]. Unfortunately, in the case of federated learning, which involves handling exceptionally long inputs, the secret-sharing approach becomes impractical due to the increased communication complexity associated with each input. Furthermore, these servers must have heavy computation power and be stateful (retaining data/state from iteration to iteration). In our protocol, the ephemeral parties are neither stateful nor require heavy computation. Finally, there must be non-collusion among a majority of the servers. Ensuring this constraint is logistically complicated to maintain.

Committee-Based MPC Committee-based MPC is widely used for handling scenarios involving a large number of parties. However, it faces a security vulnerability known as adaptive security, where an adversary can corrupt parties after committee selection. The YOSO model, introduced by Gentry *et al.* [49], proposes a model that offers adaptive security. In YOSO, committees speak once and are dynamically changed in each communication round, preventing adversaries from corrupting parties effectively. The key feature of YOSO is that the identity of the next committee is known only to its members, who communicate only once and then become obsolete to adversaries. YOSO runs generic secure computation calculations, and aggregation can be one of them. However, its efficiency is prohibitive for secure aggregation. In particular, the communication complexity of YOSO in the computational setting scales quadratic with the number of parties n (or linear in n if the cost is amortized over n gates for large circuits). Additionally, an expensive role assignment protocol is applied to select the committees. Like LERNA, in YOSO, specific committee sizes need to be fulfilled to run a protocol execution. Lastly, our protocol does not rely on any secure role assignment protocol to choose the committees since even if all committee members are corrupted, privacy is still preserved. Fluid MPC [36, 15] also considers committee-based general secure computation. However, like YOSO, it is not practical. Unlike YOSO, it lacks support for adaptive security.

Moreover, SCALES [2] considers ephemeral servers a la YOSO responsible for generic MPC computations where the clients only provide their inputs. This approach is of theoretical interest as it is based on heavy machinery such as garbling and oblivious transfer if they were to be considered for secure aggregation. Moreover, SCALES needs to make extra effort to hide the identities of the servers, which we do not require.

3.2 OPA vs Other Secure Aggregation Algorithms

Multi-Round Private Summation. We begin by revisiting the concept outlined in [56]. The first multi-round secure aggregation protocol, designed to enable a single server to learn the sum of inputs $x_1, \dots, x_n$ while hiding each input x_i , is based on the following idea. Each user i adds a mask r_i to their private input x_i . This mask remains hidden from both the server and all other users it exhibits the property of canceling out when combined with all the other masks, i.e., $\sum_{i \in [n]} r_i = 0$. Subsequently, each user forwards $X_i = x_i + r_i$ to the server. By aggregating all the X_i values, the server can then determine the sum of all x_i . More specifically, to generate these masks, a common key $k_{ij} = k_{ji} = \text{PRG}(g^{s_i s_j})$ is established by every pair of clients i, j . Here, g^{s_i} is an ephemeral “public key” associated with each client $i \in [n]$. This public key is shared with all other clients during an initial round and facilitated through the server. Importantly, the value s_i remains secret by each client i . Then, each client $i \in [n]$ computes the mask $r_i = \sum_{j < i} k_{ij} - \sum_{j > i} k_{ij}$ and due to

the cancellation property the server outputs $\sum_i X_i = \sum_i x_i$. In this protocol, users must engage in multiple rounds of communication, where each user communicates more than once. Moreover, the protocol prevents users from dropping out from an aggregation iteration.

Non-Interactive Private Summation with trusted setup. If we were to require users to communicate only once in a protocol iteration, we would encounter the challenge of mitigating residual attacks. In a prior study conducted by [90], a solution based on DDH was proposed to mitigate residual attacks by involving a trusted setup that assumes the generation of the common keys $k_{ij} = k_{ji}$ into the protocol. However, it is essential to highlight that this setup lacked the necessary mechanisms to accommodate dropouts and facilitate dynamic participation for multiple aggregation iterations. Furthermore, to ensure that the server cannot recover the masking key given a client’s masked inputs, the work relies on the DDH Assumption. An unfortunate consequence is that the server has to compute the discrete logarithm to recover the aggregate, a computationally expensive operation, particularly when dealing with large exponents. Numerous other works within this framework have emerged, each relying on distinct assumptions, effectively sidestepping the requirement for laborious discrete logarithm calculations. These include works based on the DCR assumption [58, 14], and lattice-based cryptography [10, 46, 94, 93, 98, 79].

Multi-Round Private Summation without Trusted Setup. Another research direction focuses on removing the need for a trusted setup by developing multi-round decentralized reusable setups that generate masks while ensuring the essential cancellation property. However, akin to the previously mentioned approaches, these protocols come with a caveat—they do not accommodate scenarios involving dropouts or dynamic user participation across multiple iterations. Dipsauce [21] is the first to formally introduce a definition for a distributed setup PSA with a security model based on k -regular graph, non-interactive key exchange protocols, and a distributed randomness beacon [35, 45, 83] to build their distributed setup PSA. Meanwhile, the work of Nguyen *et al.* [77], assuming a PKI (or a bulletin board where all the public keys are listed), computed the required Diffie-Hellman keys on the fly to then build a one-time decentralized sum protocol which allowed the server to sum up the inputs *one-time*, with their construction relying on class group-based cryptography. To facilitate multiple iterations of such an aggregation, they combined their one-time decentralized sum protocol with Multi-client Functional Encryption (MCFE) to build a privacy-preservation summation protocol that can work over multiple rounds without requiring a trusted setup and merely requiring a PKI. Unfortunately, per iteration, the clients need to be informed of the set of users participating in that round, and unfortunately, they cannot drop out once they are chosen.

Non-Interactive Private Summation with a collector. To circumvent the need for a trusted setup and multi-round decentralized arrangements, an approach is presented in the work of [63], which introduces an additional server known as the “collector”. The fundamental premise here is to ensure that the collector and the evaluation server do not collude, thus effectively mitigating the risks associated with residual attacks. This protocol does allow dynamic participation and dropouts per iteration.

Multi-Round Private Summation with Dynamic Participation (aka Secure Aggregation). Secure aggregation of users’ private data with the aid of a server has been well-studied in the context of federated learning. Given the iterative nature of federated learning, dynamic participation is crucial. It enables seamless integration of new parties and those chosen to participate in various learning iterations while also addressing the challenge of accommodating parties that may drop out during the aggregation phase due to communication failures or delays. Furthermore, an important problem in federated learning with user-constrained computation and wireless network resources is the computation and communication overhead, which wastes bandwidth, increases training time, and can even impact the model accuracy if many users drop out. Seminal contributions by Bonawitz *et al.* [16] and Bell *et al.* [12] have successfully proposed secure aggregation protocols designed to cater to a large number of users while addressing the dropout challenge in a federated learning setting. However, it’s important to note that these protocols have a notable drawback—substantial round complexity and overhead are incurred during each training iteration. Even in

the extensive corpus of research based on more complex cryptographic machinery (see [59] for a plethora of previous works) such as threshold additive homomorphic encryption etc., these persistent drawbacks continue to pose challenges. Notably, all follow up works [12, 64, 54, 73, 11, 68, 69] of [16] require multiple rounds of interaction based on distributed setups. With their adaptable nature, secure aggregation protocols hold relevance across a wide array of domains. They are applicable in various scenarios, including ensuring the security of voting processes, safeguarding privacy in browser telemetry as illustrated in [38], and facilitating data analytics for digital contact tracing, as seen in [5] besides enabling secure federated learning.

It is also important to note that some of these works - ACORN [11] and RoFL [69] build on top of the works of [16, 12] to tackle the problem of “input validation” using efficient zero-knowledge proof systems. The goal is for the clients to prove that encrypted inputs are “well-formed” to prevent poisoning attacks. RoFL allows for detection when a malicious client misbehaves, while ACORN presents a *non-constant* round protocol to identify and remove misbehaving clients. We leave it as a direction for future research on augmenting our protocol and supporting input validation. The above discussion is also summarized in Table 2, by looking at four properties (a) whether the aggregate can be efficiently recovered, (b) whether it allows dynamic participation, (c) whether it requires trusted setup or multi-round distributed setup, and (d) the security assumptions.

Comparison with LERNA [64]. LERNA requires a fixed, stateful committee (like Flamingo) to secret share client keys, whereas we support smaller, dynamic, stateless committees that can change in every round. Concretely, LERNA works by having *each* client (in the entire universe of clients, not just for that iteration) secret-share the keys with the committee. Consequently, LERNA’s committee needs to be much larger (2^{14} members for $\kappa = 40$ due to the number of shares they receive) and tolerate fewer dropouts than our approach. Furthermore, LERNA’s benchmarks assume 20K+ clients, while real-world deployments have 50-5000 clients per iteration. When the client count is low, the committee has to do significantly more work to handle and store the required large number of shares. That said, LERNA is not suitable for traditional FL applications. In the table, we use the same notations, as LERNA refers to committee size by utilizing κ in the committee calculations. LERNA could work for less than 16K parties, but the computation of committee members increases significantly as the number of parties decreases. Let us look at the concrete costs for the non-committee clients in both LERNA and OPA_{LWR} . Assuming 20K clients and $L = 50,000$, LERNA requires approximately 2GB of data followed by 0.91 MB per iteration. Meanwhile, OPA_{LWR} requires each client to send 2.36 MB per iteration. As a result, LERNA only becomes cost-effective after more than 1300 iterations, requiring a fixed, stateful, and large committee to stay alive. The communication cost difference becomes starker for the committee clients between LERNA and OPA_{LWR} as LERNA requires every client, in the universe of clients, to communicate with the committee clients at setup; meanwhile, OPA_{LWR} only requires the chosen clients ($\ll 20K$) to speak in every iteration, and there is no setup.

4 Technical Overview

In this work, we focus on building a primitive, One-shot Private Aggregation (OPA), that enables privacy-preserving aggregation of multiple inputs across several aggregation iterations whereby a client only speaks once on his will per iteration.

Seed-Homomorphic PRG (SHPRG) Recall that a pseudorandom generator PRG takes as input a random seed and outputs pseudorandom values. $\text{PRG} : \mathcal{K} \rightarrow \mathcal{Y}$ is seed-homomorphic if $\text{PRG}(s_1 \oplus s_2) = \text{PRG}(s_1) \otimes \text{PRG}(s_2)$ where $(\oplus, \mathcal{K})$ and $(\otimes, \mathcal{Y})$ are groups. While using the known LWR-based construction [18], we introduce the first LWE-based SHPRG. Both are almost seed-homomorphic, with induced error handled via input encoding/decoding. For the LWR construction (Construction 1): given random $\mathbf{A} \leftarrow \mathbb{Z}_q^{L \times n}$, $\text{PRG}_{\text{LWR}, \mathbf{A}}(\mathbf{s}) = \lfloor \mathbf{A}\mathbf{s} \rfloor_p$ where $p < q$, with error

$\text{PRG}_{\text{LWR}, \mathbf{A}}(\mathbf{s}_1 + \mathbf{s}_2) = \text{PRG}_{\text{LWR}, \mathbf{A}}(\mathbf{s}_1) + \text{PRG}_{\text{LWR}, \mathbf{A}}(\mathbf{s}_2) + \mathbf{e}$ for $\mathbf{e} \in \{0, 1\}^L$. Algorithm `PRG.Expand` computes the PRG output on input seed $\mathbf{sd}$. It is important to note that the LWE based SHPRG requires careful

Table 2: Comparison of Various Private Summation Protocols. **TD** stands for trusted dealer/trusted setup, **DS** stands for multi-round distributed setup. Note that **DS** implies several rounds of interaction. Here, efficient aggregate recovery refers to whether the server can recover the aggregate efficiently. For example, [90, 14, 54] require restrictions on input sizes to recover the aggregate due to the discrete logarithm computation.

	Efficient Aggregate	Dynamic Participation	TD vs DS	Assumptions
[90]	✗	✗	TD	DDH
[58]	✓	✗	TD	DCR
[14]	✗/✓	✗	TD	DDH/DCR
[10]	✓	✗	TD	LWE/R-LWE
[95]	✓	✗	TD	R-LWE
[98]	✓	✗	TD	AES
[93]	✓	✗	TD	RLWE
[63]	✗	✓	TD	DCR
[46]	✓	✗	TD	LWR
[21]	✗	✗	DS	LWR
[16, 12]	✓	✓	DS*	DDH
[54]	✓	✓	DS	DDH
[73]	✓	✓	DS	DDH
[64]	✓	✓	DS	DDH
<i>Our Work</i>	✓	✓	NA	HSM, LWR, (R)LWE

consideration of seed and error spaces and accounting for ensuing error in homomorphism while secret-sharing over appropriate seed spaces. We also provide the first construction of SHPRG based on HSM_M assumption as an independent result, which does not have any error in the homomorphism.

Secret Sharing over Finite Fields. In standard Shamir secret sharing [88], a secret s is shared via polynomial $f(X) = \sum_{i=0}^{r-1} a_i X^i$ where $a_0 = s$ and $a_1, \dots, a_{r-1}$ are random field elements. For m parties, party $i \in [m]$ receives share $f(i)$, allowing any r parties to reconstruct s while any $r - 1$ shares remain uniformly random. The corruption threshold can be lowered from $r - 1$ to t for additional properties. Packed secret sharing [47] enables hiding multiple secrets in a single polynomial. A secret sharing scheme SS consists of sharing algorithm $\{\text{sd}^{(j)}\}_{j \in [m]} \leftarrow \text{SS.Share}(\text{sd}, t, r, m)$ taking secret sd , thresholds t, r , party count m as input and outputting share $\text{sd}^{(j)}$ for each party j , and reconstruction algorithm $\text{sd} \leftarrow \text{SS.Reconstruct}(\{\text{sd}^{(j)}\}_{j \in S})$ taking shares $\{\text{sd}^{(j)}\}_{j \in S}$ as input and returning secret sd when $|S| \geq r$. Finally, Shamir’s Secret Sharing is linearly homomorphic, which we leverage. Specifically, given two secrets sd_1, sd_2 with j -th share $\text{sd}_1^{(j)}$ and $\text{sd}_2^{(j)}$, then $\text{sd}_1^{(j)} + \text{sd}_2^{(j)}$ is a valid share of $\text{sd}_1 + \text{sd}_2$. Note that since our corruption threshold $t < m/3$, we can use reconstruction techniques with error correction to guarantee that a malicious share holder does not thwart the reconstruction.

4.1 OPA based on seed-homomorphic PRG

Every client needs to ensure the privacy of their input. Therefore, a client has to mask their input. In iteration ℓ , if client i has input $\mathbf{x}_{i,\ell}$, it chooses a mask of the same length to “add” to the inputs. Let the mask be $\mathbf{mask}_{i,\ell}$ and the ciphertext is defined as $\mathbf{ct}_{i,\ell} = \mathbf{x}_{i,\ell} + \mathbf{mask}_{i,\ell}$. To ensure privacy, we need the mask chosen uniformly randomly from a large distribution. Furthermore, by performing the addition with respect to a modulus M , we get the property that for a random $\mathbf{mask}_{i,\ell}$, $\mathbf{ct}_{i,\ell}$ is identically distributed to a random element from $\mathbb{Z}_M$. The client i sends $\mathbf{ct}_{i,\ell}$ to the server. This is Message 1a in Figure 1a.

The server, upon receiving the ciphertexts, can add up the ciphertexts. This leaves it with $\sum_i \mathbf{ct}_{i,\ell} = \sum_i \mathbf{x}_{i,\ell} + \sum_i \mathbf{mask}_{i,\ell}$. The goal of the server is to recover $\sum_i \mathbf{x}_{i,\ell}$. Therefore, it requires $\sum_i \mathbf{mask}_{i,\ell}$ to complete the computation. In works on Private Stream Aggregation [90, 14, 58, 46], the assumption made is that $\sum_i \mathbf{mask}_{i,\ell} = 0$. However, this requires all the clients to participate, which is a difficult requirement in federated learning. Instead, our approach follows the prior works in federated learning [16, 12, 73, 64, 54, 13]⁶ to enlist the help of some intermediate parties to provide the server with $\sum_i \mathbf{mask}_{i,\ell}$, for only the participating clients. However, most previous works [16, 12] mandate that the masks must sum to zero, requiring mask reconstruction if a party drops out. This approach, often dependent on secret-sharing of the masks, results in multiple interaction rounds. Another recent line of works [54, 64, 73] necessitate an expensive setup for mask generation but still demands Shamir reconstruction by stateful committee members to handle dropout. Our approach is the first to break away from these paradigms, removing the need for zero-sum masks and setup, thereby significantly streamlining the process while working with ephemeral committee. In OPA, following Flamingo [73, Figure 1, Lines 2-7], m committee members are ephemerally chosen using randomness beacon [45]. More details can be found in Section 7.2.1. The list of committee members can be included in the opening message from server to client, to begin the iteration.

Working with the Committee- First Attempt. Each client i , therefore, has to communicate information about its respective $\mathbf{mask}_{i,\ell}$ to the committee members. We refer to this as “Aux info”, and we route it through the server to the committee member. The information is encrypted under the public key of the respective committee member. This is Message 1b in Figure 1a. This ensures that the server cannot recover the auxiliary information. Eventually, each committee member “combines” the aux info it has received to the server (Message 2 in Figure 1a), with the guarantee that this is sufficient to reconstruct $\sum_i \mathbf{mask}_{i,\ell}$. We rely on Shamir’s secret-sharing [88] to distribute a secret s to a committee of m such that as long as r number of them participate, the server can learn the secret s . The security of the secret is guaranteed even if the server colludes with t committee members. We require $r > (m + t)/2$. We also rely on the homomorphism property, which ensures that if a committee member receives shares of two secrets s_1, s_2 . Then, adding up these shares will help reconstruct the secret $s_1 + s_2$. While prior works relied on committees, our contribution is in (a) ensuring that the committee’s computation and communication cost is minimal and (b) ensuring that the client and committee speak once per iteration.

Optimizing Committee Performance The solution laid out above requires the clients to “secret-share” $\mathbf{mask}_{i,\ell}$. However, note that $\mathbf{mask}_{i,\ell}$ is as long as $\mathbf{x}_{i,\ell}$. Therefore, each committee member will receive communication $O(L)$ where n is the number of clients, and L is the length of the vector. It must also perform computations proportional to $O(nL)$. We can further reduce the complexity to $O(\log n \cdot L)$ by sampling a much larger committee such that each committee member receives communication from $\log n$ clients while each client speaks with m committee members. OPA reduces the burden of the committee by introducing a succinct communication independent of L to the committee. We rely on a structured pseudorandom generator (PRG) called a “seed-homomorphic PRG”. A seed-homomorphic PRG ensures that $\text{PRG}(\mathbf{sd}_1 + \mathbf{sd}_2) = \text{PRG}(\mathbf{sd}_1) + \text{PRG}(\mathbf{sd}_2)$. Now, each client i secret-shares their respective seeds $\mathbf{sd}_{i,\ell}$ (and not $\mathbf{mask}_{i,\ell}$), while setting $\mathbf{mask}_{i,\ell} = \text{PRG}(\mathbf{sd}_{i,\ell})$. By the homomorphism of the secret-sharing scheme, the server

⁶[16] had all the clients be the intermediate parties. In contrast, [12] created neighborhoods of clients where the intermediate parties were only the neighbors. [73, 64, 13] explicitly defined a committee of chosen clients, who may or may not be participating in that round.

can reconstruct $\sum_i \text{sd}_{i,\ell}$, and then expand it as $\text{PRG}(\sum_i \text{sd}_{i,\ell})$. By seed-homomorphism, this is also equal to $\sum_i \text{PRG}(\text{sd}_{i,\ell}) = \sum_i \text{mask}_{i,\ell}$.

Three key challenges arise: (1) server learns $\sum_{i=1}^n \text{sd}_{i,\ell}$, requiring leakage-resilient SHPRG (satisfied by our LWR/LWE constructions), (2) almost-homomorphic LWR/LWE PRGs need careful encoding/decoding, and (3) active security simulation requires valid ciphertext simulation for honest clients before finalizing dropout set. For (3), we resolve this in programmable random oracle model: each client adds a mask from hashing seed $\text{dig}_{i,\ell}$, shares $\{\text{dig}_{i,\ell}^{(j)}\}_{j \in [m]} \leftarrow \text{SS.Share}(\text{dig}_{i,\ell}, t, r, m)$, which committee decrypts and forwards for server reconstruction. Notably, committee information remains vector-length independent.

Though the underpinning idea of OPA combines seed-homomorphic PRG and an appropriate secret-sharing scheme, there are technical issues with presenting a generic construction. We begin by presenting the construction based on the Learning with Rounding Assumption. We present constructions from both LWR and LWE assumptions in Section 7.2.1 and Section 7.2.2, respectively.

Malicious Client Behavior and Heterogeneity. In applications of secure aggregation to Federated Learning, it is important for the model updates to be tolerant to any heterogeneity in data distribution. In Section F.1, we show how to extend FedOpt [86] to the setting involving secure aggregation. This two pronged approach of first securely aggregating the updates and then using an optimization technique to update the model weights, as a function of the aggregated updated, helps bring prior research on FedOpt to the secure aggregation setting. Additionally, we also detail how to combine OPA with Byzantine-Robust Stochastic Aggregation (bRSA). The latter was conceived to handle both heterogeneity in data and client’s malicious behavior when it came to providing inputs. Finally, we also present a protocol where a server can abort when malicious behavior is encountered. Specifically, when a client sends inconsistent information to the server and the committee members. We eschew the computation heavy Feldman’s Verifiable Secret Sharing (VSS) and instead rely on SCRAPE [26], along with other zero-knowledge proof techniques based on lattices [70]. Critically our proofs ensure that the bulk of the verification is done by the server with only *constant* work done by a client. We refer readers to Section 8 for more details.

4.2 OPA’ based on Threshold, Key-Homomorphic PRF

We present an alternative approach to building OPA’ based on a threshold, key-homomorphic PRF. We present this alternative approach that is suitable for small L .

CL Framework. We use the generalized version of the framework, as presented by Bouvier *et al.* [19]. Broadly, there exists a group $\widehat{\mathbb{G}}$ whose order is $M \cdot \widehat{s}$ where $\gcd(M, \widehat{s}) = 1$ and $\widehat{s}$ is unknown while M is a parameter of the scheme. Then, $\widehat{\mathbb{G}}$ admits a cyclic group $\mathbb{F}$, generated by f whose order is M . Consider the cyclic subgroup $\mathbb{H}$, generated by $h = x^M$, for a random $x \in \widehat{\mathbb{G}}$. Then, one can consider the cyclic subgroup $\mathbb{G}$ generated by $g = f \cdot h$ with $\mathbb{G}$ factoring as $\mathbb{F} \cdot \mathbb{H}$. The order of $\mathbb{G}$ is also unknown. The HSM_M assumption states that an adversary cannot distinguish between an element in $\mathbb{G}$ and $\mathbb{H}$, while a discrete logarithm is easy in $\mathbb{F}$, $\widehat{s}$, an upper bound for $\widehat{s}$ is provided as input. Note that for $M = N$ where N is an RSA modulus, the HSM_M assumption reduces to the DCR assumption. Therefore, the HSM_M assumption can be viewed as a generalization of the DCR assumption.

Secret Sharing over Integers. Braun *et al.* [20] was the first to identify how to suitably modify Shamir’s secret sharing protocol to ensure that the operations can work over a group of unknown order, such as the ones we use on the CL framework. This stems from two reasons. The first is leakage in that a share $f(i)$ corresponding to some sharing polynomial f always leaks information about the secret s , mod i when the operation is over the set of integers. Meanwhile, the standard approach to reconstructing the polynomial requires the computation of the Lagrange coefficients, which involves dividing by an integer, which again needs to be “reinterpreted” to work over the set of integers. The solution to these problems is multiplying with an offset $\Delta = m!$ where m is the total number of shares.

(Almost) Key Homomorphic Pseudorandom Functions Naor *et al.* [76] introduced the concept of key homomorphic PRFs (KH-PRFs), demonstrating that $H(x)^k$ is a secure KH-PRF under the DDH assumption in the Random Oracle Model, where H is a hash function, k is the key, and x is the input. [18] later constructed an almost KH-PRF under the Learning with Rounding assumption [8], which was formally proven secure by [46]. Our work leverages almost KH-PRF constructions from both LWR and LWE assumptions in the Random Oracle Model. Additionally, we present a novel KH-PRF construction in the CL framework, yielding new constructions under the HSM assumption (including DCR-based constructions). We adapt the DDH-based construction to the CL framework and prove that $F(k, x) = H(x)^k$, where $k \leftarrow \mathcal{K}$ and $H : \{0, 1\}^* \rightarrow \mathbb{H}$ is modeled as a random oracle, is a secure KH-PRF under the HSM assumption. This adaptation requires careful consideration of appropriate groups, input spaces, and key spaces.

Distributed Key Homomorphic Pseudorandom Functions (KHPRF) [18] presented generic constructions of Distributed PRFs from any KH-PRF using secret sharing techniques. However, the CL framework’s use of groups with unknown order necessitates working over integer spaces. While Linear Integer Secret Sharing [42] exists, it can be computationally expensive. Instead, we utilize Shamir Secret Sharing over Integer Space as described by [20], refining it with appropriate offsets to construct Distributed PRFs from our HSM-based construction. In other words, rather than reconstructing the secret, the solution reconstructs a deterministic function of the secret, which is accounted for in the PRF evaluation. This induces complications in reducing the security of our distributed key homomorphic PRF to that of the HSM-based KH-PRF. Finally, we demonstrate that our construction maintains the key homomorphism property, allowing combinations of partial evaluations of secret key shares to match the evaluation of the sum of keys at the same input point.

Constructions of *almost* Distributed KHPRFs. [18] introduced a generic construction of distributed KHPRFs from almost key homomorphic PRFs. They proposed $F_{\text{LWR}}(\mathbf{k}, x) := \lfloor \langle H(x), \mathbf{k} \rangle \rfloor_p$ as an almost key homomorphic PRF, where $q > p$ are primes, $\mathbf{k} \leftarrow \mathbb{Z}_q^p$, and H is a suitably defined hash function. Their reduction to build a distributed PRF utilizes standard Shamir’s Secret Sharing over fields, simplifying the process compared to integer secret sharing due to the prime nature of q and p . However, their proposed construction contained shortcomings affecting both correctness and security proofs. We will now provide a brief overview of these issues. An almost KH-PRF satisfies $F(k_1 + k_2, x) = F(k_1, x) + F(k_2, x) - e$ for some error e . For the LWR construction, $e \in \{0, 1\}$. However, this implies $F(T \cdot k, x) = T \cdot F(k, x) - e_T$ where $e_T \in \{0, \dots, T-1\}$ for any integer T , leading to error growth and affecting Lagrange interpolation. The authors proposed multiplying by an offset $\Delta = m!$ to bound the error and use rounding to mitigate its impact by ensuring that the error terms are “eaten” up. However, their security reduction works by reducing the security of the Distributed KHPRF (DKHPRF) to the underlying KHPRF. For a particular i^* , the adversary uses its oracle access to the KHPRF security game to obtain partial evaluations. In other words, the key of the KHPRF is implicitly set as the share of the key corresponding to some i^* . However, to use the KHPRF Challenger’s response to construct the actual evaluation of the DKHPRF (via Lagrange interpolation), one has to rely on the “clearing out the denominator” technique by multiplying with some offset Δ . Unfortunately, this induces additional errors when dealing with an *almost* KH-PRF such as the one based on the LWR assumption. Thus, their definition of partial evaluation function needs to be updated to be consistent with what is simulatable. We identified further issues in their rounding choices. Specifically, partial evaluations should be rounded down to u where $\lfloor p/u \rfloor > (\Delta + 1) \cdot r \cdot \Delta$ (r being the reconstruction threshold). Moreover, their framework only addressed single-key PRFs, not vector-key cases like LWR-based construction. We address these issues in constructing a distributed, almost key homomorphic PRF based on LWR. We formally prove that $F_{\text{LWR}}(\mathbf{k}, x) = \lfloor \Delta \lfloor \Delta \lfloor \langle H(x), \mathbf{k} \rangle_p \rfloor_u \rfloor_v$ for appropriate choices of u and v is a secure, distributed, almost key homomorphic, PRF.

One-shot Private Aggregation without Leakage Simulation. The construction of OPA’ is similar to the earlier ones based on seed-homomorphic PRG. There exist the following differences:

- A client i has to sample L different keys. Each key in is used to evaluate the PRF at a point ℓ and is used to mask the input $x_i^{(in)}$.

- It then secret shares each of the L keys by running the `DPRF.Share`, the algorithm to generate the shares of the DPRF key.
- Each share for committee member j is evaluated at ℓ . This evaluation is sent to the committee member.
- Finally, the server runs `DPRF.Combine` to combine the information from the committee members to get the PRF evaluation at ℓ under the sum of the keys for each index in . `Combine` is the algorithm that helps reconstruct the evaluation from partial evaluations.

Stronger Security Definitions and Construction. We also present a stronger security guarantee, which was not provided by the committees of Lerna and Flamingo, whereby the committee members can all collude and observe all encrypted ciphertexts and all auxiliary information and cannot mount an IND-CPA-style attack. Unfortunately, our current construction, where the inputs are solely blinded by the PRF evaluation, which is also provided to the committee members in shares, can be unblinded by the committee leaking information about the inputs. Instead, we will have the server choose its key material k_0 , and the corresponding key for that iteration communicated with the “initiate transaction” message. The client can then compute an “ephemeral” Diffie-Hellman key on the fly and use this to mask the input. Since the adversary does not receive k_0 , it intuitively provides a sufficient mask for the inputs and the actual key k_i of the honest client. Note that the adversary only receives the honest client’s “iteration” public key. The actual construction is presented in Section G.1.

5 Preliminaries and Cryptographic Building Blocks

Notations. For a distribution X , we use $x \leftarrow^* X$ to denote that x is a random sample drawn from the distribution X . We denote by $\mathbf{u}$ a vector and by $\mathbf{A}$ a matrix. For a set S we use $x \leftarrow^* S$ to denote that x is chosen uniformly at random from the set S . By $[n]$ for some integer n , we denote the set $\{1, \dots, n\}$. For $x \in \mathbb{R}$, $\lfloor x \rfloor$ (resp. $\lceil x \rceil$) refers to $y \in \mathbb{Z}$ such that $y \leq x < y + 1$ (resp. $y - 1 < x \leq y$).

Definition 1 (Hypergeometric Distribution). *A Hypergeometric Distribution $\text{HyperGeom}(N, \eta, m)$ is a discrete probability distribution that describes the probability of successes in m draws, without replacement from a finite population of size N that contains η fraction with that feature. We use the following tail bounds for $X \sim \text{HyperGeom}(N, \eta, m) \forall d > 0$, $\Pr[X \leq (\eta - d)m] \leq e^{-2d^2 \cdot m}$ and $\Pr[X \geq (\eta + d)m] \leq e^{-2d^2 \cdot m}$*

5.1 Cryptographic Preliminaries

We begin by discussing lattice-based assumptions in Section 5.2. Later, in Section 5.3, we introduce the syntax and security of a seed-homomorphic pseudorandom generator and present our lattice-based constructions.

Due to space constraints, we defer discussion on the syntax of secret-sharing schemes and constructions over fields and integers in Section A.1. This is followed by an exposition of various pseudorandom functions’ syntax and security definitions through Sections A.2 and A.3. Finally, we introduce the CL Framework in Section A.4.

5.2 Lattice-Based Assumptions

This section will look at constructions based on three different lattice-based assumptions.

5.2.1 Learning with Rounding Assumption

We will begin by defining the learning with rounding (LWR) assumption, which can be viewed as a deterministic version of the learning with errors (LWE) assumption [87]. LWR was introduced by Banerjee *et al.* [8].

Definition 2 (Learning with Rounding). Let $\lambda, q, p \leftarrow \text{LWRGen}(1^\rho)$ be functions of the security parameter ρ , with $L, \lambda, q, p \in \mathbb{N}$ such that $q > p$. Then, the Learning with Rounding assumption ($\text{LWR}_{\lambda, L, q, p}$) states that for all PPT adversaries $\mathcal{A}$, there exists a negligible function negl such that:

$$\Pr \left[b = b' \mid \begin{array}{l} \mathbf{s} \leftarrow \mathbb{Z}_q^\lambda, \mathbf{A} \leftarrow \mathbb{Z}_q^{\lambda \times L}, \\ \mathbf{u}_0 := \lfloor \mathbf{A}^\top \cdot \mathbf{s} \rfloor_p, \mathbf{u}_1 \leftarrow \mathbb{Z}_p^L \\ b \leftarrow \{0, 1\}, b' \leftarrow \mathcal{A}(\mathbf{A}, \mathbf{u}_b) \end{array} \right] = \frac{1}{2} + \text{negl}(\rho)$$

where $\lfloor x \rfloor_p := \lfloor x \cdot p/q \rfloor$.

5.2.2 Learning with Errors Assumption

Definition 3 (Learning with Errors Assumption (LWE)). Consider integers $\lambda, L, q \in \mathbb{N}$ that are functions of the security parameter ρ , and a probability distribution χ on $\mathbb{Z}_q$, typically taken to be a normal distribution that has been discretized. Then, the $\text{LWE}_{\lambda, L, q, \chi}$ assumption states that for all PPT adversaries $\mathcal{A}$, there exists a negligible function negl such that:

$$\Pr \left[b = b' \mid \begin{array}{l} \mathbf{A} \leftarrow \mathbb{Z}_q^{L \times \lambda}, \mathbf{x} \leftarrow \mathbb{Z}_q^\lambda, \mathbf{e} \leftarrow \chi^L \\ \mathbf{y}_0 := \mathbf{A}\mathbf{x} + \mathbf{e} \\ \mathbf{y}_1 \leftarrow \mathbb{Z}_q^L, b \leftarrow \{0, 1\}, b' \leftarrow \mathcal{A}(\mathbf{A}, \mathbf{y}_b) \end{array} \right] = \frac{1}{2} + \text{negl}(\rho)$$

Definition 4 (Hint-LWE [34, 44]). Consider integers λ, L, q and a probability distribution χ on $\mathbb{Z}_q$, typically taken to be a normal distribution that has been discretized. Then, the Hint-LWE assumption⁷ states that for all PPT adversaries $\mathcal{A}$, there exists a negligible function negl such that:

$$\Pr \left[b = b' \mid \begin{array}{l} \mathbf{A} \leftarrow \mathbb{Z}_q^{L \times \lambda}, \mathbf{k} \leftarrow \mathbb{Z}_q^\lambda, \mathbf{e} \leftarrow \chi^L \\ \mathbf{r} \leftarrow \mathbb{Z}_q^\lambda, \mathbf{f} \leftarrow \chi'^L \\ \mathbf{y}_0 := \mathbf{A}\mathbf{k} + \mathbf{e}, \mathbf{y}_1 \leftarrow \mathbb{Z}_q^L, b \leftarrow \{0, 1\} \\ b' \leftarrow \mathcal{A}(\mathbf{A}, (\mathbf{y}_b, \mathbf{k} + \mathbf{r}, \mathbf{e} + \mathbf{f})) \end{array} \right] = \frac{1}{2} + \text{negl}(\rho)$$

Where ρ is the security parameter.

5.3 Seed Homomorphic PRG

Definition 5 (Seed Homomorphic PRG (SHPRG)). Consider an efficiently computable function $\text{PRG} : \mathcal{K} \rightarrow \mathcal{Y}$, parametrized by $\text{PRG} = (\text{PRG.Gen}, \text{PRG.Expand})$ where $(\mathcal{K}, \oplus), (\mathcal{Y}, \otimes)$ are groups. Then $(\text{PRG}, \oplus, \otimes)$ is said to be a secure seed homomorphic pseudorandom generator (SHPRG) if:

- PRG is a secure pseudorandom generator (PRG), i.e., for all PPT adversaries $\mathcal{A}$, there exists a negligible function negl such that:

$$\Pr \left[b = b' \mid \begin{array}{l} \text{pp}_{\text{PRG}} \leftarrow \text{PRG.Gen}, b \leftarrow \{0, 1\}, \text{sd} \leftarrow \mathcal{K} \\ Y_0 = \text{PRG.Expand}(\text{pp}_{\text{PRG}}, \text{sd}), Y_1 \leftarrow cY \\ b' \leftarrow \mathcal{A}(Y_b) \end{array} \right] \leq \frac{1}{2} + \text{negl}(\kappa)$$

- For every $\text{sd}_1, \text{sd}_2 \in \mathcal{K}$, we have that $\text{PRG.Expand}(\text{sd}_1) \otimes \text{PRG.Expand}(\text{sd}_2) = \text{PRG.Expand}(\text{sd}_1 \oplus \text{sd}_2)$

We abuse notation and drop pp_{PRG} from the input of Expand .

⁷Kim *et al.* [61] demonstrates that the Hint-LWE assumption is computationally equivalent to the standard LWE assumption. This assumption posits that $\mathbf{y}_0$ maintains its pseudorandom properties from an adversary's perspective, even when provided with certain randomized information about the secret and error vectors. Consider a secure LWE instance defined by parameters (λ, m, q, χ) , where χ represents a discrete Gaussian distribution with standard deviation σ . The corresponding Hint-LWE instance, characterized by (λ, m, q, χ') , where χ' denotes a discrete Gaussian distribution with standard deviation σ' , remains secure under the condition $\sigma' = \sigma/\sqrt{2}$. As a result, we can decompose any $\mathbf{e} \in \chi$ into the sum $\mathbf{e}_1 + \mathbf{e}_2$, where both $\mathbf{e}_1$ and $\mathbf{e}_2$ are drawn from χ' .

6 Constructions of SHPRG

In this section, we first recap the construction of SHPRG based on LWR assumption as presented by Boneh *et al.* [18]. We then show that it is leakage-resilient. Later, we present the *first* construction based on LWE assumption and then prove that it is indeed leakage resilient under Hint-LWE assumption.

6.1 Construction from LWR Assumption

Construction 1 (SHPRG from LWR Assumption). Let $\text{pp}_{\text{PRG}} := \mathbf{A} \leftarrow \mathbb{Z}_q^{\lambda \times L}$ be the output of PRG.Gen , then consider the following: $\text{PRG}_{\text{LWR}} : \mathbb{Z}_q^\lambda \rightarrow \mathbb{Z}_p^L$ where $L > \lambda$ is defined as $\text{PRG.Expand}(\text{sd} = \mathbf{s}) = \lfloor \mathbf{A}^\top \cdot \mathbf{s} \rfloor_p$ where $q > p$ with $\lfloor x \rfloor_p = \lfloor x \cdot p/q \rfloor$ for $x \in \mathbb{Z}_q$.

It is easy to see that Construction 1 is a secure PRG under the LWR assumption, which gives us the following theorem

Theorem 1 (PRG Security of Construction 1). *If $\text{LWR}_{\lambda, L, q, p}$ assumption holds, then PRG_{LWR} is a secure PRG.*

This is almost seed homomorphic in that: $\text{PRG.Expand}(\mathbf{s}_1 + \mathbf{s}_2) = \text{PRG.Expand}(\mathbf{s}_1) + \text{PRG.Expand}(\mathbf{s}_2) + \mathbf{e}$ where $\mathbf{e} \in \{0, 1\}^L$.

Theorem 2 (Leakage Resilience of Construction 1). *Let PRG_{LWR} be the PRG defined in Construction 1. Then, it is leakage resilient in the following sense:*

$$\left\{ \begin{array}{ll} \text{PRG.Expand}(\text{sd}) \bmod p & \text{sd} \leftarrow \mathbb{Z}_q^\lambda \\ \text{sd} + \text{sd}' \bmod q & \text{sd}' \leftarrow \mathbb{Z}_q^\lambda \end{array} \right\} \approx_c \left\{ \begin{array}{ll} \mathbf{y} & \text{sd}, \text{sd}' \leftarrow \mathbb{Z}_q^\lambda \\ \text{sd} + \text{sd}' \bmod q & \mathbf{y} \leftarrow \mathbb{Z}_p^L \end{array} \right\}$$

Proof. The proof proceeds through a sequence of hybrids.

Hybrid₀(κ): The left distribution is provided to the adversary. In other words, the adversary gets:

$$\{\text{PRG}_{\text{LWR}}(\text{sd}) \bmod p, \text{sd} + \text{sd}' \bmod q : \text{sd}, \text{sd}' \leftarrow \mathbb{Z}_q^\lambda\}$$

Hybrid₁(κ): In this hybrid, we replace $\text{sd} + \text{sd}' \bmod q$ with a uniformly random value $\text{sd}'' \leftarrow \mathbb{Z}_q^\lambda$.

$$\{\text{PRG}_{\text{LWR}}(\text{sd}) \bmod p, \text{sd}'' : \text{sd}, \text{sd}'' \leftarrow \mathbb{Z}_q^\lambda\}$$

Note that $(\text{sd} + \text{sd}') \bmod q$ and sd'' are identically distributed. Let us assume that there exists a leakage function oracle L that can be queried with an input sd , for which it either outputs $\text{sd} + \text{sd}' \bmod q$ for a randomly sampled $\text{sd}' \leftarrow \mathbb{Z}_q^\lambda$ or outputs $\text{sd}'' \leftarrow \mathbb{Z}_q^\lambda$. Then, an adversary $\mathcal{A}$ that can distinguish between Hybrid₀ and Hybrid₁ can be used by an adversary $\mathcal{B}$ to break the leakage function as follows: $\mathcal{B}$ samples sd . It sends this value to the leakage function. Meanwhile, it also computes $\text{PRG}_{\text{LWR}}(\text{sd})$ and appends the outputs of the leakage function. Note that if the leakage output was $\text{sd} + \text{sd}'$, the Hybrid₀ is simulated by $\mathcal{B}$, and if it was sd'' then Hybrid₁ is simulated by $\mathcal{B}$. However, the output of the leakage function is an identical distribution. Therefore, Hybrid₀, Hybrid₁ are also identically distributed.

Hybrid₂(κ): In this hybrid, we will replace the PRG computation with a random value from the range.

$$\{\mathbf{y}, \text{sd}'' : \mathbf{y} \leftarrow \mathbb{Z}_p^L, \text{sd}'' \leftarrow \mathbb{Z}_q^\lambda\}$$

Under the security of the PRG, we get that Hybrid₁, Hybrid₂ are computationally indistinguishable.

Hybrid₃(κ): We replace sd'' with $(\text{sd} + \text{sd}') \bmod q$.

$$\{\mathbf{y}, (\text{sd} + \text{sd}') \bmod q : \mathbf{y} \leftarrow \mathbb{Z}_p^L, \text{sd}, \text{sd}' \leftarrow \mathbb{Z}_q^\lambda\}$$

As argued before Hybrid₂, Hybrid₃ are identically distributed.

Note that Hybrid₃ is the right distribution from the theorem statement. This completes the proof. $\square$

6.2 Construction from LWE Assumption

Construction 2 (SHPRG from LWE Assumption). Let $\mathbf{A} \leftarrow \mathbb{Z}_q^{L \times \lambda}$ be the output of PRG.Gen . Then, consider the following seed homomorphic PRG $\text{PRG}_{\text{LWE}} : \mathbb{Z}_q^\lambda \times \chi^L \rightarrow \mathbb{Z}_q^L$ is defined as: $\text{PRG.Expand}((\mathbf{s}, \mathbf{e})) = \mathbf{A}\mathbf{s} + \mathbf{e}$

The proof of security directly applies the LWE Assumption. However, we now prove the (almost) seed homomorphic property.

$$\begin{aligned} \text{PRG.Expand}(\mathbf{s}_1, \mathbf{e}_1) &= \mathbf{A}\mathbf{s}_1 + \mathbf{e}_1; \text{PRG.Expand}(\mathbf{s}_2, \mathbf{e}_2) = \mathbf{A}\mathbf{s}_2 + \mathbf{e}_2 \\ \text{PRG.Expand}(\mathbf{s}_1, \mathbf{e}_1) + \text{PRG.Expand}(\mathbf{s}_2, \mathbf{e}_2) &= \mathbf{A}(\mathbf{s}_1 + \mathbf{s}_2) + (\mathbf{e}_1 + \mathbf{e}_2) \\ &= \text{PRG.Expand}(\mathbf{s}_1 + \mathbf{s}_2) + \mathbf{e} \end{aligned}$$

where $\mathbf{e} \leq \mathbf{e}_1 + \mathbf{e}_2 + 1$.

Looking ahead, when we use this PRG to mask the inputs, we will do the following: $\text{PRG.Expand}(s_i, \mathbf{e}_i) = \mathbf{A}\mathbf{s}_i + \mathbf{e}_i + \lfloor q/p \rfloor \cdot \mathbf{x}_i$ where $\mathbf{x}_i \in \mathbb{Z}_p^m$. Upon adding n such ciphertexts (mod q), we get:

$$\mathbf{A} \sum_{i=1}^n \mathbf{s}_i + \mathbf{e} + \lfloor q/p \rfloor \cdot \sum_{i=1}^n \mathbf{x}_i$$

where $\mathbf{e} \leq 1 + \sum_{i=1}^n \mathbf{e}_i$. Therefore, to eventually recover $\sum_{i=1}^n \mathbf{x}_i \bmod p$ from just the value of $\sum_{i=1}^n \mathbf{s}_i$, we will require $\|\mathbf{e}\|_\infty < \frac{q}{2p}$. Looking ahead, we will rely on the Hint-LWE Assumption 4 to show that it is leakage resilient when we use it to build our aggregation tool.

Remark 1 (Construction based on Ring-LWE). The above LWE construction can be extended to the Ring-LWE [71] setting.

Recall the R-LWE Assumption. Let N be a power of two and $m > 0$ be an integer. Let R be a cyclotomic ring of degree N , and let R_q be its residue ring modulo $q > 0$. Then, the following holds:

$$\{(\mathbf{a}, \mathbf{a} \cdot k + \mathbf{e}) : \mathbf{a} \leftarrow R_q^m, k \leftarrow R_q, \mathbf{e} \leftarrow \chi^m\} \approx_c \{(\mathbf{a}, \mathbf{u}) : \mathbf{a} \leftarrow R_q^m, \mathbf{u} \leftarrow R_q^m\}$$

This gives us the following construction: $\text{PRG}_{\text{R-LWE}}((k, \mathbf{e})) : \mathbf{a}k + \mathbf{e}$

7 One-shot Private Aggregation

In this section, we construct One-shot Private Aggregation (OPA). Broadly speaking, the goal of this primitive is to support a server (aka aggregator) to sum up the clients' inputs encrypted to a particular label (which can be a tag, timestamp, etc.), without it learning any information about the inputs beyond just the sum.

7.1 Simulation-Based Privacy

Our proof approach is based on the standard simulation-based framework [52, 66] where we demonstrate that any attacker against our protocol can be simulated by an attacker Sim in an ideal world where a trusted party $\mathcal{T}$ computes a function F on the clients' inputs X . In our case, this function is that of vector summation. We consider an attacker $\mathcal{A}$ that controls at most $\eta \cdot n$ clients and possibly the server. Further, η_C will be the proportion of corrupt clients controlled by the adversary and those belonging to the committee. The ideal world consists of the following steps, which are adapted to the more straightforward setting where only one party, i.e., the server, has the output:

- (a) The honest clients provide the inputs to the trusted party $\mathcal{T}$.
- (b) Sim chooses which corrupted clients send the input and which abort.
- (c) If the server is corrupted, then Sim can either choose to abort the protocol or continue.

- (d) If the protocol is not aborted, then $\mathcal{T}$ computes the function $F(X)$ and sends to the server.
- (e) Finally, if the server is not corrupted, it outputs what it has received from $\mathcal{T}$.

Our function F is parametrized by the following: (a) the set of inputs $X = \{\mathbf{x}_{i,\ell}\}_{i \in [n]}$, (b) the set of client dropouts $D \subseteq [n]$, and (c) $\delta \in [0, 1]$ which is the maximum fraction of dropouts permitted. In addition, we also parametrize it by the iteration ID ℓ . Then,

$$F_{D,\delta}^\ell = \begin{cases} \sum_{i \in [n] \setminus D} \mathbf{x}_{i,\ell} & \text{if } |D| \leq \delta n \\ \perp & \text{otherwise} \end{cases} \quad (1)$$

7.2 Our Construction of One-shot Private Aggregation Scheme

7.2.1 Construction of OPA_{LWR}

We now present OPA_{LWR} and prove its correctness and security. We rely on the seed-homomorphic PRG (Construction 1) PRG with seed space $\text{PRG.K} := \mathbb{Z}_q^\lambda$ and combine it with Shamir's Secret Sharing Scheme over $\mathbb{F}_q$ (Construction 5). We will also employ a public key encryption scheme Enc , which clients use to encrypt the shares to the committee. To facilitate, we assume a PKI or any other mechanism, such as authenticated channels, to ensure that every client knows a public key pk_j to encrypt to committee member j .⁸

For ease of presentation, we will abuse notation and drop the packing factor ρ (of the secret-sharing scheme) and the consequent vector slicing needed for the seed. We chunk the vector of length λ into smaller vectors, each of length ρ , before secret-sharing each smaller vector.

Correctness. First, recall that Construction 1 is only almost seed homomorphic. In other words,

$$\text{PRG}(\text{sd}_1 + \text{sd}_2) = \text{PRG}(\text{sd}_1) + \text{PRG}(\text{sd}_2) + e$$

where $e \in \{0, 1\}$.

For ease of presentation, our correctness proof is for $L = 1$, but it extends to any arbitrary L . Therefore, while the correctness of Shamir's Secret Sharing scheme guarantees that sd_ℓ , computed by the server, is indeed $\sum_{i=1}^n \text{sd}_i \bmod q$, there is an error growth in AUX_ℓ . Specifically, we get that:

$$\text{AUX}_\ell := \text{PRG.Expand} \left(\text{sd}_\ell = \sum_{i=1}^n \text{sd}_{i,\ell} \right) = \sum_{i=1}^n \text{PRG.Expand}(\text{sd}_{i,\ell}) + e'$$

Or,

$$\sum_{i=1}^n \text{Expand}(\text{sd}_{i,\ell}) = \text{AUX}_\ell - e' \quad (2)$$

where $e' \in \{0, \dots, n-1\}$. We know that $\text{Encode}(x_{i,\ell}) := \Delta \cdot x_{i,\ell} + r_i + 1$ where $\Delta = n \cdot 2^{\kappa_s}$ and $r_i \leftarrow \{0, \dots, 2^{\kappa_s} - 1\}$ which gets:

$$\sum_{i=1}^n \text{ct}_{i,\ell} = \left(\sum_{i=1}^n (x_{i,\ell} \Delta + r_i) + \text{Expand}(\text{sd}_{i,\ell}) \right) \bmod p$$

⁸Recent work by Pasquini *et al.* [80] describes an attack where the server can send different models to different client updates with the goal that the model sent to a particular client can negate the training done by other clients on different models. In our case, this attack can be easily remedied with no overhead. Rather than evaluating the PRG with just the public matrix $\mathbf{A}$, one can first compute a hash $\mathbf{H}(\text{iteration}, \text{model})$ and multiply it with $\mathbf{A}$. If $\mathbf{H}$ was preimage-resistant, then the adversary cannot find a suitable model to force computation. As a result, the client's computation is tied to the model update sent. If different clients use different $\mathbf{A}$, the seed homomorphism fails. This would make it difficult for a malicious server to send different models to different clients to ensure that a particular client's contributions are not aggregated and, therefore, can be recovered. We can also switch to the key-homomorphic PRF constructions described in Section C.

Protocol Construction of OPA_{LWR}

One-Time System Parameters Generation

Run $\text{pp} \leftarrow \text{SS.Setup}(1^\kappa, 1^t, 1^r, 1^m)$
 Run $\text{pp}_{\text{PRG}} \leftarrow \text{PRG.Gen}(1^\kappa)$
 Set $\text{pp} = (\text{pp}_{\text{PRG}}, t, r, m)$
return Committee of size m and pp .

Data Encryption Phase by Client i in iteration ℓ

Sample $\text{sd}_{i,\ell} \leftarrow \text{PRG.K}$, $\text{dig}_{i,\ell} \leftarrow \text{PRG.Expand}(\text{sd}_{i,\ell})$
 Compute $(x_1, \dots, x_L) \leftarrow \text{Encode}(\mathbf{x}_i)$
 Compute $\text{mask}_{i,\ell} = \text{PRG.Expand}(\text{sd}_{i,\ell})$, $\text{mask}'_{i,\ell} := \text{H}(\text{dig}_{i,\ell})$
 Compute $\text{ct}_{i,\ell} = (x_1, \dots, x_L) + \text{mask}_{i,\ell} + \text{mask}'_{i,\ell}$
 $(\text{sd}_i^{(j)})_{j \in [m]} \leftarrow \text{SS.Share}(\text{sd}_{i,\ell}, t, r, m)$
 $(\text{dig}_{i,\ell}^{(j)})_{j \in [m]} \leftarrow \text{SS.Share}(\text{dig}_{i,\ell}, t, r, m)$
for $j = 1, \dots, m$ **do**
 $\text{aux}_{i,\ell}^{(j)} \leftarrow \text{sd}_i^{(j)}, \text{dig}_{i,\ell}^{(j)}$
 Send $\text{ct}_{i,\ell}$ to the Server
 Send $c_{i,\ell}^{(j)} = \mathcal{E}.\text{Enc}_{\text{pk}_j}(\ell, i, \text{aux}_{i,\ell}^{(j)})$ to committee member j for each $j \in [m]$, via server.

Set Intersection Phase by Server in iteration ℓ

For $j \in [m]$, let $\mathcal{C}^{(j)} := \{i : c_{i,\ell}^{(j)} \text{ was received by server}\}$
 Let $\mathcal{C}^{(0)} := \{i : \text{ct}_{i,\ell} \text{ was received by the server}\}$
 Compute $\mathcal{C} := \bigcap_{j \in S \cup \{0\}} \mathcal{C}^{(j)}$ // This is bit-wise AND operation of the $r+1$ bit strings.
assert $|\mathcal{C}| \geq (1-\delta)n$
 Send $\mathcal{C}, \{c_{i,\ell}^{(j)}\}_{i \in \mathcal{C}}$ for every j in $[m]$

Data Combination Phase by Committee Member j in iteration ℓ

Decrypt $(i, \ell, \{\text{aux}_{i,\ell}^{(j)} = (\text{sd}_{i,\ell}^{(j)}, \text{dig}_{i,\ell}^{(j)})\})$ from $\{c_{i,\ell}^{(j)}\}_{i \in \mathcal{C}}$ using sk_j
 Verify $i \in \mathcal{C}$ and ℓ is the current iteration, else abort.
 Compute $\text{AUX}^{(j)} \leftarrow \sum_{i \in \mathcal{C}} \text{sd}_{i,\ell}^{(j)}$
 Send $\text{AUX}^{(j)}, \{\text{dig}_{i,\ell}^{(j)}\}_{i \in \mathcal{C}}$ to server

Data Aggregation Phase by Server in iteration ℓ

Let $\{\text{AUX}_\ell^{(j)}\}_{j \in S}, \{\text{ct}_{i,\ell}\}_{i \in \mathcal{C}}$ be the inputs received by the server with $|\mathcal{S}| \geq r$.
 Run $\text{sd}_\ell \leftarrow \text{SS.Reconstruct}(\{\text{AUX}_\ell^{(j)}\}_{j \in S})$
 $\text{AUX}_\ell = \text{PRG.Expand}(\text{sd}_\ell)$
for $i \in \mathcal{C}$ **do**
 $\text{dig}_{i,\ell} \leftarrow \text{SS.Reconstruct}(\{\text{dig}_{i,\ell}^{(j)}\}_{j \in S})$
 Compute $\text{CT}_\ell \leftarrow \sum_{i \in \mathcal{C}} \text{ct}_{i,\ell}$
 Compute $(X_1, \dots, X_L) = \text{CT}_\ell - \text{AUX}_\ell - \sum_{i \in \mathcal{C}} \text{H}(\text{dig}_{i,\ell})$
 Compute $\mathbf{X}_\ell \leftarrow \text{Decode}(\text{pp}, (X_1, \dots, X_L))$
return $\mathbf{X}_\ell$

Figure 2: Our Construction of OPA built from $\text{PRG} = (\text{PRG.Gen}, \text{PRG.Expand})$ with key space $\mathcal{K} = \mathbb{Z}_q^\lambda$ (Construction 1) and the (t, r, m) -secret sharing scheme $\text{SS} = (\text{SS.Share}, \text{SS.Reconstruct})$ (Construction 5). Here, $\text{Encode}(\mathbf{x}_{i,\ell}) := \Delta \cdot \mathbf{x}_{i,\ell} + \mathbf{r}_i + 2^{\kappa_s}$ and $\text{Decode}(X_i) := \lceil X_i / \Delta \rceil - 1$ with $\Delta = 2^{\kappa_s} \cdot n$ and $\mathbf{r}_i \leftarrow \{0, \dots, 2^{\kappa_s} - 1\}^L$ for statistical security parameter κ_s and δ is the protocol's dropout parameter. The lines are for security against an active server. The second mask $\text{mask}'_{i,\ell}$, as the output of a H is for simulation proof, for an active server. We will model H as a programmable random oracle.

$$\begin{aligned}
 &= \Delta \cdot \sum_{i=1}^n x_{i,\ell} + \sum_{i=1}^n r_i + n + \sum_{i=1}^n \text{Expand}(\text{sd}_{i,\ell}) \bmod p \\
 &= \Delta \cdot \sum_{i=1}^n x_{i,\ell} + \sum_{i=1}^n r_i + n + \text{AUX}_\ell - e' \bmod p
 \end{aligned}$$

$$\sum_{i=1}^n \text{ct}_{i,\ell} - \text{AUX}_\ell = \Delta \cdot \sum_{i=1}^n x_{i,\ell} + \sum_{i=1}^n r_i + n - e' \bmod p$$

$$\bar{X}_\ell = \Delta \cdot \sum_{i=1}^n x_{i,\ell} + \sum_{i=1}^n r_i + n - e'$$

The jump from the second to the third step follows from Equation 2, and to make the last jump in the proof, we require:

$$0 \leq \Delta \cdot \sum_{i=1}^n x_{i,\ell} + \sum_{i=1}^n r_i + n - e' < p$$

First, $e' \leq n - 1$. This guarantees that: $0 \leq \Delta \cdot \sum_{i=1}^n x_{i,\ell} + \sum_{i=1}^n r_i + n - e'$. Now, if $\sum_{i=1}^n x_{i,\ell} < (p - \Delta)/\Delta$ then we also get:

$$\Delta \cdot \sum_{i=1}^n x_{i,\ell} + \sum_{i=1}^n r_i + n - e' < p$$

Now, we show the correctness of Decode algorithm to recover $\sum_{i=1}^n x_{i,\ell}$ from $\bar{X}_\ell$.

- $\bar{X}_\ell/\Delta = \sum x_{i,\ell} + (\sum_{i=1}^n r_i + n - e')/\Delta$
- $0 \leq \sum_{i=1}^n r_i < \Delta$ and $0 \leq e' \leq n - 1 \Rightarrow 1/\Delta \leq (\sum_{i=1}^n r_i + n - e')/\Delta \leq 1$
- Therefore, $\lceil \bar{X}_\ell/\Delta \rceil = \sum x_{i,\ell} + 1$

Theorem 3. Let δ, η be the dropout and corruption fraction among the universe of clients and let δ_C, η_C be the dropout and corruption fraction among the clients in committee. Let κ be the security parameter. Let N be the total universe of clients and n be the number of clients chosen for summation in each iteration while m be the number of committee clients chosen to help in each iteration. Let L be the length of the vector.

Let $\text{PRG} = (\text{PRG.Gen}, \text{PRG.Expand})$ be the leakage-resilient, seed-homomorphic PRG defined in Construction 1 and $\text{SS} = (\text{SS.Share}, \text{SS.Reconstruct})$ be the (t, r, m) -secret sharing scheme such that $r > (m + t)/2$ defined in Construction 5. Further, assuming a PKI (or authenticated channels) associated with an IND-CPA secure public key encryption scheme $\mathcal{E}$. Then, if $\delta_C + \eta_C < 1/3$, OPA_{LWR} securely realizes the functionality $F_{D,\delta}^\ell(X)$ (defined in Equation 1) with server malicious security with abort where $X = \{\mathbf{x}_{i,\ell}\}_{i \in [n] - \mathcal{K}}$ and $\mathcal{K} \subset [N]$ and $|\mathcal{K}| \cap [n] \leq \eta n$, under the LWR assumption.

Proof. We will prove the theorem statement by defining a simulator Sim , through a sequence of hybrids such that the view of the adversary $\mathcal{A}$ between any two subsequent hybrids are computationally indistinguishable. Let $H = [n] \setminus \mathcal{K}$, the set of honest clients. Further, let $\mathcal{C} = [n] \setminus D$ where D is the set of dropout clients.

It is important to note that the server is semi-honest. Therefore, it is expected to compute the set intersection of online clients $\mathcal{C}$, as expected. In other words, all committee members (and specifically the honest committee members) receive the same $\mathcal{C}$. This is an important contrast from active adversaries as a corrupt and active server could deviate from expected behavior and send different $\mathcal{C}^{(j)}$, for different committee members. This could help it glean some information about the honest clients.

We now sketch the proof below:

Hybrid₀(κ): This is the real execution of the protocol where the adversary $\mathcal{A}$ interacts with the honest parties.

Hybrid₁(κ): In this hybrid, we will rely on the security of the secret sharing scheme to do two things:

- On the one hand, all corrupt committee members receive a random share from the honest client's seed. Note that there can be only a maximum of t corrupt committee members. By appropriately choosing m , conditioned on η , we can guarantee that this holds with overwhelming probability. Then, for an honest client i , these are the shares denoted by $\{\text{sd}_i^{(j)}\}_{j \in [m] \cap \mathcal{K}}$ and are generated randomly.

- On the other hand, all the honest committee members receive a valid share of the honest client's seeds. However, each honest client i need to generate this from a polynomial $p(X)$ that satisfies $p(0) = \text{sd}_i$, while also ensuring $p(j) = \text{sd}_i^{(j)}$ for $j \in [\mathbf{m}] \cap \mathbf{K}$. Note that this is a polynomial time operation and is similar to the way packed secret sharing is done, where multiple secrets are embedded at distinct points of the polynomial. See Construction 5 for how to build such a polynomial.

It is clear that by relying on the privacy of the secret sharing scheme, $\text{Hybrid}_0, \text{Hybrid}_1$ are indistinguishable for the adversary.

Hybrid₂(κ): In this hybrid, we change the definition of the last honest party's ciphertext. WLOG, let n be the last honest party in $\mathcal{C}$. Then, we will set $\text{ct}_n := \text{PRG.Expand}(\text{sd}_\ell) + \mathbf{x}_\tau - \sum_{i \in \mathcal{C} \cap H} \text{ct}_i$. Here, $\mathbf{x}_\tau$ is the sum of the honest clients inputs. Note that we are still in the hybrid where Sim knows all the inputs.

It is clear that $\text{Hybrid}_1, \text{Hybrid}_2$ are identically distributed, by the almost seed-homomorphism property of PRG , provided Sim chooses the inputs for the honest parties such that they sum up to the value in $\mathbf{x}_\tau$.

Hybrid₃(κ): Again, without loss of generality, let client 1 be the first honest client in $\mathcal{C} \cap H$. We will modify the way $\text{ct}_{1,\ell}$ is generated. We will set it as $\text{ct}_{1,\ell} := \mathbf{x}_{1,\ell} + u$ where $u \leftarrow \text{PRG}(\mathcal{Y})$.

$\text{Hybrid}_2, \text{Hybrid}_3$ are indistinguishable, provided Theorem 2 holds. In the reduction, we will implicitly set $\text{sd}_1 + \text{sd}_n$ to be the leakage obtained from the Theorem 2's challenger. In this hybrid, Sim still knows all the inputs. If it was a real PRG output, then we can simulate Hybrid_2 , while simulating Hybrid_3 in the random case.

Hybrid₄(κ): In this hybrid, we will replace $\text{ct}_{1,\tau} := u'$ for $u' \leftarrow \text{PRG}(\mathcal{Y})$. It is clear that $\text{Hybrid}_3, \text{Hybrid}_4$ are identically distributed.

We have successfully replaced the first honest client's ciphertext with a uniformly random value independent of its input. Sim will continue to modify this for every non-dropout honest client $i \in \mathcal{C} \cap H$. This leaves the clients with all but the last honest clients' ciphertext to be independent of the input while leaving the previous honest client's ciphertext to be only a function of the sum of the inputs, which can be obtained by Sim 's query to the functionality. Sim beings its interaction with the functionality. After all the honest clients have provided inputs to the trusted party $\mathcal{T}$, in Step (b), Sim does not instruct any corrupted client to abort but rather set their inputs to be 0. Then in Step (c), Sim does not abort the server. Therefore, in Step (d), Sim will learn the sum of the honest parties inputs. Denote it as $\mathbf{x}_\ell$, which is also the sum of the inputs of the honest, surviving clients. With this information, Sim uses the last hybrid to interact with the adversary $\mathcal{A}$, who's expecting the real world interaction. This will enable Sim to run $\mathcal{A}$ internally. This is crucial to ensure that Sim can get the output of $\mathcal{A}$, in the real world, which might depend on its view (including the output) of the server. This view will, in turn, depend on the the honest clients' inputs. Since Sim sets the honest inputs, in this internal execution, to match the sum of inputs in the real world, we can guarantee that the output of $\mathcal{A}$ in the internal simulation is indistinguishable from $\mathcal{A}$'s interaction in the real world by the aforementioned hybrid arguments. $\square$

Our constructions so far have relied on providing security against a semi-honest server. Note that, as shown in the proof of security for Theorems 3, we can use the functionality query to obtain the sum of all the honest non-drop out clients, as before.

In the semi-honest setting, it is easy to see that the set $\mathcal{C}$, with respect to which aggregation is performed, includes *all* the honest, non-dropout clients' inputs. Therefore, querying the functionality, Sim does indeed get the sum of all the honest clients' inputs that are also included in the summation in the real world. This is imperative to ensure that Sim , when internally invoking $\mathcal{A}$, can get the output of $\mathcal{A}$ which should be indistinguishable from $\mathcal{A}$'s output in the real world. Specifically, this output of $\mathcal{A}$ (in either the internal invocation or the actual execution) will depend on the view which consists of the output of the server.

Therefore, if the output of the server in the real world does not include any of these honest clients' inputs, then the output produced by the internal invocation of $\mathcal{A}$ can be different from that in the real world.

Let us look at the case when the server is corrupted. Such a server can mount an attack whereby the real-world execution of the protocol may exclude inputs of some of those honest parties but actually included in the output of the ideal functionality. The proof of malicious security is tricky in this setting. Specifically, a malicious server can drop clients after seeing the honest input. This is an issue in the simulation as the simulator has to generate the masked inputs for the honest clients without knowing which of them would be dropped later.

Proof of Security against Active Server. Let us look at the setting when the server is corrupted. Then, we need to look at the information that the adversary $\mathcal{A}$ gleans from this, in the real world. As before, let K denote the corrupted clients. Then, $H_{\text{Cli}} := [n] \setminus K$ is the set of honest clients, $H_{\text{Com}} := [m] \setminus K$ is the set of honest committee members. Let $K_{\text{Cli}} := [n] \cap K$ denote the corrupted clients and $K_{\text{Com}} := [m] \cap K$ denotes the corrupted committee members.

- It receives every honest client's masked input, i.e., $\{\mathbf{ct}_i\}_{i \in H}$
- It receives every honest client's shares sent to the corrupted committee members, i.e., $\left\{ \mathbf{sd}_i^{(j)} \right\}_{i \in H_{\text{Cli}}, j \in K_{\text{Com}}}$
- It receives the combined shares sent by every honest committee member j , $\sum_{i \in H^*} \mathbf{sd}_i^{(j)}$ where H^* is the final set over which the aggregation occurs.

Remark 2 (Using Signatures). Figure 2, as previously described, does not incorporate the use of signatures. Nevertheless, the implementation of signatures is crucial to prevent a specific type of residual attack. Specifically, a malicious server can craft a message that encrypts 0 for all clients except one. Consequently, the committee inadvertently assists in reconstructing the information of the targeted client, which can then be exploited to reveal the inputs. This attack is successful due to the absence of a binding between the client's identity and the message it sends. It is important to note that this attack can succeed even when all clients are acting honestly. However, this issue can be effectively addressed by employing signatures, whereby each client signs each message. This solution presupposes the existence of a Public Key Infrastructure (PKI) setup for all clients, and aggregate signatures can be utilized to minimize communication overhead. Nonetheless, this requirement may be burdensome for certain applications, such as the one required in Willow [13]. If one chooses to forgo signatures, they must be prepared to endure this attack. While incorporating differential privacy, as suggested by Willow [13], can mitigate data leakage, it cannot fully prevent the attack, as it only offers differentially private guarantees for the clients' inputs.

Committee Speaks Once. Our construction currently has the server identify $\mathcal{C}^{(j)}$, for each $j \in [m]$, based on the information it has received from the client. Then, the server forwards the message to the committee member along with its computed intersection. This setting allows the server to selectively forward shares to committee member and also choose different sets for different committee members. We will show that if $r > (m + t)/2$ where r is the reconstruction threshold in the committee and t is the corruption threshold, then the server doing so will receive meaningless information. Formally, we will show that there does not exist two sets of users $\mathcal{C} \neq \mathcal{C}'$ such that the server can reconstruct the shares over these two sets.

Observe that the server controls t committee members. We require each honest committee member to participate once per iteration. This is easily enforced as the share from the honest client encrypts, along with the share for the honest committee member, the identity of the honest client and the iteration count. Therefore, a server cannot replay the same share, in another iteration. With this guarantee, a malicious server, in order to reconstruct the shares of two distinct sets $\mathcal{C}, \mathcal{C}'$, will require the cooperation of at least $r - t$ honest users, while there are $r - t$ honest users present. We will therefore need $2(r - t) > m - t$. Or, $r > (m + t)/2$. This ensures that the server can only effectively reconstruct with respect to a unique set $\mathcal{C}$ and H^* is the set of honest users in this set. Note that the above inequality holds for $r = 2 * m/3, t < m/3$.

Indeed, prior works such as Bonawitz *et al.* [16] and most recently LERNA [64] also tolerated only upto a $m/3$ corruption threshold.

While we have shown that there is a unique set H^* of honest users, H^* is only revealed after all the honest clients have sent their inputs. Therefore, the simulator, during its internal execution of $\mathcal{A}$, needs to be able to generate the masked inputs for the honest users and it only knows the sum of *all* the honest clients that have not dropped out. This set may be distinct from H^* . Therefore, we need a way for the simulator to generate masked inputs, independent of the sum of the inputs, and then ensure that the correct sum is computed during reconstruction.

The simulator does the following:

- For every honest client that hasn't dropped out, i.e., for all $i \in H_{\text{Cli}}$, the simulator does the following:
 - Samples $\text{dig}_{i,\ell} \leftarrow_{\$} \{0, 1\}^{\log q}$
 - Samples $\text{sd}_{i,\ell} \leftarrow_{\$} \text{PRG.K}$
 - It computes $\text{mask}'_{i,\ell} := H(\text{dig}_{i,\ell})$
 - Computes $\text{mask}_{i,\ell} = \text{PRG.Expand}(\text{sd}_{i,\ell})$
 - Sets $\text{ct}_{i,\ell} := \text{mask}_{i,\ell} + \text{mask}'_{i,\ell}$
 - Like shown in proof of Theorem 3, the shares of $\text{sd}_{i,\ell}^{(j)}, \text{dig}_{i,\ell}^{(j)}$ for corrupt committee members $j \in K_{\text{Com}}$ are chosen at random. Meanwhile, the shares for the honest committee members are to be sampled in the second phase, with a specific purpose. However, the server still expects an encryption of shares from honest client to honest committee members. Therefore, it simply encrypts some random shares for the honest committee members too and sends it to the server.
 - It sends to $\mathcal{A}$, $\text{ct}_{i,\ell}$ and $\text{sd}_{i,\ell}^{(j)}$ and $\text{dig}_{i,\ell}^{(j)}$ for $j \in [m]$, which is encrypted appropriately.
- This concludes the client phase of the operation. Then, comes the interaction with the committee. Note that the simulator is also required to simulate the honest committee member j .
- The simulator, which has received $\mathcal{C}^{(j)}$ for each honest committee member j does the check to make sure that there exists at least $r - t$ such committee members with the same $\mathcal{C}^{(j)}$. We will call this client set as $\mathcal{C}$, while calling the set of these committee members to be C_{good} . Meanwhile, it records those committee members with a different $\mathcal{C}^{(j)}$. We will call this as some set C_{bad} . Looking ahead, for those honest committee members in C_{bad} , the shares of the honest clients that are to be added up is going to be random values. Note that $|C_{\text{bad}}| \leq m - r$.
- The simulator now operates in two phases for honest committee member $j \in H_{\text{Com}}$. First is the share generation phase for honest clients i . It does the following:
 - If $j \in C_{\text{bad}}$, then for honest client $i \in \mathcal{C}^{(j)} \cap H_{\text{Cli}}$, set $\text{sd}_{i,\ell}^{(j)}, \text{dig}_{i,\ell}^{(j)}$ to be random values.
 - Now, the simulator computes the shares for all honest clients i to $j \in C_{\text{good}}$. These are valid shares of $\text{sd}_{i,\ell}, \text{dig}_{i,\ell}$ subject to the constraint that random values were fixed for those $j \in C_{\text{bad}}$ where $i \in \mathcal{C}^{(j)}$, and for those $j \in K_{\text{Com}}$.
 - The honest committee member j receives from $\mathcal{A}$, $\text{sd}_{i,\ell}^{(j)}$ and $\text{dig}_{i,\ell}^{(j)}$ for $i \in \mathcal{C}^{(j)}$. Note that the maximum number of prefixed values is $m - r + t$, and by our constraint $r > m - r + t$ which guarantees that these prefixed values cannot uniquely determine a polynomial of degree r .
- The second phase, is the combination phase. It responds, as expected, subject to the set $\mathcal{C}^{(j)}$ that it receives.
- Sim now queries the functionality. First, it provides $[n] \setminus \mathcal{C}$ as the set of dropped out clients. Then, it sends for those corrupted clients $K \cap \mathcal{C}$, input as 0 to the functionality. In response, it gets $\sum_{\mathcal{C} \cap H} \mathbf{x}_{i,\ell}$. Call this $\mathbf{x}_H$.

- Sim now picks $i^* \in H^*$. It programs the random oracle by setting $H(\text{dig}_{i^*,\ell}) = \mathbf{x}_H - \text{mask}'_{i^*,\ell}$.
- Sim continues to respond, on behalf of the honest committee member, as expected.
- Finally, $\mathcal{A}$ (which controls the server) will make queries to random oracle and it answers as expected. Sim outputs whatever $\mathcal{A}$ outputs at the end.

We will now need to show that the above simulation is indistinguishable from the real world execution that $\mathcal{A}$ expects when it is internally run. The hybrids proceeds as follows:

Hybrid₀(κ): This is the real world execution.

Hybrid₁(κ): In this execution, we replace the shares sent by the honest client i to honest committee member j , which are encrypted under pk_j with a random value. Under the semantic security of this encryption scheme, we can guarantee that this is indistinguishable from the previous hybrid. Meanwhile, these honest committee members (which the simulator controls) will receive the shares directly from the simulator. The view of $\mathcal{A}$, in this hybrid, is indistinguishable from the real world execution, under the semantic security of the encryption scheme.

Hybrid₂(κ): We will rely on the security of the secret sharing scheme to sample the shares for the honest clients, similar to Hybrid₁ of semi-honest security. For those $j \in K_{\text{Com}}$, the shares are randomly chosen. Furthermore, for those $j \in C_{\text{bad}}$ also the shares are randomly chosen. Finally, for those $j \in C_{\text{good}}$ it gets a valid share subject to those previously chosen random values. This is similar to Hybrid₁ in the proof of semi-honest security.

Hybrid₃(κ): In this hybrid, for all those honest clients i that are not in $\mathcal{C}$, we will set $\text{ct}_{i,\ell} = \text{mask}_{i,\ell} + \text{mask}'_{i,\ell}$, effectively setting the input to be 0. Observe that the view of $\mathcal{A}$ remains unchanged as these honest clients inputs were never incorporated in the final sum anyway. Furthermore, if any of these $i \in \mathcal{C}^{(j)}$ for $j \in C_{\text{bad}}$, the shares from these honest clients i to these j are completely random *and* independent of $\text{mask}_{i,\ell}$ and $\text{mask}'_{i,\ell}$.

Hybrid₄(κ): In this hybrid, we pick an honest surviving client $i^* \in H^*$. It sets the inputs for all $i \neq i^* \in H^*$ to be 0. Then sets $\mathbf{x}_{i^*,\ell}$ to be the sum of all the $i \in H^*$. Call this sum as $\mathbf{x}_H$. Observe that the values are still correlated and pseudorandom.

Hybrid₅(κ): In this hybrid, we will program $H(\text{dig}_{i^*,\ell}) = \mathbf{x}_H - \text{mask}'_{i^*,\ell}$, while setting $\text{ct}_{i^*,\ell} = \text{mask}_{i^*,\ell} + \text{mask}'_{i^*,\ell}$. Note that because $\text{dig}_{i^*,\ell}$ is chosen uniformly at random from q values where q is a large prime. The probability of collision is negligible. There is only negligible difference in the view of $\mathcal{A}$.

Hybrid₆(κ): In this hybrid, we will set $\text{ct}_{i,\ell}$ for $i \neq i^*$ to be some random term in the ciphertext space. Then, we will set $\text{ct}_{i^*,\ell} = H(\text{dig}_{i^*,\ell}) - \sum_{i \neq i^*} \text{ct}_{i,\ell}$.

Note that under the leakage resilience property of the seed-homomorphic PRG, we can conclude that the two hybrids are computationally indistinguishable.

Hybrid₇(κ): In this hybrid, we will replace $\text{ct}_{i,\ell} = \text{mask}_{i,\ell} + \text{mask}'_{i,\ell}$ for $i \neq i^*$.

Observe that this last hybrid is exactly what the simulator produces. This concludes the proof. $\square$

Achieving malicious security with abort is outlined in Section 8.

Sampling the Committee. Let C be a committee of size m . Specifically, to ensure $\eta_C + \delta_C < 1/3$, we need $\eta_C < 1/3 - \delta_C$. While the server controls δ_C , η_C is a random variable. Given the universe of N clients where η fraction of them are malicious, randomly sample m clients from them. Then, the number of malicious clients X in the committee should follow the tail bound of hypergeometric distribution (Definition 1).

$$\Pr[X \geq (\eta + (1/3 - \delta_C - \eta))m] \leq e^{-2 \cdot m(1/3 - \delta_C - \eta)^2} \leq 2^{-\gamma}$$

for some γ . Observe that $X/m = \eta_C$. Assuming $\eta = \delta_C = 0.01$, and committee of size 50, we get that $\Pr[\eta_C \geq 1/3 - \delta_C] \leq 5 \cdot 10^{-5}$. Extending it to the setting where we have a much larger pool of committee members, say $M \cdot m$, corresponding with M groups each of size m . Call the smaller groups $C_1, \dots, C_M$. We want to upper bound the probability that $\bigcup_{i=1}^M \Pr[\delta_{C_i} + \eta_{C_i} > 1/3]$. Using Union Bound, we get that $\bigcup_{i=1}^M \Pr[\delta_{C_i} + \eta_{C_i} > 1/3] \leq M \cdot 2^{-\gamma}$. Let $M = N/\log n$ (thereby assigning $\log n$ clients for each subcommittee of size m), then $\bigcup_{i=1}^M \Pr[\delta_{C_i} + \eta_{C_i} > 1/3] \leq 2^{\log n - \log \log n - \gamma}$.

7.2.2 Construction of OPA_{LWE}

Construction 3. As alluded to before, OPA_{LWE} is largely similar to OPA_{LWR} with the following differences:

- While the seed of PRG_{LWE} is $(\text{sd}, \mathbf{e})$, we will only secret share sd . We will argue below that the correctness still holds for a suitable definition of χ .
- The plaintext space for OPA_{LWE} , like the one for OPA_{LWR} , is $\mathbb{Z}_p$. Meanwhile the seed space for both OPA_{LWE} and OPA_{LWR} will be $\mathbb{Z}_q$. Let $\Delta := \lfloor q/p \rfloor$.
- We will use Shamir's Secret Sharing over q , as before, which is the seed space.
- There is a change in the server's last phase. To compute AUX_ℓ , the server uses the reconstructed seed sd_ℓ , and additionally sets the error component of the PRG seed to be 0.
- $\text{Encode}(\mathbf{x}_{i,\ell}) := \Delta \cdot \mathbf{x}_{i,\ell}$
- $\text{Decode}(X_i) := \lceil X_i/\Delta \rceil - 1$

Due to the similarities, we do not present the construction entirely. Meanwhile, we present the proof of correctness and proof of security.

Correctness. First, observe that Construction 1 is only almost seed-homomorphic, i.e.

$$\text{PRG}((\text{sd}_1 + \text{sd}_2, \mathbf{e}), \ell) = \text{PRG}((\text{sd}_1, \mathbf{e}_1), \ell) + \text{PRG}((\text{sd}_2, \mathbf{e}_2), \ell) + \mathbf{e}'$$

for some error $\mathbf{e}'$. Indeed, assuming the correctness of Shamir's Secret Sharing, we get that the server computes:

$$\text{AUX}_\ell := \text{PRG.Expand}\left(\left(\sum_{i=1}^n \text{sd}_i, 0\right), \ell\right) := \mathbf{A} \cdot \sum_{i=1}^n \text{sd}_i$$

Meanwhile,

$$\begin{aligned} \sum_{i=1}^n \text{ct}_{i,\ell} &= \sum_{i=1}^n (\mathbf{A} \text{sd}_i + \mathbf{e}_i + \Delta \cdot x_{i,\ell}) \\ &= \mathbf{A} \sum_{i=1}^n \text{sd}_i + \sum_{i=1}^n \mathbf{e}_i + \sum_{i=1}^n \Delta \cdot x_{i,\ell} \\ \text{Let } X_\ell &:= \sum_{i=1}^n \text{ct}_{i,\ell} - \text{AUX}_\ell = \sum_{i=1}^n \mathbf{e}_i + \sum_{i=1}^n \Delta \cdot x_{i,\ell} \\ \frac{X_\ell}{\Delta} &= \frac{\sum_{i=1}^n \mathbf{e}_i}{\Delta} + \sum_{i=1}^n x_{i,\ell} \end{aligned}$$

If $\sum_{i=1}^n \mathbf{e}_i < \frac{\Delta}{2}$, then $\lceil \frac{X_\ell}{\Delta} \rceil = \sum_{i=1}^n x_{i,\ell} + 1$. This shows the correctness of our algorithm.

Proof. The proof proceeds similar to that of Theorem 3, through a sequence of hybrids. However, there are a few differences. Construction 2 has the error vector $\mathbf{e} \leftarrow \chi$. However, we will replace $\mathbf{e} = \mathbf{e}' + \mathbf{f}'$ where $\mathbf{e}', \mathbf{f}' \leftarrow \chi'$, the distribution present in Hint-LWE Assumption (see Definition 4). The hybrid descriptions are similar, so we only specify the differences:

- In Hybrid_2 we will set:

$$\mathbf{ct}_{n,\ell} = \mathbf{A} \cdot \mathbf{sd}_\ell - \sum_{i \in (\mathcal{C} \cap H) \setminus \{n\}} \mathbf{ct}_i + \mathbf{e}_\ell + \Delta \mathbf{x}_\ell$$

- We will argue that $\text{Hybrid}_2, \text{Hybrid}_3$ are indistinguishable under Hint-LWE Assumption. We will sketch the reduction now.
 - Recall that, from the Hint-LWE Challenge, we get $(\mathbf{A}, \mathbf{u}^*, \mathbf{s}^* := \mathbf{s} + \mathbf{r}, \mathbf{e}^* := \mathbf{e}' + \mathbf{f}')$.
 - As done for the LWR construction, we will set the $\mathbf{s}_1 + \mathbf{s}_n = \mathbf{s}^*$, the leakage on key.
 - For generating $\mathbf{ct}_{1,\ell}$ we will use $\mathbf{u}^*$, while also sampling a separate $\mathbf{f}_1 \leftarrow \chi'$. This gives us: $\mathbf{ct}_{1,\ell} = \mathbf{u}^* + \mathbf{f}_{n,\ell} + \Delta \cdot \mathbf{x}_{1,\ell}$
 - We will set $\mathbf{ct}_{n,\ell} := \mathbf{A} \cdot \mathbf{sd}_\ell - \sum_{i \in (\mathcal{C} \cap H) \setminus \{n\}} \mathbf{ct}_i + \mathbf{e}^* + \sum_{i \in (\mathcal{C} \cap H) \setminus \{1\}} (\mathbf{e}'_{i,\ell} + \mathbf{f}'_{i,\ell}) + \Delta \cdot \mathbf{x}_\ell$
 - When $\mathbf{u}^*$ is the real sample, then $\mathbf{ct}_{1,\ell}$ satisfies Hybrid_2 's definition. Meanwhile, the $\mathbf{ct}_{n,\ell}$ is also correctly simulated. Similarly, the case when it is a random sample.

The proof of security against malicious servers also follows the previous theorem. $\square$

7.3 One-shot Private Aggregation without Leakage Simulation

OPA requires generating a new key in every iteration. We will now present a construction OPA' such that: (a) a client's keys can be reused across multiple iterations, and (b) the server does not get the sum of the keys but rather a function of pseudorandom values, which can be argued as itself being pseudorandom. Our core technique in this work is a distributed, key-homomorphic PRF. We formally present constructions from the CL framework in Section B. Specifically, we defer OPA_{CL} to the appendix in Section B.2. Similarly, we present LWR based construction in Section D. Next, we broadly describe the intuition behind our construction. It is important to emphasize that this is a purely theoretical construction as the committee's performance depends on L . However, we document this alternative approach for completeness.

A distributed-key-homomorphic PRF has three specific algorithms: **Eval**, which allows the evaluation of the PRF with key k_i at a point, **P-Eval** allows the PRF to be evaluated at a share of the key $k_i^{(j)}$ to get a partial evaluation, and **Combine** allows for the combination of partial evaluations to recover the actual evaluation. Key Homomorphism implies that both partial and actual evaluations are key homomorphic. Therefore, in our construction, the clients mask a vector of inputs $\mathbf{x}_{i,\ell}$ by computing a pseudorandom evaluation of $\text{DPRF.Eval}(k_{i,1}, \ell, \dots, \text{DPRF.Eval}(k_{i,L}, \ell))$. Meanwhile, the auxiliary information sent to the committee member will be $\text{P-Eval}(k_{i,k}^{(j)}, \ell)$ for $k = 1, \dots, L$ and $j \in [m]$. Then, the committee members combine the auxiliary information. The server then reconstructs on its end using **Combine**. Correctness follows from the key homomorphism. Note that the server only computes $\text{DPRF.Eval}(\sum_{i=1}^n k_{i,k}, \ell)$ for $k = 1, \dots, L$. This is a PRF evaluation; therefore, the leakage is pseudorandom and can be easily simulated, replacing it with random.

Finally, we also strengthen the security of OPA' , instantiated with the $\text{HSM}_{M=p}$ based construction (dubbed OPA_{CL}), by offering the privacy of honest user's inputs when all the committee are corrupt with each other while the server is honest. This is covered in Section G.

8 Malicious Security with Abort

We detail how to provide security against clients who behave maliciously. We detail how:

- a client proves that it has shared its key correctly. This is done so using SCRAPE Test [26] which minimizes the computation on the part of the client (unlike traditional verifiable secret sharing),
- a client proves that it has computed the masking correctly,

- (c) a client combines the previous two properties into one relation that is linear over $\mathbb{Z}_q$, and
- (d) a client proves that its masked input satisfies some particular constraints.

We present instantiations of the above using the proof system from Lyubashevsky *et al.* [70]. Importantly, OPA with bRSA (described in Section F.2) implies that the client's input is a binary vector. This shall simplify the zero-knowledge proof needed for input validation.

Proof of Correct Sharing. At its core, OPA (both the LWR and LWE based version) involves the use of a $\text{sd}_{i,\ell}$ that is secret-shared using Shamir's Secret Sharing. For simplicity, we will focus on the unpacked version where only one secret is shared. However, the proof extends to the packed setting as it only tests the existence of a polynomial. A standard approach to catch clients sending inconsistent shares would be for the client to rely on a verifiable secret sharing which would empower each committee member to verify if the share received by the committee member is consistent with the commitments to the polynomial that was sent by the client. However, this requires each committee client to perform $n \cdot r$ exponentiations which can be expensive where r is the reconstruction threshold. Instead, we take an alternative approach to verifying that the secret and the shares lie on the same polynomial, while ensuring that the bulk of the verification is done by the powerful server.

Our public verifiability will rely on a modification of SCRAPE [26]. SCRAPE test is done to check if $(\text{sd}_{i,\ell}^{(1)}, \dots, \text{sd}_{i,\ell}^{(m)})$ is a Shamir sharing over $\mathbb{Z}_q$ of degree $d = r - 1$ (namely there exists a polynomial p of degree $\leq d$ such that $p(i) = s_i$ for $i = 1, \dots, n$), one can sample $w^{(1)}, \dots, w^{(m)}$ uniformly from the dual code to the Reed-Solomon code formed by the evaluations of polynomials of degree $\leq d$, and check if $\sum_{i=1}^m w^{(i)} \cdot \text{sd}_{i,\ell}^{(i)} = 0$ in $\mathbb{Z}_q$. If the test passes, then $\text{sd}_{i,\ell}^{(1)}, \dots, \text{sd}_{i,\ell}^{(m)}$ are Shamir Shares, except with probability $1/|\mathbb{Z}_q|$. Recall that q is a prime number.

Lemma 4 (SCRAPE Test [27]). *Let $\mathbb{Z}_q$ be a finite field and let $d = r - 1, m$ be parameters of the Shamir's Secret Sharing scheme such that $0 \leq d \leq m - 2$, and inputs $\text{sd}_{i,\ell}^{(1)}, \dots, \text{sd}_{i,\ell}^{(m)} \in \mathbb{Z}_q$. Define $v_i := \prod_{j \in [m] \setminus \{i\}} (i - j)^{-1}$ and let $m^*(X) := \sum_{i=0}^{m-d-2} m_i \cdot X^i \leftarrow_{\$} \mathbb{Z}_q[X]_{\leq m-d-2}$ (i.e., a random polynomial over the field of degree at most $m - d - 2$). Now, let $\mathbf{w} := (v_1 \cdot m^*(1), \dots, v_n \cdot m^*(m))$ and $\mathbf{s} := (\text{sd}_{i,\ell}^{(1)}, \dots, \text{sd}_{i,\ell}^{(m)})$. Then,*

- *If there exists $p \in \mathbb{Z}_q[X]_{\leq d}$ such that $\text{sd}_{i,\ell}^{(i)} = p(i)$ for all $i \in [n]$, then $\langle \mathbf{w}, \mathbf{s} \rangle = 0$.*
- *Otherwise, $\Pr[\langle \mathbf{w}, \mathbf{s} \rangle = 0] = 1/|\mathbb{Z}_q|$.*

Typically, we compute the polynomial $m^*(X)$ by using the Fiat-Shamir transform over public values. Then, the vector $\mathbf{w}$ is a public vector. One simply has to hide the vector $\mathbf{s}$. As a result, we have proved that the shares do lie on the same polynomial. Note that in standard Shamir's Secret Sharing, we set $p(0) = \text{sd}_{i,\ell}$, i.e., the secret. Therefore, we will have to perform inner product over a vector of length $m + 1$.

Thus, we get the following relation for the proof of correct-sharing:

$$\mathcal{R}_{\text{Sharing}} := \{ (\mathbf{w}; \mathbf{s}) \mid \langle \mathbf{w}, \mathbf{s} \rangle = 0, \mathbf{w} \in \mathbb{Z}_q^{m+1}, \mathbf{s} \in \mathbb{Z}_q^{m+1} \}$$

While we will employ a commit-and-prove paradigm, observe that the server can only verify if the commitment satisfies the required proof. A malicious client can send a completely arbitrary share to the committee member, under the hood of decryption. To solve this problem, the server will forward the commitment to the share it has received. This will allow the committee member to receive if the share (and necessary opening information) received by the committee member opens the commitment forwarded by the server. If it fails, the committee member complains and the protocol aborts. This guarantees malicious security with abort.

Proof of Correct Masking. $\mathcal{R}_{\text{Sharing}}$ guarantees that the shares are consistent. In contrast, the commitment opening by the committee member guarantees that the share received matches the share committed to; it is

important to ensure that the correct seed is used in the computation of the seed-homomorphic PRG. For example, in OPA_{LWR} (for $L = 1$), client i computes

$$\mathbf{ct}_{i,\ell} = \mathbf{A} \cdot \mathbf{sd}_{i,\ell} + \mathbf{e}_{i,\ell} + \Delta \cdot \mathbf{x}_{i,\ell}$$

Here, $\mathbf{A}$ is public while $\mathbf{sd}_{i,\ell} \leftarrow \mathbb{Z}_q^\lambda, \mathbf{e} \leftarrow \chi^L$ are secret along with $\mathbf{x}_{i,\ell} \in \mathbb{Z}_p^L$. Therefore, we get:

$$\mathcal{R}_{\text{Masking}} := \left\{ (\mathbf{ct}_{i,\ell}, \mathbf{A}; \mathbf{sd}_{i,\ell}, \mathbf{e}_{i,\ell}, \mathbf{x}_{i,\ell}) \mid \begin{array}{l} \mathbf{ct}_{i,\ell} = \mathbf{A} \cdot \mathbf{sd}_{i,\ell} + \mathbf{e}_{i,\ell} + \Delta \cdot \mathbf{x}_{i,\ell}, \\ \mathbf{ct}_{i,\ell} \in \mathbb{Z}_q^L, \mathbf{sd}_{i,\ell} \in \mathbb{Z}_q^\lambda, \mathbf{e} \in \chi^L, \mathbf{x}_{i,\ell} \in \mathbb{Z}_p^L \end{array} \right\}$$

Note that $\mathbf{e} \in \chi^L$ is typically proved by showing that its L_2 (or L_∞) norm is bounded.

Proof of Linear Relations over $\mathbb{Z}_q$. It is critical to observe that the relations $\mathcal{R}_{\text{Sharing}}$ and $\mathcal{R}_{\text{Masking}}$ are both linear relations over $\mathbb{Z}_q$ (except showing that $\mathbf{e}$ is “small”, i.e., $\|\mathbf{e}\|_2 \leq \beta_e$ for some parameter β). We will combine to get the following relation:

$$\mathcal{R}_{\text{OPA}} := \left\{ \left(\mathbf{w}, \mathbf{A}, \mathbf{ct}_{i,\ell}; \mathbf{sd}_{i,\ell}, \left\{ \mathbf{sd}_{i,\ell}^{(j)} \right\}, \mathbf{e}_{i,\ell}, \mathbf{x}_{i,\ell} \right) \mid \begin{array}{l} \mathbf{ct}_{i,\ell} = \mathbf{A} \cdot \mathbf{sd}_{i,\ell} + \mathbf{e}_{i,\ell} + \Delta \cdot \mathbf{x}_{i,\ell} \\ \left\langle \mathbf{w}, \left(\mathbf{sd}_{i,\ell}, \mathbf{sd}_{i,\ell}^{(1)}, \dots, \mathbf{sd}_{i,\ell}^{(m)} \right) \right\rangle = 0 \\ \|\mathbf{e}_{i,\ell}\|_2 \leq \beta_e, \mathbf{A} \in \mathbb{Z}_q^{L \times \lambda}, \mathbf{sd}_{i,\ell} \in \mathbb{Z}_q^\lambda, \\ \mathbf{ct}_{i,\ell} \in \mathbb{Z}_p^L \end{array} \right\}$$

One can employ the techniques of Lyubashevsky *et al.* [70, Figure 5] to prove the linear relations between the various secrets. An alternative approach for $\mathcal{R}_{\text{Sharing}}$ is described in Figure 11. Then, one can use Lyubashevsky *et al.* [70, §4] to prove the L_2 norm on $\mathbf{e}$.

Input Validation. A malicious client can provide incorrect or malicious inputs under encryption. Therefore, adding a validation restriction on the input is also essential. Typically, this is done by proving that the L_2, L_∞ norm is bounded. Therefore, it is clear that one can do techniques similar to bounding $\mathbf{e}$, to prove that $\|\mathbf{x}_{i,\ell}\|$ also satisfies some prior bounds. However, looking ahead, in Section F, we describe how to combine bRSA with OPA, where each client’s input to server is a vector of 0s and 1s. This proof technique is much simpler. Specifically, when combining OPA with bRSA [65] as described in Section F, we require clients only to prove that their input is a binary vector.

8.1 Heterogeneity and Poisoning Attacks

In Section F, we present two approaches, FedOpt [86] and Byzantine-Robust Stochastic Aggregation [65]. The former is designed to handle heterogeneity in data distribution, while the latter is designed to handle poisoning attacks where some clients behave arbitrarily. We further describe how to combine OPA with these two approaches, bringing these techniques to the much-needed privacy-preserving alternatives.

9 Experiments

In this section, we benchmark OPA_{LWR} , the construction based on leakage resilient seed-homomorphic PRG, combined with a secret-sharing scheme. We run our experiments on an Apple M1 Pro CPU with 16 GB of unified memory without multi-threading or related parallelization. We use the ABIDES simulation [24] to simulate real-world network connections. ABIDES supports a latency model, represented as a base delay and jitter, which controls the number of messages arriving within a specified time. Our base delay is set with the range from 21 microseconds to 100 microseconds), and we use the default parameters for the jitter. This delay is set to correspond to devices locally situated. This framework was used to measure the performance of other prior work, including [73, 54]. More details on the framework can be found in [54, §G].

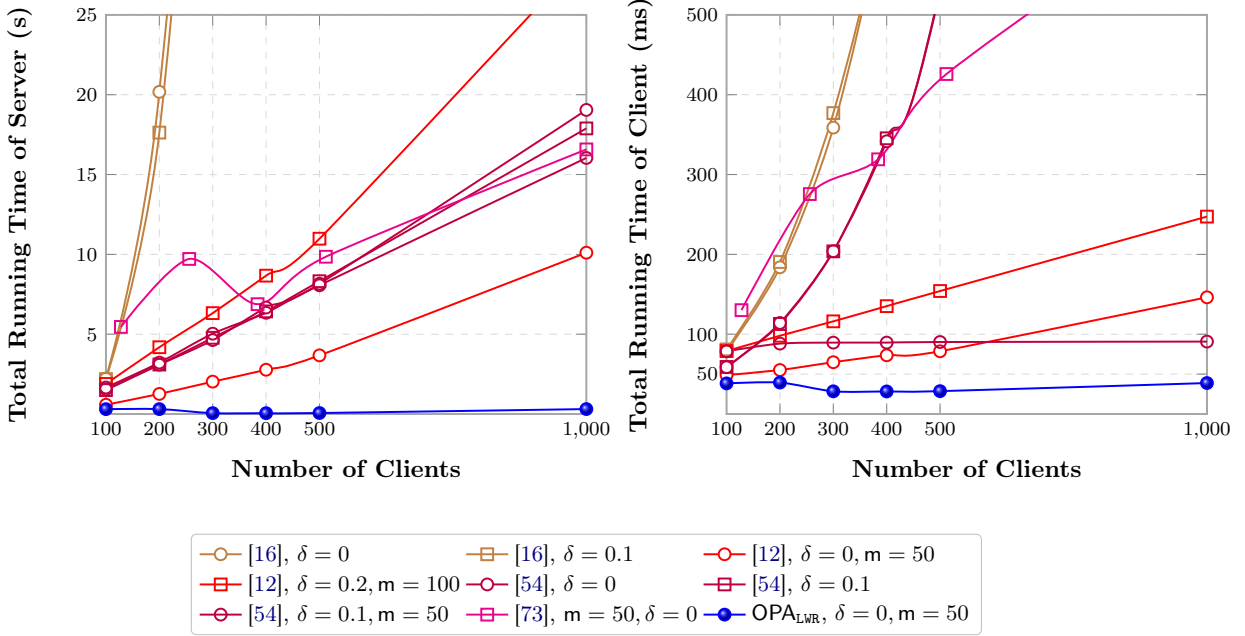


Figure 3: We plot the total client and server running time as a function of client count across various protocols. Our running time includes both computation and communication time. Here, δ indicates the offline rate and m is used to parametrize the number of parties each client has to communicate with (i.e., number of neighbors or committee size). In all our experiments on MicroSecAgg [54], the clients’ maximum input is 10^4 . The running time of the committee members are added to the Client’s running time in OPA_{LWR} .

Parameter Choices for OPA_{LWR} . OPA_{LWR} is parametrized by λ, L, q, p (See Definition 2). Given an LWR instance $(\mathbf{A}, \mathbf{t} = \lfloor \frac{p}{q} \cdot \mathbf{A}\mathbf{s} \rfloor) \in \mathbb{Z}_q^{L \times \lambda} \times \mathbb{Z}_p^L$, one can convert it to an LWE instance as: $(\mathbf{A}, q/p \cdot \mathbf{t} = \mathbf{A}\mathbf{s} + \mathbf{e})$, where $\mathbf{e} = q/p \cdot \mathbf{e}'$ with $\mathbf{e}' = \mathbf{t} - p/q \mathbf{A} \cdot \mathbf{s}$. Now, $\mathbf{e}'$ is uniformly distributed over $(-\frac{1}{2}, \frac{1}{2})$. Thus, we get that the variance of the error in the LWE samples ($\mathbf{e}$) is $\frac{q^2}{12p^2}$. This analysis is similar to the work of Okada *et al.* [78, §5.5]. In other words, the Gaussian Error distribution with standard deviation $\alpha \cdot q$ has error rate $\alpha \approx 1/p$ (as used by [46]). Using the LWE estimator, we set $\lambda := 2048$. q to match the field used for Shamir’s Secret Sharing, which is a 128-bit prime, and set $p = 2^{53}$. The hardness estimated is 2^{129} , i.e., we get the security of 129 bits.

We use Packed Secret Sharing to benchmark the server and client computation cost. One can pack ρ secrets into a single polynomial using packed secret sharing. However, an implicit trade-off exists as the total number of parties $m \geq 3/2 \cdot \rho$. An increase in ρ implies an increase in m . Here, we set $\rho = 16$, corresponding to requiring $\lambda/16$ polynomials, i.e., each committee member receives 64 shares. Consequently, an $m = 50$ satisfies the requirement. We then set the reconstruction threshold $r = 34$, while Packed Secret Sharing guarantees a tolerance for a corruption threshold of $m - \rho$. However, we need to ensure this among the committee members, even in the face of dropouts. Specifically, if δ_C, η_C are dropout and corruption rates among the committee members, we need to compute the $\Pr[\delta_C + \eta_C > 1/3]$. Using Hypergeometric Distribution (Definition 1) and setting $\delta = 0.01$, we get that $m = 50$ ensures $\Pr[\delta_C + \eta_C > 1/3] \leq 5 \cdot 10^{-5}$. Note that this construction achieves committee performance independent of the vector length and is preferred. This also satisfies the constraint for reconstruction with error correction which requires that $2m \cdot \eta_C < (1 - \delta_C)m - \rho + 1$.

Microbenchmarking Secure Aggregation. Our first series of experiments is to run OPA_{LWR} to build a secure aggregation protocol for $L = 1$. We also compare with existing work, including [16, 12, 54, 73]⁹. We vary the offline rates (δ) and the ability to group clients (m), along with increasing the number of clients to study the performance of related work. It is important to note that OPA’s one-shot communication (client and committee member) implies that as δ increases, the client’s performance remains the same while the server and the committee’s performance improves as fewer clients participate in the aggregation. The results are plotted in Figure 3.

Performance of OPA_{LWR} . The running time performance of OPA_{LWR} is notably superior to all other protocols tested across server and client computation times.

- **Server Total Time:** At 1000 clients, OPA_{LWR} shows the most significant improvement, with a server computation time of just 0.31 seconds, drastically outperforming other protocols. For example, [16] with $\delta = 0$ takes approximately 71.3 seconds, while [12] with $\delta = 0$ and $m = 50$ reaches 10.1 seconds. Even protocols from [54] with various settings, such as $\delta = 0$ and $\delta = 0.1$, take between 16 and 19 seconds. These results make OPA_{LWR} an outstanding choice for minimizing server computation time.
- **Client Total Time:** Similarly, for client computation time at 1000 clients, OPA_{LWR} again stands out with only 38.8 milliseconds, which is significantly faster than the other protocols. For instance, the protocol in [16] with $\delta = 0$ reaches over 5700 milliseconds, while [12] with $\delta = 0$, $m = 50$ results in 146.2 milliseconds. In contrast, [54], even at lower offline rates ($\delta = 0$ and $\delta = 0.1$), shows client computation times exceeding 2000 milliseconds, making OPA_{LWR} ’s performance at 1000 clients a clear advantage.

The remarkable speed of OPA_{LWR} in server and client performance, especially as client count grows, emphasizes its efficiency and scalability in real-world applications. It significantly reduces the computational burden and communication overhead compared to existing protocols, making it an attractive solution for privacy-preserving aggregation tasks.

Communication Cost of OPA_{LWR} . Let k be the number of elements being shared. With naive secret sharing, this would be 1024. Instead, when we pack into 64 different polynomials, we get $k = 64$. Thus, OPA_{LWR} has a communication cost, in terms of field elements, as follows:

- Total Sent/Received per Client: $L + k \cdot m / \perp$ field elements
- Total Sent/Received by Server: $nL + m \cdot n \cdot k / n + k$ field elements
- Total Sent/Received per Committee Member: $\log n \cdot k + k / \log n + k$ field elements,

9.1 Computation Time

We also micro-benchmark in the same environment and setting as before to measure the cost of server and client computation alone. This is shown in Figure 4. This does not account for the time of communication. As can be seen, OPA_{LWR} still significantly outperforms prior work.

9.2 Running Time vs length of vector L

We also benchmark the performance of the protocols, with dropout rate $\delta = 0$ for $L = 1000$. This is covered in Figure 5. The server performance of OPA_{LWR} significantly outperforms existing protocol. Meanwhile, we observe that Flamingo has a better client performance for smaller choices of client count, while OPA_{LWR} begins performing better for a larger number of clients. It is noted that OPA_{LWR} ’s unique one-shot design implies that the client performance is independent of the number of clients. Owing to the number of exponentiation that is proportional to L , MicroSecAgg [54] performs degrades with L , and our benchmarks indicated that the performance bounds were outside the range of this graph and are omitted from our plot.

⁹We do not benchmark LERNA [64] given that it is not suitable for the FL setting.

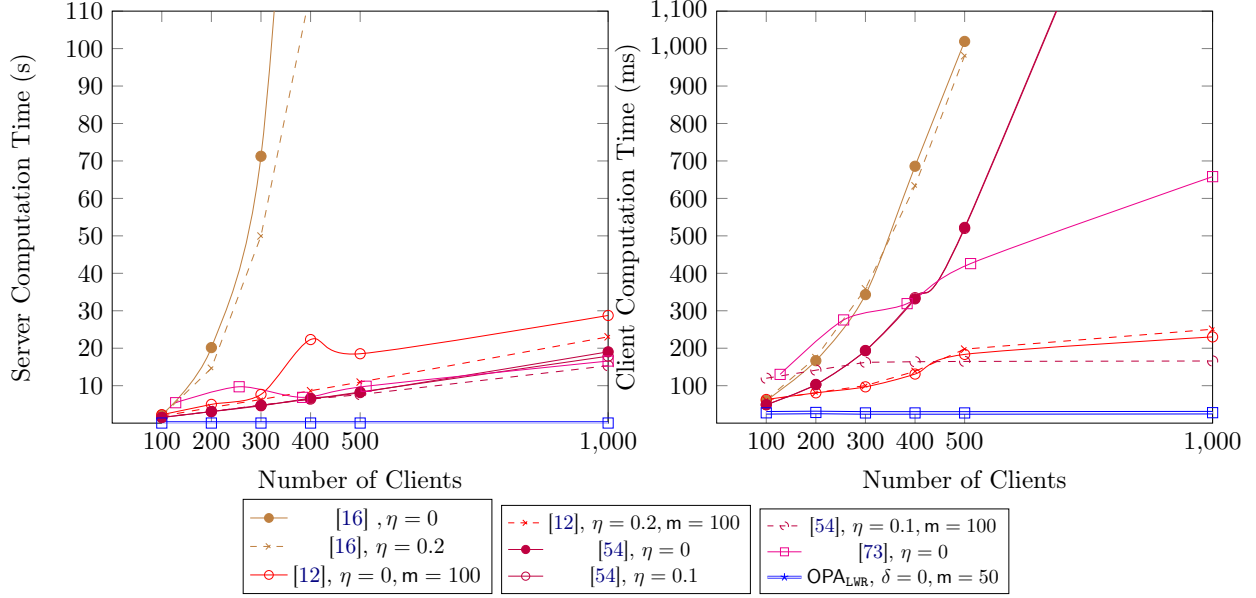


Figure 4: Client and Server Computation Time as a function of client count across different algorithms.

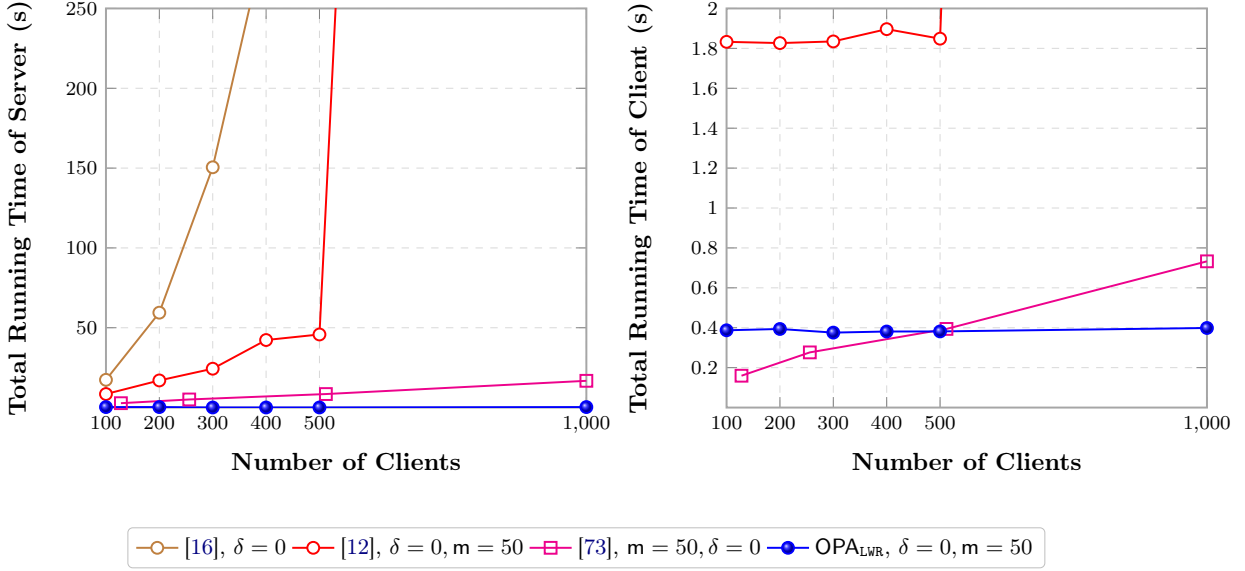


Figure 5: We plot the total client and server running time as a function of client count across various protocols. Our running time includes both computation and communication time. Here, $\delta = 0$ indicates the offline rate and m is used to parametrize the number of parties each client has to communicate with (i.e., number of neighbors or committee size) and $L = 1000$. In all our experiments on MicroSecAgg [54], the clients' maximum input is 10^4 . The running time of the committee members are added to the Client's running time in OPA_{LWR} .

9.3 Performance of OPA_{CL}

OPA' is an alternate framework for instantiating One-shot Private Aggregation from threshold key-homomorphic PRF unlocking constructions based on additional assumptions, including class groups. While these contribu-

Table 3: Comparison of Computation Times for Server and Client between OPA_{CL} and OPA_{LWR} for $L = 1$.

Clients	Server Computation (s)		Client Computation (s)	
	OPA_{LWR}	OPA_{CL}	OPA_{LWR}	OPA_{CL}
100	0.0262	1.9347	0.0276	1.6579
200	0.0268	1.9052	0.0286	1.6622
300	0.0260	1.9036	0.0273	1.6526
400	0.0266	1.9067	0.0273	1.6333
500	0.0267	1.9201	0.0273	1.6420
1000	0.0292	1.9310	0.0280	1.6350

tions are practically more expensive, we include benchmarks for completeness. We use the Threshold PRF based on Class Groups (See Section 10). We rely on the BICYCL library [19] and use pybind to convert the C++ code to Python. Our implementation will assume that the plaintext space is $\mathbb{Z}_p$ for a prime p . Our experiments will take that $m = 50$.

It is to be noted that the client’s performance scales significantly, owing to 50 group exponentiations. However, the server performance is less than 2 seconds, outperforming several existing protocols’ computations.

9.4 Performance in Federated Learning Use-case

We discuss the accuracy experiments conducted on various classifiers and datasets and plot the performance of OPA_{LWR} with respect to learning in the clear (i.e., without cryptographic protections) in Figures 7,6. Our results show that the accuracy of OPA_{LWR} is statistically indistinguishable from that of the classifier when learning in the clear.

- **Adult Census Dataset:** We first run experiments on the adult census income dataset from [25, 57] to predict if an individual earns over \$50,000 per year. The preprocessed dataset has 105 features and 45,222 records with a 25% positive class. We randomly split into training and testing, with further splitting by the clients. First, we train in the clear with weights sent to the server to aggregate. With 100 clients and 50 iterations, we achieve 82.85% accuracy and 0.51 MCC. We repeat with OPA_{LWR} with 100 clients and 50 committee members. With 10 iterations, we achieve 82.38% accuracy and 0.48 MCC. With 20 iterations, we achieve 82% accuracy and 0.51 MCC. Our quantization technique divides weights into integer and decimal parts (2 integer and 8 decimal values per weight). Training with 50 clients takes under 1 minute per client per iteration with no accuracy loss. This quantization yields a vector size of 1050 (10 per feature).
- We use the Kaggle Credit Card Fraud dataset [82], comprising 26 transformed principal components and amount and time features. We omit time and use the raw amount, adding an intercept. The goal is to predict if a transaction was indeed fraudulent or not. There are 30 features and 284,807 rows, with <0.2% fraudulent. Weights are multiplied by 10,000 and rounded to an integer, accounted for in aggregation. Figure 7 shows OPA_{LWR} ’s MCC versus clear learning for varying clients and iterations. With the accuracy multiplier, OPA_{LWR} ’s MCC is close to clear learning and sometimes outperforms. The highly unbalanced dataset demonstrates OPA_{LWR} can achieve substantial performance even in challenging real-world scenarios.
- We then train a vanilla multi-layer perceptron (MLP) classifier on three datasets: MNIST, CIFAR-10, and CIFAR-100. We quantize the weights by multiplying with 2^{16} . The MLP accuracy, as a function of the iteration count, is plotted in Figures 6,7. Our experiments demonstrate that OPA_{LWR} preserves accuracy while ensuring the privacy of client data. Note that vanilla MLP classifiers do not typically offer good performance for CIFAR datasets, but note that our experiments aimed to show that OPA_{LWR} does not impact accuracy.

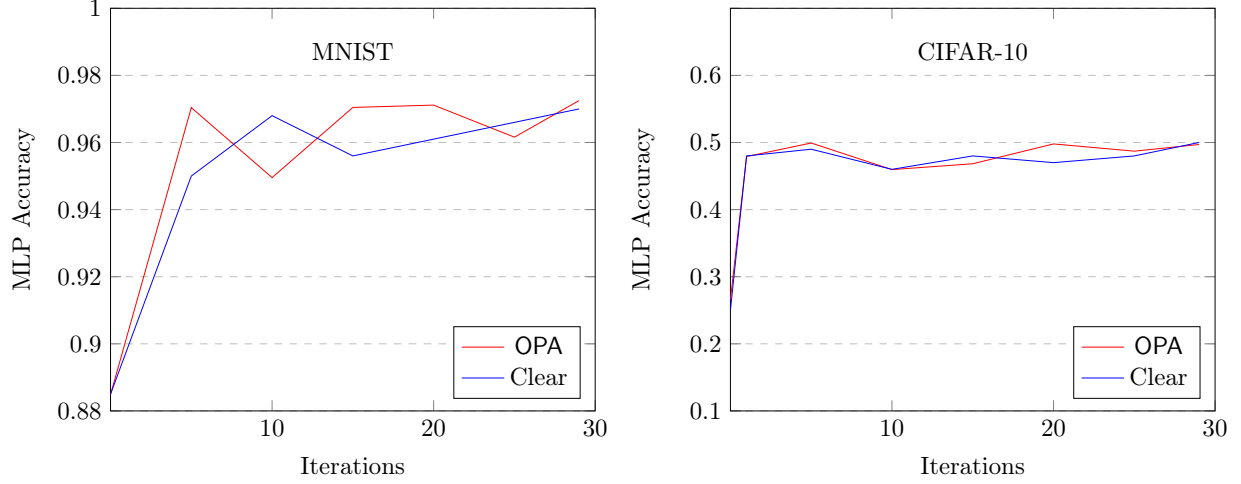


Figure 6: MLP Accuracy for different datasets: MNIST and CIFAR-10. We use $n = 100$ clients in this training, and $m = 50$.

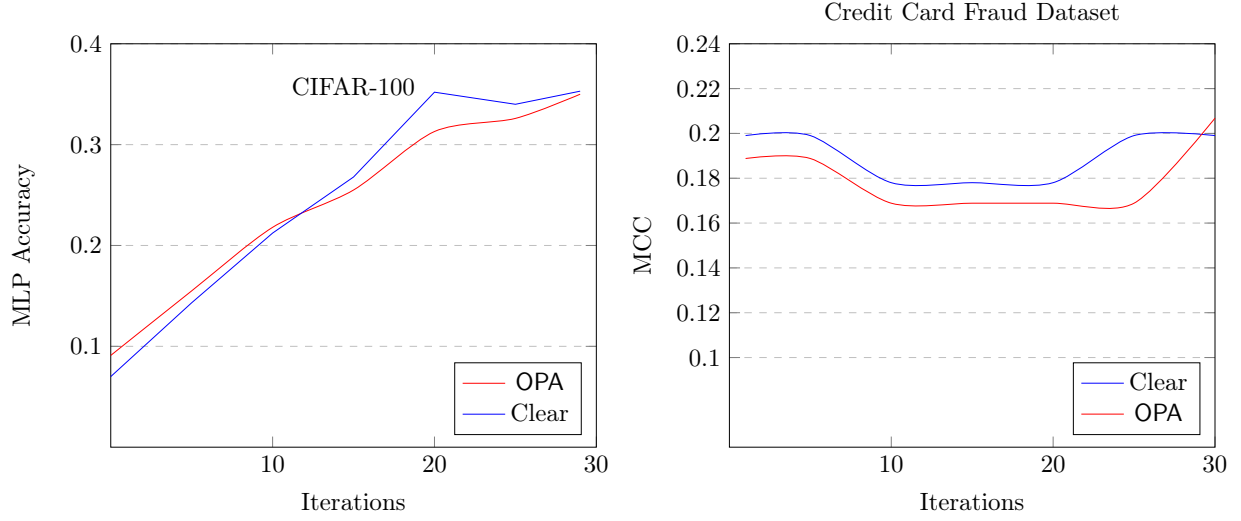


Figure 7: MLP Accuracy for CIFAR-100. We use $n = 100$ clients in this training, and $m = 50$. In the last figure, we have the MCC score as a function of the number of iterations for the credit card fraud dataset.

10 Future Work

OPA offers malicious security with abort, yet achieving guaranteed output delivery (GoD) or robustness while ensuring that clients communicate only once remains an unresolved challenge. The first work addressing GoD is presented in [11], which necessitates multiple rounds of interaction, scaling logarithmically with the number of clients in the worst-case scenario, with a minimum of 6 rounds. The recent advancement by [72] enhances the approach of [11] by introducing a more efficient 3-round protocol.

While we present lattice-based instantiations to achieve security in the face of malicious clients, we leave it as future work to implement and optimize the performance of these constructions. Another direction of research is to balance lattice-based zero-knowledge proofs with techniques from Bulletproof to achieve optimized performance.

Disclaimer. This paper was prepared for informational purposes by the Artificial Intelligence Research group of JPMorgan Chase & Co and its affiliates (“J.P. Morgan”) and is not a product of the Research Department of J.P. Morgan. J.P. Morgan makes no representation and warranty whatsoever and disclaims all liability for the completeness, accuracy or reliability of the information contained herein. This document is not intended as investment research or investment advice, or a recommendation, offer or solicitation for the purchase or sale of any security, financial instrument, financial product or service, or to be used in any way for evaluating the merits of participating in any transaction, and shall not constitute a solicitation under any jurisdiction or to any person, if such solicitation would be unlawful.

References

- [1] Damiano Abram, Ivan Damgård, Claudio Orlandi, and Peter Scholl. An algebraic framework for silent preprocessing with trustless setup and active security. In Yevgeniy Dodis and Thomas Shrimpton, editors, *Advances in Cryptology – CRYPTO 2022, Part IV*, volume 13510 of *Lecture Notes in Computer Science*, pages 421–452. Springer, Cham, August 2022.
- [2] Anasuya Acharya, Carmit Hazay, Vladimir Kolesnikov, and Manoj Prabhakaran. SCALES - MPC with small clients and larger ephemeral servers. In Eike Kiltz and Vinod Vaikuntanathan, editors, *Theory of Cryptography - 20th International Conference, TCC 2022, Chicago, IL, USA, November 7-10, 2022, Proceedings, Part II*, volume 13748 of *Lecture Notes in Computer Science*, pages 502–531. Springer, 2022.
- [3] Surya Addanki, Kevin Garbe, Eli Jaffe, Rafail Ostrovsky, and Antigoni Polychroniadou. Prio+: Privacy preserving aggregate statistics via boolean shares. In Clemente Galdi and Stanislaw Jarecki, editors, *Security and Cryptography for Networks - 13th International Conference, SCN 2022, Amalfi, Italy, September 12-14, 2022, Proceedings*, volume 13409 of *Lecture Notes in Computer Science*, pages 516–539. Springer, 2022.
- [4] Shweta Agrawal, Xavier Boyen, Vinod Vaikuntanathan, Panagiotis Voulgaris, and Hoeteck Wee. Functional encryption for threshold functions (or fuzzy ibe) from lattices. In Marc Fischlin, Johannes Buchmann, and Mark Manulis, editors, *PKC 2012: 15th International Conference on Theory and Practice of Public Key Cryptography*, volume 7293 of *Lecture Notes in Computer Science*, pages 280–297. Springer, Berlin, Heidelberg, May 2012.
- [5] Apple and Google. Exposure notification privacy-preserving analytics (ENPA), 2021.
- [6] Arasu Arun, Chaya Ganesh, Satya V. Lokam, Tushar Mopuri, and Sriram Sridhar. Dew: A transparent constant-sized polynomial commitment scheme. In Alexandra Boldyreva and Vladimir Kolesnikov, editors, *PKC 2023: 26th International Conference on Theory and Practice of Public Key Cryptography, Part II*, volume 13941 of *Lecture Notes in Computer Science*, pages 542–571. Springer, Cham, May 2023.
- [7] Thomas Attema, Ignacio Cascudo, Ronald Cramer, Ivan Bjerre Damgård, and Daniel Escudero. Vector commitments over rings and compressed σ -protocols. Cryptology ePrint Archive, Report 2022/181, 2022.
- [8] Abhishek Banerjee, Chris Peikert, and Alon Rosen. Pseudorandom functions and lattices. In David Pointcheval and Thomas Johansson, editors, *Advances in Cryptology – EUROCRYPT 2012*, volume 7237 of *Lecture Notes in Computer Science*, pages 719–737. Springer, Berlin, Heidelberg, April 2012.
- [9] Laasya Bangalore, Mohammad Hossein Faghihi Sereshgi, Carmit Hazay, and Muthuramakrishnan Venkitasubramaniam. Flag: A framework for lightweight robust secure aggregation. In Joseph K. Liu, Yang Xiang, Surya Nepal, and Gene Tsudik, editors, *ASIACCS 23: 18th ACM Symposium on Information, Computer and Communications Security*, pages 14–28. ACM Press, July 2023.

- [10] Daniela Becker, Jorge Guajardo, and Karl-Heinz Zimmermann. Revisiting private stream aggregation: Lattice-based PSA. In *ISOC Network and Distributed System Security Symposium – NDSS 2018*. The Internet Society, February 2018.
- [11] James Bell, Adrià Gascón, Tancrede Lepoint, Baiyu Li, Sarah Meiklejohn, Mariana Raykova, and Cathie Yun. ACORN: Input validation for secure aggregation. In *32nd USENIX Security Symposium (USENIX Security 23)*, pages 4805–4822, Anaheim, CA, August 2023. USENIX Association.
- [12] James Henry Bell, Kallista A Bonawitz, Adrià Gascón, Tancrede Lepoint, and Mariana Raykova. Secure single-server aggregation with (poly) logarithmic overhead. In *Proceedings of the 2020 ACM SIGSAC Conference on Computer and Communications Security*, pages 1253–1269, 2020.
- [13] James Bell-Clark, Adrià Gascón, Baiyu Li, Mariana Raykova, and Phillipp Schoppmann. Willow: Secure aggregation with one-shot clients. In *Advances in Cryptology – CRYPTO 2025*, 2025. To appear.
- [14] Fabrice Benhamouda, Marc Joye, and Benoît Libert. A new framework for privacy-preserving aggregation of time-series data. *ACM Trans. Inf. Syst. Secur.*, 18(3), mar 2016.
- [15] Alexander Bienstock, Daniel Escudero, and Antigoni Polychroniadou. On linear communication complexity for (maximally) fluid MPC. In Helena Handschuh and Anna Lysyanskaya, editors, *Advances in Cryptology – CRYPTO 2023, Part I*, volume 14081 of *Lecture Notes in Computer Science*, pages 263–294. Springer, Cham, August 2023.
- [16] Keith Bonawitz, Vladimir Ivanov, Ben Kreuter, Antonio Marcedone, H. Brendan McMahan, Sarvar Patel, Daniel Ramage, Aaron Segal, and Karn Seth. Practical secure aggregation for privacy-preserving machine learning. In Bhavani M. Thuraisingham, David Evans, Tal Malkin, and Dongyan Xu, editors, *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security, CCS 2017, Dallas, TX, USA, October 30 - November 03, 2017*, pages 1175–1191. ACM, 2017.
- [17] Dan Boneh, Benedikt Bünz, and Ben Fisch. Batching techniques for accumulators with applications to IOPs and stateless blockchains. In Alexandra Boldyreva and Daniele Micciancio, editors, *Advances in Cryptology – CRYPTO 2019, Part I*, volume 11692 of *Lecture Notes in Computer Science*, pages 561–586. Springer, Cham, August 2019.
- [18] Dan Boneh, Kevin Lewi, Hart William Montgomery, and Ananth Raghunathan. Key homomorphic PRFs and their applications. In Ran Canetti and Juan A. Garay, editors, *Advances in Cryptology – CRYPTO 2013, Part I*, volume 8042 of *Lecture Notes in Computer Science*, pages 410–428. Springer, Berlin, Heidelberg, August 2013.
- [19] Cyril Bouvier, Guilhem Castagnos, Laurent Imbert, and Fabien Laguillaumie. I want to ride my BICYCL : BICYCL implements cryptography in class groups. *J. Cryptol.*, 36(3):17, 2023.
- [20] Lennart Braun, Ivan Damgård, and Claudio Orlandi. Secure multiparty computation from threshold encryption based on class groups. In Helena Handschuh and Anna Lysyanskaya, editors, *Advances in Cryptology – CRYPTO 2023, Part I*, volume 14081 of *Lecture Notes in Computer Science*, pages 613–645. Springer, Cham, August 2023.
- [21] Joakim Brorsson and Martin Gunnarsson. Dipsauce: Efficient private stream aggregation without trusted parties. To Appear in NordSec23, 2023.
- [22] Johannes Buchmann and Hugh C. Williams. A key-exchange system based on imaginary quadratic fields. *Journal of Cryptology*, 1(2):107–118, June 1988.
- [23] Benedikt Bünz, Ben Fisch, and Alan Szepieniec. Transparent SNARKs from DARK compilers. In Anne Canteaut and Yuval Ishai, editors, *Advances in Cryptology – EUROCRYPT 2020, Part I*, volume 12105 of *Lecture Notes in Computer Science*, pages 677–706. Springer, Cham, May 2020.

- [24] David Byrd, Maria Hybinette, and Tucker Hybinette Balch. Abides: Towards high-fidelity multi-agent market simulation. In *Proceedings of the 2020 ACM SIGSIM Conference on Principles of Advanced Discrete Simulation*, SIGSIM-PADS '20, page 11–22, New York, NY, USA, 2020. Association for Computing Machinery.
- [25] David Byrd, Vaikkunth Mugunthan, Antigoni Polychroniadou, and Tucker Balch. Collusion resistant federated learning with oblivious distributed differential privacy. In *Proceedings of the Third ACM International Conference on AI in Finance*, ICAIF '22, page 114–122, New York, NY, USA, 2022. Association for Computing Machinery.
- [26] Ignacio Cascudo and Bernardo David. SCRAPE: Scalable randomness attested by public entities. In Dieter Gollmann, Atsuko Miyaji, and Hiroaki Kikuchi, editors, *ACNS 17: 15th International Conference on Applied Cryptography and Network Security*, volume 10355 of *Lecture Notes in Computer Science*, pages 537–556. Springer, Cham, July 2017.
- [27] Ignacio Cascudo and Bernardo David. Publicly verifiable secret sharing over class groups and applications to DKG and YOSO. In Marc Joye and Gregor Leander, editors, *Advances in Cryptology – EUROCRYPT 2024, Part V*, volume 14655 of *Lecture Notes in Computer Science*, pages 216–248. Springer, Cham, May 2024.
- [28] Guilhem Castagnos, Dario Catalano, Fabien Laguillaumie, Federico Savasta, and Ida Tucker. Two-party ECDSA from hash proof systems and efficient instantiations. In Alexandra Boldyreva and Daniele Micciancio, editors, *Advances in Cryptology – CRYPTO 2019, Part III*, volume 11694 of *Lecture Notes in Computer Science*, pages 191–221. Springer, Cham, August 2019.
- [29] Guilhem Castagnos, Dario Catalano, Fabien Laguillaumie, Federico Savasta, and Ida Tucker. Bandwidth-efficient threshold EC-DSA. In Aggelos Kiayias, Markulf Kohlweiss, Petros Wallden, and Vassilis Zikas, editors, *PKC 2020: 23rd International Conference on Theory and Practice of Public Key Cryptography, Part II*, volume 12111 of *Lecture Notes in Computer Science*, pages 266–296. Springer, Cham, May 2020.
- [30] Guilhem Castagnos, Laurent Imbert, and Fabien Laguillaumie. Encryption switching protocols revisited: Switching modulo p . In Jonathan Katz and Hovav Shacham, editors, *Advances in Cryptology – CRYPTO 2017, Part I*, volume 10401 of *Lecture Notes in Computer Science*, pages 255–287. Springer, Cham, August 2017.
- [31] Guilhem Castagnos and Fabien Laguillaumie. Linearly homomorphic encryption from DDH. In Kaisa Nyberg, editor, *Topics in Cryptology – CT-RSA 2015*, volume 9048 of *Lecture Notes in Computer Science*, pages 487–505. Springer, Cham, April 2015.
- [32] Guilhem Castagnos, Fabien Laguillaumie, and Ida Tucker. Practical fully secure unrestricted inner product functional encryption modulo p . In Thomas Peyrin and Steven Galbraith, editors, *Advances in Cryptology – ASIACRYPT 2018, Part II*, volume 11273 of *Lecture Notes in Computer Science*, pages 733–764. Springer, Cham, December 2018.
- [33] Pyrros Chaidos and Geoffroy Couteau. Efficient designated-verifier non-interactive zero-knowledge proofs of knowledge. In Jesper Buus Nielsen and Vincent Rijmen, editors, *Advances in Cryptology – EUROCRYPT 2018, Part III*, volume 10822 of *Lecture Notes in Computer Science*, pages 193–221. Springer, Cham, April / May 2018.
- [34] Jung Hee Cheon, Dongwoo Kim, Duhyeon Kim, Joohee Lee, Junbum Shin, and Yongsoo Song. Lattice-based secure biometric authentication for hamming distance. In Joonsang Baek and Sushmita Ruj, editors, *ACISP 21: 26th Australasian Conference on Information Security and Privacy*, volume 13083 of *Lecture Notes in Computer Science*, pages 653–672. Springer, Cham, December 2021.

- [35] Kevin Choi, Aathira Manoj, and Joseph Bonneau. SoK: Distributed randomness beacons. In *2023 IEEE Symposium on Security and Privacy*, pages 75–92. IEEE Computer Society Press, May 2023.
- [36] Arka Rai Choudhuri, Aarushi Goel, Matthew Green, Abhishek Jain, and Gabriel Kaptchuk. Fluid MPC: Secure multiparty computation with dynamic participants. In Tal Malkin and Chris Peikert, editors, *Advances in Cryptology – CRYPTO 2021, Part II*, volume 12826 of *Lecture Notes in Computer Science*, pages 94–123, Virtual Event, August 2021. Springer, Cham.
- [37] Jean-Sébastien Coron. On the exact security of full domain hash. In Mihir Bellare, editor, *Advances in Cryptology – CRYPTO 2000*, volume 1880 of *Lecture Notes in Computer Science*, pages 229–235. Springer, Berlin, Heidelberg, August 2000.
- [38] Henry Corrigan-Gibbs. Privacy-preserving firefox telemetry with prio, 2020.
- [39] Henry Corrigan-Gibbs and Dan Boneh. Prio: Private, robust, and scalable computation of aggregate statistics. In Aditya Akella and Jon Howell, editors, *14th USENIX Symposium on Networked Systems Design and Implementation, NSDI 2017, Boston, MA, USA, March 27-29, 2017*, pages 259–282. USENIX Association, 2017.
- [40] Geoffroy Couteau, Dahmun Goudarzi, Michael KlooSS, and Michael Reichle. Sharp: Short relaxed range proofs. In Heng Yin, Angelos Stavrou, Cas Cremers, and Elaine Shi, editors, *ACM CCS 2022: 29th Conference on Computer and Communications Security*, pages 609–622. ACM Press, November 2022.
- [41] Geoffroy Couteau, Michael KlooSS, Huang Lin, and Michael Reichle. Efficient range proofs with transparent setup from bounded integer commitments. In Anne Canteaut and François-Xavier Standaert, editors, *Advances in Cryptology – EUROCRYPT 2021, Part III*, volume 12698 of *Lecture Notes in Computer Science*, pages 247–277. Springer, Cham, October 2021.
- [42] Ivan Damgård and Rune Thorbek. Linear integer secret sharing and distributed exponentiation. In Moti Yung, Yevgeniy Dodis, Aggelos Kiayias, and Tal Malkin, editors, *PKC 2006: 9th International Conference on Theory and Practice of Public Key Cryptography*, volume 3958 of *Lecture Notes in Computer Science*, pages 75–90. Springer, Berlin, Heidelberg, April 2006.
- [43] Yi Deng, Shunli Ma, Xinxuan Zhang, Hailong Wang, Xuyang Song, and Xiang Xie. Promise Σ -protocol: How to construct efficient threshold ECDSA from encryptions based on class groups. In Mehdi Tibouchi and Huaxiong Wang, editors, *Advances in Cryptology – ASIACRYPT 2021, Part IV*, volume 13093 of *Lecture Notes in Computer Science*, pages 557–586. Springer, Cham, December 2021.
- [44] Nico Döttling, Dimitris Kolonelos, Russell W. F. Lai, Chuanwei Lin, Giulio Malavolta, and Ahmadreza Rahimi. Efficient laconic cryptography from learning with errors. In Carmit Hazay and Martijn Stam, editors, *Advances in Cryptology – EUROCRYPT 2023, Part III*, volume 14006 of *Lecture Notes in Computer Science*, pages 417–446. Springer, Cham, April 2023.
- [45] Drand. Drand/drاند: a distributed randomness beacon daemon - go implementation.
- [46] Johannes Ernst and Alexander Koch. Private stream aggregation with labels in the standard model. *Proc. Priv. Enhancing Technol.*, 2021(4):117–138, 2021.
- [47] Matthew K. Franklin and Moti Yung. Communication complexity of secure computation (extended abstract). In *24th Annual ACM Symposium on Theory of Computing*, pages 699–710. ACM Press, May 1992.
- [48] Olive Franzese, Adam Dziedzic, Christopher A. Choquette-Choo, Mark R. Thomas, Muhammad Ahmad Kaleem, Stephan Rabanser, Congyu Fang, Somesh Jha, Nicolas Papernot, and Xiao Wang. Robust and actively secure serverless collaborative learning. In *Proceedings of the 37th International Conference on Neural Information Processing Systems, NIPS ’23*, Red Hook, NY, USA, 2023. Curran Associates Inc.

- [49] Craig Gentry, Shai Halevi, Hugo Krawczyk, Bernardo Magri, Jesper Buus Nielsen, Tal Rabin, and Sophia Yakoubov. YOSO: You only speak once - secure MPC with stateless ephemeral roles. In Tal Malkin and Chris Peikert, editors, *Advances in Cryptology – CRYPTO 2021, Part II*, volume 12826 of *Lecture Notes in Computer Science*, pages 64–93, Virtual Event, August 2021. Springer, Cham.
- [50] Niv Gilboa and Yuval Ishai. Distributed point functions and their applications. In Phong Q. Nguyen and Elisabeth Oswald, editors, *Advances in Cryptology - EUROCRYPT 2014 - 33rd Annual International Conference on the Theory and Applications of Cryptographic Techniques, Copenhagen, Denmark, May 11-15, 2014. Proceedings*, volume 8441 of *Lecture Notes in Computer Science*, pages 640–658. Springer, 2014.
- [51] Noemi Glaeser, Matteo Maffei, Giulio Malavolta, Pedro Moreno-Sanchez, Erkan Tairi, and Sri AravindaKrishnan Thyagarajan. Foundations of coin mixing services. *Cryptology ePrint Archive*, Report 2022/942, 2022.
- [52] Oded Goldreich. *Foundations of Cryptography: Volume 2, Basic Applications*. Cambridge University Press, USA, 2004.
- [53] Yue Guo, Antigoni Polychroniadou, Elaine Shi, David Byrd, and Tucker Balch. MicroFedML: Privacy preserving federated learning for small weights. *Cryptology ePrint Archive*, Report 2022/714, 2022.
- [54] Yue Guo, Antigoni Polychroniadou, Elaine Shi, David Byrd, and Tucker Balch. Microsecagg: Streamlined single-server secure aggregation. *Proceedings on Privacy Enhancing Technologies*, 2024:246–275, 07 2024.
- [55] Shai Halevi, Yuval Ishai, Eyal Kushilevitz, and Tal Rabin. Additive randomized encodings and their applications. In Helena Handschuh and Anna Lysyanskaya, editors, *Advances in Cryptology - CRYPTO 2023 - 43rd Annual International Cryptology Conference, CRYPTO 2023, Santa Barbara, CA, USA, August 20-24, 2023, Proceedings, Part I*, volume 14081 of *Lecture Notes in Computer Science*, pages 203–235. Springer, 2023.
- [56] Shai Halevi, Yehuda Lindell, and Benny Pinkas. Secure computation on the web: Computing without simultaneous interaction. In Phillip Rogaway, editor, *Advances in Cryptology - CRYPTO 2011 - 31st Annual Cryptology Conference, Santa Barbara, CA, USA, August 14-18, 2011. Proceedings*, volume 6841 of *Lecture Notes in Computer Science*, pages 132–150. Springer, 2011.
- [57] Bargav Jayaraman, Lingxiao Wang, David Evans, and Quanquan Gu. Distributed learning without distrust: privacy-preserving empirical risk minimization. In *Proceedings of the 32nd International Conference on Neural Information Processing Systems, NIPS’18*, page 6346–6357, Red Hook, NY, USA, 2018. Curran Associates Inc.
- [58] Marc Joye and Benoît Libert. A scalable scheme for privacy-preserving aggregation of time-series data. In Ahmad-Reza Sadeghi, editor, *FC 2013: 17th International Conference on Financial Cryptography and Data Security*, volume 7859 of *Lecture Notes in Computer Science*, pages 111–125. Springer, Berlin, Heidelberg, April 2013.
- [59] Peter Kairouz, H. Brendan McMahan, Brendan Avent, Aurélien Bellet, Mehdi Bennis, Arjun Nitin Bhagoji, Kallista A. Bonawitz, Zachary Charles, Graham Cormode, Rachel Cummings, Rafael G. L. D’Oliveira, Hubert Eichner, Salim El Rouayheb, David Evans, Josh Gardner, Zachary Garrett, Adrià Gascón, Badi Ghazi, Phillip B. Gibbons, Marco Gruteser, Zaïd Harchaoui, Chaoyang He, Lie He, Zhouyuan Huo, Ben Hutchinson, Justin Hsu, Martin Jaggi, Tara Javidi, Gauri Joshi, Mikhail Khodak, Jakub Konečný, Aleksandra Korolova, Farinaz Koushanfar, Sanmi Koyejo, Tancrède Lepoint, Yang Liu, Prateek Mittal, Mehryar Mohri, Richard Nock, Ayfer Özgür, Rasmus Pagh, Hang Qi, Daniel Ramage, Ramesh Raskar, Mariana Raykova, Dawn Song, Weikang Song, Sebastian U. Stich, Ziteng Sun, Ananda Theertha Suresh, Florian Tramèr, Praneeth Vepakomma, Jianyu Wang, Li Xiong, Zheng Xu,

- Qiang Yang, Felix X. Yu, Han Yu, and Sen Zhao. Advances and open problems in federated learning. *Found. Trends Mach. Learn.*, 14(1-2):1–210, 2021.
- [60] Aniket Kate, Easwar Vivek Mangipudi, Pratyay Mukherjee, Hamza Saleem, and Sri Aravinda Krishnan Thyagarajan. Non-interactive VSS using class groups and application to DKG. *Cryptology ePrint Archive*, Report 2023/451, 2023.
- [61] Duhyeong Kim, Dongwon Lee, Jinyeong Seo, and Yongsoo Song. Toward practical lattice-based proof of knowledge from hint-MLWE. In Helena Handschuh and Anna Lysyanskaya, editors, *Advances in Cryptology – CRYPTO 2023, Part V*, volume 14085 of *Lecture Notes in Computer Science*, pages 549–580. Springer, Cham, August 2023.
- [62] Russell W. F. Lai and Giulio Malavolta. Subvector commitments with application to succinct arguments. In Alexandra Boldyreva and Daniele Micciancio, editors, *Advances in Cryptology – CRYPTO 2019, Part I*, volume 11692 of *Lecture Notes in Computer Science*, pages 530–560. Springer, Cham, August 2019.
- [63] Iraklis Leontiadis, Kaoutar Elkhayaoui, and Refik Molva. Private and dynamic time-series data aggregation with trust relaxation. In Dimitris Gritzalis, Aggelos Kiayias, and Ioannis G. Askoxylakis, editors, *CANS 14: 13th International Conference on Cryptology and Network Security*, volume 8813 of *Lecture Notes in Computer Science*, pages 305–320. Springer, Cham, October 2014.
- [64] Hanjun Li, Huijia Lin, Antigoni Polychroniadou, and Stefano Tessaro. LERNA: Secure single-server aggregation via key-homomorphic masking. In Jian Guo and Ron Steinfeld, editors, *Advances in Cryptology – ASIACRYPT 2023, Part I*, volume 14438 of *Lecture Notes in Computer Science*, pages 302–334. Springer, Singapore, December 2023.
- [65] Liping Li, Wei Xu, Tianyi Chen, Georgios B. Giannakis, and Qing Ling. Rsa: Byzantine-robust stochastic aggregation methods for distributed learning from heterogeneous datasets. In *Proceedings of the Thirty-Third AAAI Conference on Artificial Intelligence and Thirty-First Innovative Applications of Artificial Intelligence Conference and Ninth AAAI Symposium on Educational Advances in Artificial Intelligence*, AAAI’19/IAAI’19/EAAI’19. AAAI Press, 2019.
- [66] Yehuda Lindell. *How to Simulate It – A Tutorial on the Simulation Proof Technique*, pages 277–346. Springer International Publishing, Cham, 2017.
- [67] Helger Lipmaa. Secure accumulators from euclidean rings without trusted setup. In Feng Bao, Pierangela Samarati, and Jianying Zhou, editors, *ACNS 12: 10th International Conference on Applied Cryptography and Network Security*, volume 7341 of *Lecture Notes in Computer Science*, pages 224–240. Springer, Berlin, Heidelberg, June 2012.
- [68] Zizhen Liu, Si Chen, Jing Ye, Junfeng Fan, Huawei Li, and Xiaowei Li. SASH: efficient secure aggregation based on SHPRG for federated learning. In James Cussens and Kun Zhang, editors, *Uncertainty in Artificial Intelligence, Proceedings of the Thirty-Eighth Conference on Uncertainty in Artificial Intelligence, UAI 2022, 1-5 August 2022, Eindhoven, The Netherlands*, volume 180 of *Proceedings of Machine Learning Research*, pages 1243–1252. PMLR, 2022.
- [69] Hidde Lycklama, Lukas Burkhalter, Alexander Viand, Nicolas Küchler, and Anwar Hithnawi. RoFL: Robustness of secure federated learning. In *2023 IEEE Symposium on Security and Privacy*, pages 453–476. IEEE Computer Society Press, May 2023.
- [70] Vadim Lyubashevsky, Ngoc Khanh Nguyen, and Maxime Plançon. Lattice-based zero-knowledge proofs and applications: Shorter, simpler, and more general. In Yevgeniy Dodis and Thomas Shrimpton, editors, *Advances in Cryptology – CRYPTO 2022, Part II*, volume 13508 of *Lecture Notes in Computer Science*, pages 71–101. Springer, Cham, August 2022.

- [71] Vadim Lyubashevsky, Chris Peikert, and Oded Regev. On ideal lattices and learning with errors over rings. In Henri Gilbert, editor, *Advances in Cryptology – EUROCRYPT 2010*, volume 6110 of *Lecture Notes in Computer Science*, pages 1–23. Springer, Berlin, Heidelberg, May / June 2010.
- [72] Yiping Ma, Yue Guo, Harish Karthikeyan, and Antigoni Polychroniadou. Armadillo: Robust secure aggregation for federated learning with input validation. In *ACM CCS 2025: 32nd Conference on Computer and Communications Security*. ACM Press, October 2025.
- [73] Yiping Ma, Jess Woods, Sebastian Angel, Antigoni Polychroniadou, and Tal Rabin. Flamingo: Multi-round single-server secure aggregation with applications to private federated learning. In *2023 IEEE Symposium on Security and Privacy*, pages 477–496. IEEE Computer Society Press, May 2023.
- [74] K. MCCURLEY. Cryptographic key distribution and computation in class groups. *Proceedings of NATO ASI Number Theory and applications*, pages 459–479, 1989.
- [75] Brendan McMahan, Eider Moore, Daniel Ramage, Seth Hampson, and Blaise Agüera y Arcas. Communication-Efficient Learning of Deep Networks from Decentralized Data. In Aarti Singh and Jerry Zhu, editors, *Proceedings of the 20th International Conference on Artificial Intelligence and Statistics*, volume 54 of *Proceedings of Machine Learning Research*, pages 1273–1282. PMLR, 20–22 Apr 2017.
- [76] Moni Naor, Benny Pinkas, and Omer Reingold. Distributed pseudo-random functions and KDCs. In Jacques Stern, editor, *Advances in Cryptology – EUROCRYPT’99*, volume 1592 of *Lecture Notes in Computer Science*, pages 327–346. Springer, Berlin, Heidelberg, May 1999.
- [77] Dinh Duy Nguyen, Duong Hieu Phan, and David Pointcheval. Verifiable decentralized multi-client functional encryption for inner product. In Jian Guo and Ron Steinfeld, editors, *Advances in Cryptology – ASIACRYPT 2023, Part V*, volume 14442 of *Lecture Notes in Computer Science*, pages 33–65. Springer, Singapore, December 2023.
- [78] Hiroki Okada, Atsushi Takayasu, Kazuhide Fukushima, Shinsaku Kiyomoto, and Tsuyoshi Takagi. A compact digital signature scheme based on the module-lwr problem. In Weizhi Meng, Dieter Gollmann, Christian D. Jensen, and Jianying Zhou, editors, *Information and Communications Security*, pages 73–90, Cham, 2020. Springer International Publishing.
- [79] Johannes Ottenhues and Alexander Koch. LaPSuS - A lattice-based private stream aggregation scheme under scrutiny. In Clemente Galdi and Duong Hieu Phan, editors, *SCN 24: 14th International Conference on Security in Communication Networks, Part II*, volume 14974 of *Lecture Notes in Computer Science*, pages 228–248. Springer, Cham, September 2024.
- [80] Dario Pasquini, Danilo Francati, and Giuseppe Ateniese. Eluding secure aggregation in federated learning via model inconsistency. In Heng Yin, Angelos Stavrou, Cas Cremers, and Elaine Shi, editors, *ACM CCS 2022: 29th Conference on Computer and Communications Security*, pages 2429–2443. ACM Press, November 2022.
- [81] Christopher Patton, Richard Barnes, and Phillipp Schoppmann. Verifiable Distributed Aggregation Functions. Internet-Draft draft-patton-cfrg-vdaf-01, Internet Engineering Task Force, March 2022. Work in Progress.
- [82] Andrea Dal Pozzolo, Olivier Caelen, Reid A. Johnson, and Gianluca Bontempi. Calibrating probability with undersampling for unbalanced classification. In *2015 IEEE Symposium Series on Computational Intelligence*, pages 159–166, 2015.
- [83] Mayank Raikwar and Danilo Gligoroski. SoK: Decentralized randomness beacon protocols. In Khoa Nguyen, Guomin Yang, Fuchun Guo, and Willy Susilo, editors, *ACISP 22: 27th Australasian Conference on Information Security and Privacy*, volume 13494 of *Lecture Notes in Computer Science*, pages 420–446. Springer, Cham, November 2022.

- [84] Mayank Rathee, Conghao Shen, Sameer Wagh, and Raluca Ada Popa. Elsa: Secure aggregation for federated learning with malicious actors. In *2023 IEEE Symposium on Security and Privacy (SP)*, pages 1961–1979, 2023.
- [85] Mayank Rathee, Conghao Shen, Sameer Wagh, and Raluca Ada Popa. ELSA: Secure aggregation for federated learning with malicious actors. In *2023 IEEE Symposium on Security and Privacy*, pages 1961–1979. IEEE Computer Society Press, May 2023.
- [86] Sashank J. Reddi, Zachary Charles, Manzil Zaheer, Zachary Garrett, Keith Rush, Jakub Konečný, Sanjiv Kumar, and Hugh Brendan McMahan. Adaptive federated optimization. In *International Conference on Learning Representations*, 2021.
- [87] Oded Regev. On lattices, learning with errors, random linear codes, and cryptography. *J. ACM*, 56(6), sep 2009.
- [88] Adi Shamir. How to share a secret. *Communications of the Association for Computing Machinery*, 22(11):612–613, November 1979.
- [89] Jiawei Shao, Yuchang Sun, Songze Li, and Jun Zhang. Dres-fl: dropout-resilient secure federated learning for non-iid clients via secret data sharing. In *Proceedings of the 36th International Conference on Neural Information Processing Systems, NIPS ’22*, Red Hook, NY, USA, 2022. Curran Associates Inc.
- [90] Elaine Shi, T.-H. Hubert Chan, Eleanor G. Rieffel, Richard Chow, and Dawn Song. Privacy-preserving aggregation of time-series data. In *ISOC Network and Distributed System Security Symposium – NDSS 2011*. The Internet Society, February 2011.
- [91] Reza Shokri, Marco Stronati, Congzheng Song, and Vitaly Shmatikov. Membership inference attacks against machine learning models. In *2017 IEEE Symposium on Security and Privacy (SP)*, pages 3–18. IEEE, 2017.
- [92] Victor Shoup. Practical threshold signatures. In Bart Preneel, editor, *Advances in Cryptology – EUROCRYPT 2000*, volume 1807 of *Lecture Notes in Computer Science*, pages 207–220. Springer, Berlin, Heidelberg, May 2000.
- [93] Jonathan Takeshita, Zachariah Carmichael, Ryan Karl, and Taeho Jung. TERSE: tiny encryptions and really speedy execution for post-quantum private stream aggregation. In Fengjun Li, Kaitai Liang, Zhiqiang Lin, and Sokratis K. Katsikas, editors, *Security and Privacy in Communication Networks - 18th EAI International Conference, SecureComm 2022, Virtual Event, October 2022, Proceedings*, volume 462 of *Lecture Notes of the Institute for Computer Sciences, Social Informatics and Telecommunications Engineering*, pages 331–352. Springer, 2022.
- [94] Jonathan Takeshita, Ryan Karl, Ting Gong, and Taeho Jung. SLAP: Simple lattice-based private stream aggregation protocol. Cryptology ePrint Archive, Report 2020/1611, 2020.
- [95] Jonathan Takeshita, Ryan Karl, Ting Gong, and Taeho Jung. SLAP: Simpler, improved private stream aggregation from ring learning with errors. *Journal of Cryptology*, 36(2):8, April 2023.
- [96] Sri Aravinda Krishnan Thyagarajan, Guilhem Castagnos, Fabien Laguillaumie, and Giulio Malavolta. Efficient CCA timed commitments in class groups. In Giovanni Vigna and Elaine Shi, editors, *ACM CCS 2021: 28th Conference on Computer and Communications Security*, pages 2663–2684. ACM Press, November 2021.
- [97] Ida Tucker. *Functional encryption and distributed signatures based on projective hash functions, the benefit of class groups*. Theses, Université de Lyon, October 2020.

- [98] Hendrik Waldner, Tilen Marc, Miha Stopar, and Michel Abdalla. Private stream aggregation from labeled secret sharing schemes. Cryptology ePrint Archive, Report 2021/081, 2021.
- [99] Benjamin Wesolowski. Efficient verifiable delay functions. In Yuval Ishai and Vincent Rijmen, editors, *Advances in Cryptology – EUROCRYPT 2019, Part III*, volume 11478 of *Lecture Notes in Computer Science*, pages 379–407. Springer, Cham, May 2019.
- [100] Benjamin Wesolowski. Efficient verifiable delay functions. *Journal of Cryptology*, 33(4):2113–2147, October 2020.
- [101] Tsz Hon Yuen, Handong Cui, and Xiang Xie. Compact zero-knowledge proofs for threshold ECDSA with trustless setup. In Juan Garay, editor, *PKC 2021: 24th International Conference on Theory and Practice of Public Key Cryptography, Part I*, volume 12710 of *Lecture Notes in Computer Science*, pages 481–511. Springer, Cham, May 2021.
- [102] Yulin Zhao, Hualin Zhou, and Zhiguo Wan. SuperFL: Privacy-preserving federated learning with efficiency and robustness. Cryptology ePrint Archive, Report 2024/081, 2024.

A Preliminaries

For completeness, we discuss secret sharing in Section A.1. We discuss pseudorandom functions in Section A.2. We then introduce lattice-based cryptographic assumptions in Section 5.2.

A.1 Secret Sharing

A key component of threshold cryptography is the ability to compute distributed exponentiation by sharing a secret. More formally, the standard approach is to compute g^s for some $g \in \mathbb{G}$ where $\mathbb{G}$ is a finite group and s is a secret exponent that has been secret-shared among multiple parties. This problem is much simpler when you assume that the group order is a publicly known prime p which then requires you to share the secret over the field $\mathbb{Z}_p$. This was the observation of Shamir [88] whereby a secret s can be written as a linear combination of $\sum_{i \in \mathcal{S}} \alpha_i s_i \bmod p$ where $\mathcal{S}$ is a set of servers that is sufficiently large and holds shares of the secret s_i and α_i is only a function of the indices in $\mathcal{S}$. It follows that if each server provides $g_i = g^{s_i}$, then one can compute $g^s = g^{\sum_{i \in \mathcal{S}} \alpha_i s_i} = \prod_{i \in \mathcal{S}} g_i^{\alpha_i}$. Formally, this is defined below.

Construction 4 (Shamir’s Secret Sharing over $\mathbb{F}_q$). Consider the following (t, r, m) Secret Sharing Scheme where m is the total number of parties, t is the corruption threshold, r is the threshold for reconstruction. Then, we have the following scheme:

- **Share**(s, t, r, m): Sample a random polynomial $f(X) \in \mathbb{F}_q[X]$ of degree $r - 1$ such that $f(0) = s$. Then, **return** $\{s^{(j)} := f(j)\}_{j \in [m]}$
- **Coeff**($\mathcal{S}$): On input of a set $\mathcal{S} = \{i_1, \dots, i_r, \dots\} \subseteq [m]$ of at least r indices, compute $\lambda_{i_j} = \prod_{\zeta \in [r] \setminus \{j\}} \frac{i_\zeta}{i_\zeta - i_j}$. Then, **return** $\{\lambda_{i_j}\}_{i_j \in \{i_1, \dots, i_r\}}$
- **Reconstruct**($\{s^{(j)}\}_{j \in \mathcal{S}}$): If $|\mathcal{S}| \geq r$, then output $\sum_{j \in \mathcal{S}} \lambda_j \cdot s^{(j)}$ where $\{\lambda_j\} \leftarrow \text{Coeff}(\mathcal{S})$.

The correctness of the scheme guarantees that the secret s is correctly reconstructed.

Construction 5 (Packed Secret Sharing over $\mathbb{F}_q$). Consider the following (t, r, m) Secret Sharing Scheme where m is the total number of parties, t is the corruption threshold, r is the threshold for reconstruction. Further, let ρ be the number of secrets being packed which are to be embedded at points $\text{pos}_1, \dots, \text{pos}_\rho$ where $\text{pos}_i = m + i$. Here, $t := r - \rho$. Then, we have the following scheme:

Share($\mathbf{s} = (s_1, \dots, s_\rho), \mathbf{t}, \mathbf{r}, \mathbf{m}$)	Reconstruct($\{s^{(i)}\}_{i \in \mathcal{S}}$)
$(r_0, \dots, r_{r-\rho-1}) \leftarrow \mathbb{F}_q$ $q(X) := \sum_{i=0}^{r-\rho-1} X^i \cdot r_i$ $\text{pos}_i = \mathbf{m} + i$ for $i = 1, \dots, \rho$ for $i \in [\rho]$ do $L_i(X) := \prod_{j \in [\rho] \setminus i} \frac{X - \text{pos}_j}{\text{pos}_i - \text{pos}_j} \cdot \Delta$ $f(X) := q(X) \prod_{i=1}^{\rho} (X - \text{pos}_i) + \sum_{i=1}^{\rho} s_i \cdot L_i(X)$ return $\{s^{(i)}\}_{i \in [m]}$	if $ \mathcal{S} < r$ return $\perp$ Parse $\mathcal{S} := \{i_1, \dots, i_t, \dots\}$ for $k \in [\rho]$ for $j \in [t]$ $\Lambda_{i_j}(X) := \prod_{\zeta \in [t] \setminus j} \frac{i_\zeta - X}{i_\zeta - i_j}$ $s'_k := \sum_{j \in [t]} \Lambda_{i_j}(m + k) \cdot s^{(j)}$ return $\mathbf{s}' := (s'_1, \dots, s'_\rho)$

Parameters.

- For reconstruction, we require $r < m(1 - \delta_C)$ where δ_C is the dropout rate within the committee.
- For security, we require that $(r - \rho) > m \cdot \eta_C$ where η_C is the corruption rate.

Combining, we get $m > \rho / (1 - \delta_C - \eta_C)$. Recall that we need $\delta_C + \eta_C < 1/3$, for byzantine fault tolerance. In other words, setting $m \geq 3\rho/2$ is sufficient.

Remark 3 (Optimizations for Packed Secret Sharing). Observe that the polynomial $L_i(X)$ is only dependent on points pos_i , which are the points where the secrets are embedded. This can be pre-processed, and indeed, can be a part of the setup algorithm which distributes it to all the clients. Furthermore, rather than naively reconstructing the Lagrange polynomial, one can also rely on FFT techniques to achieve speed up.

Unfortunately, the above protocols do not extend to settings where the order of the group is not prime, not publicly known, or even possibly unknown to everyone. In this setting, the work of Damgård and Thorbek presents a construction to build Linear Integer Secret Sharing (LISS) schemes. In this work, we rely on the simpler scheme that extends Shamir's secret sharing into the integer setting from the work of Braun *et al.* [20]. We also extend the Packed Secret Sharing scheme to this integer setting in Construction 7.

Definition 6 (Secret Sharing over $\mathbb{Z}$). A $(\mathbf{t}, \mathbf{r}, \mathbf{m})$ Linear Integer Secret Sharing Scheme LISS is a tuple of PPT algorithms $\text{LISS} := (\text{Share}, \text{GetCoeff}, \text{Reconstruct})$, with the following public parameters: the statistical security parameter κ_s , the number of parties $\mathbf{m}$, the corruption threshold $\mathbf{t}$, and reconstruction threshold $\mathbf{r}$ of secrets needed for reconstruction, the randomness bit length ℓ_r , the bit length of the secret ℓ_s and the offset by which the secret is multiplied, denoted by $\Delta = \mathbf{m}!$, and the following syntax:

- $(s_1, \dots, s_m) \leftarrow \text{Share}(\mathbf{s}, \mathbf{m}, \mathbf{r}, \mathbf{t})$: On input of the secret $\mathbf{s}$, the number of parties $\mathbf{m}$, and the threshold $\mathbf{t}$, the share algorithm outputs shares $s_1, \dots, s_m$ such that party i receives s_i .
- $\{\lambda_i\}_{i \in \mathcal{S}} \leftarrow \text{GetCoeff}(\mathcal{S})$: On input of a set $\mathcal{S}$ of at least $\mathbf{r}$ indices, the GetCoeff algorithm outputs the set of coefficients for polynomial reconstruction.
- $s' \leftarrow \text{Reconstruct}(\{s_i\}_{i \in \mathcal{S}})$: On input of a set of secrets of at least $\mathbf{r}$ shares, the reconstruction algorithm outputs the secret s' .

We further require the following security properties.

- *Correctness:* For any $\mathbf{m}, \kappa_s, \mathbf{t}, r, \ell_s, \ell_r \in \mathbb{Z}$ with $\mathbf{t} < r \leq \mathbf{m}$, and any set $\mathcal{S} \subseteq [m]$ with $|\mathcal{S}| \geq r$, for any $\mathbf{s} \in \mathbb{Z}$ such that $\mathbf{s} \in [0, 2^{\ell_s})$ the following holds:

$$\Pr \left[\mathbf{s}' = f(\mathbf{s}) \mid \begin{array}{l} (\mathbf{s}_1, \dots, \mathbf{s}_m) \leftarrow^* \text{Share}(\mathbf{s}, \mathbf{m}, r, \mathbf{t}) \\ \mathbf{s}' \leftarrow \text{Reconstruct}(\{\mathbf{s}_i\}_{i \in \mathcal{S}}) \end{array} \right]$$

where f is some publicly computable function, usually $f(\mathbf{s}) = \Delta^2 \cdot \mathbf{s}$.

- *Statistical Privacy [42]:* We say that a $(\mathbf{t}, r, \mathbf{m})$ linear integer secret sharing scheme LISS is statistically private if for any set of corrupted parties $\mathcal{C} \subset [m]$ with $|\mathcal{C}| \leq \mathbf{t}$, and any two secrets $\mathbf{s}, \mathbf{s}' \in [0, 2^{\ell_s})$ and for independent random coins ρ, ρ' such that $\{\mathbf{s}_i\}_{i \in [m]} \leftarrow^* \text{Share}(\mathbf{s}; \rho)$, $\{\mathbf{s}'_i\}_{i \in [m]} \leftarrow^* \text{Share}(\mathbf{s}'; \rho')$ we have that the statistical distance between: $\{\mathbf{s}_i | i \in \mathcal{C}\}$ and $\{\mathbf{s}'_i | i \in \mathcal{C}\}$ is negligible in the statistical security parameter κ_s .

Construction 6 (Shamir's Secret Sharing over $\mathbb{Z}$). Consider the following $(\mathbf{t}, r, \mathbf{m})$ Integer Secret Sharing scheme where $\mathbf{m}$ is the number of parties, $\mathbf{t}$ is the corruption threshold, and r is the threshold for reconstruction. Further, let κ_s be a statistical security parameter. Let ℓ_s be the bit length of the secret and let ℓ_r be the bit length of the randomness. Then, we have the following scheme:

Share($\mathbf{s}, \mathbf{t}, r, \mathbf{m}$)	GetCoeff($\mathcal{S}$)	Reconstruct($\{\mathbf{s}^{(i)}\}_{i \in \mathcal{S}}$)
$\Delta := \mathbf{m}!, \tilde{\mathbf{s}} := \mathbf{s} \cdot \Delta$	if $ \mathcal{S} \geq r$	if $ \mathcal{S} \geq r$
$(r_1, \dots, r_{r-1}) \leftarrow^* [0, 2^{\ell_r + \kappa_s})$	for $i \in \mathcal{S}$ do	$\{\Lambda_i\}_{i \in \mathcal{S}} \leftarrow \text{GetCoeff}(\mathcal{S})$
$f(X) := \tilde{\mathbf{s}} + \sum_{i=1}^{r-1} r_i \cdot X^i$	$\Lambda_i := \prod_{j \in \mathcal{S} \setminus \{i\}} \frac{x_j}{x_j - x_i} \cdot \Delta$	$\mathbf{s}' := \sum_{i \in \mathcal{S}} \Lambda_i \cdot \mathbf{s}^{(i)}$
return $\{\mathbf{s}^{(i)} = f(i)\}_{i \in [m]}$	return $\{\Lambda_i\}_{i \in \mathcal{S}}$	return $\mathbf{s}'$

We omit the proof of correctness as it is similar to the original Shamir's Secret Sharing scheme. However, we highlight the critical differences:

- Unlike Shamir's Secret Sharing over fields, the secret here is already multiplied by the offset Δ . Therefore, any attempt to reconstruct can only yield $\mathbf{s} \cdot \Delta$
- However, note that the inverse of $x_j - x_i$ which was defined over the field $\mathbb{Z}_q$ might not exist or be efficiently computable in a field of unknown order. Instead, we multiply the Lagrange coefficients by Δ . Consequently, the reconstruction yields $\Delta \cdot \tilde{\mathbf{s}}$ which equals $\mathbf{s} \cdot \Delta^2$.

Theorem 5 ([20]). *Construction 6 is statistically private provided $\ell_r \geq \ell_s + \lceil \log_2(h_{\max} \cdot (\mathbf{t} - 1)) \rceil + 1$ where $h_{\max}$ is an upper bound on the coefficients of the sweeping polynomial.*

We refer the readers to the proof in [20, §B.1]. The key idea behind the proof is first to show that there exists a "sweeping polynomial" such that at each of the points that the adversary has a share of, the polynomial evaluates to 0 while at the point where the secret exists, it contains the offset Δ . Implicitly, one can add the sweeping polynomial to the original polynomial whereby the sweeping polynomial "sweeps" away the secret information that the adversary has gained knowledge of. Meanwhile, in the later section, we present the proof for the generic construction that uses Shamir's Packed Secret Sharing over the integer space. This again uses the idea of a sweeping polynomial.

Construction 7 (Shamir's Packed Secret Sharing over $\mathbb{Z}$). Let $\mathbf{m}$ be the number of parties and ρ be the number of secrets that are packed in one sharing. Further, let $\mathbf{t}$ denote the threshold for reconstruction (implies that corruption threshold is $\mathbf{t} - \rho$). Then, consider the following $(\mathbf{m}, \mathbf{t}, \rho)$ Integer Secret Sharing Scheme with system parameters κ_s as the statistical security parameter, ℓ_s is the bit length of the a secret, and let ℓ_r be the bit length of the randomness. Then, we have the following scheme:

PackedShare($\mathbf{s} = (s_1, \dots, s_\rho), \mathbf{t}, m$)	Reconstruct($\{\mathbf{s}^{(i)}\}_{i \in S}$)
$\Delta := m!, \tilde{\mathbf{s}} := \mathbf{s} \cdot \Delta$	if $ S < t$ return $\perp$
$(r_0, \dots, r_{t-\rho-1}) \leftarrow \$_{[0, 2^{\ell_r + \kappa_s}]}$	Parse $S := \{i_1, \dots, i_t, \dots\}$
$q(X) := \sum_{i=0}^{t-\rho-1} X^i \cdot r_i$	for $k \in [\rho]$
$\text{pos}_i = m + i$ for $i = 1, \dots, \rho$	for $j \in [t]$
for $i \in [\rho]$ do	$\Lambda_{i_j}(X) := \prod_{\zeta \in [t] \setminus j} \frac{i_\zeta - X}{i_\zeta - i_j} \cdot (\Delta)$
$L_i(X) := \prod_{j \in [\rho] \setminus i} \frac{X - \text{pos}_j}{\text{pos}_i - \text{pos}_j} \cdot \Delta$	$\mathbf{s}'_k := \sum_{j \in [t]} \Lambda_{i_j}(m + k) \cdot \mathbf{s}^{(j)}$
$f(X) := q(X) \prod_{i=1}^{\rho} (X - \text{pos}_i) + \sum_{i=1}^{\rho} \tilde{s}_i \cdot L_i(X)$	return $\mathbf{s}' := (s'_1, \dots, s'_\rho)$
return $\{\mathbf{s}^{(i)}\}_{i \in [m]}$	

Correctness. Observe that for all $i = 1, \dots, \rho$, we have the following:

- $L_i(\text{pos}_i) = \Delta$
- $L_j(\text{pos}_i) = \Delta$ for all $j \in [\rho], j \neq i$
- $f(\text{pos}_i) = \tilde{s}_i \cdot \Delta = s_i \cdot \Delta^2$

Meanwhile, for $\lambda_{i_j}(X) := \prod_{\zeta \in [t] \setminus [j]} \frac{i_\zeta - X}{i_\zeta - i_j}$, the polynomial we will be able to compute the polynomial $f(x) = \sum_{j \in [t]} \lambda_{i_j} \cdot s^{(j)}$ by correctness of Lagrange Interpolation. Consequently, $f(\text{pos}_i)$ would return $s_i \cdot \Delta^2$. However, we compute Λ_{i_j} instead, by multiplying with Δ to remove need for division. Consequently, the resulting polynomial has Δ multiplied throughout yielding a Δ^3 as the total offset.

Definition 7 (Vector of Sweeping Polynomials). *Let $\mathcal{C} \subset [m]$ such that $|\mathcal{C}| = t - \rho$. Then, we have a vector of sweeping polynomials, denoted by $\mathbf{sp}_{\mathcal{C}}(X) = (\mathbf{sp}_{1,\mathcal{C}}, \dots, \mathbf{sp}_{\rho,\mathcal{C}})$ where $\mathbf{sp}_{i,\mathcal{C}}(X) := \sum_{j=0}^{t-\rho} \mathbf{sp}_{i,j} \cdot X^j \in \mathbb{Z}[X]_{\leq t-1}$ is the unique polynomial whose degree is at most $t - 1$ such that $\mathbf{sp}_{i,\mathcal{C}}(m + i) = \Delta^2$, $\mathbf{sp}_{i,\mathcal{C}}(m + j) = 0$ for $j \in [\rho], j \neq i$, and $\mathbf{sp}_{i,\mathcal{C}}(j) = 0$ for all $j \in \mathcal{C}$. Further, one can define $\mathbf{sp}_{\max}$ as the upper bound for the coefficients for the sweeping polynomials, i.e., $\mathbf{sp}_{\max} := \{\mathbf{sp}_{i,j} | i \in \{1, \dots, \rho\}, j \in \{0, \dots, t - 1\}\}$*

Lemma 6 (Existence of Sweeping Polynomial). *For any $\mathcal{C} \subset [m]$ with $|\mathcal{C}| = t - \rho$, there exists $\mathbf{sp}_{\mathcal{C}} \in (\mathbb{Z}[X]_{\leq t-\rho})^\rho$ satisfying Definition 7.*

Proof. For any $i = 1, \dots, \rho$, we have that $\mathbf{sp}_{i,\mathcal{C}}(m + i) = \Delta^2$ and $\mathbf{sp}_{i,\mathcal{C}}(j) = 0$ for $j \in \mathcal{C}$. Let $\mathcal{C} := (i_1, \dots, i_{t-\rho})$. In other words, we can use these evaluations to construct a polynomial as follows:

$$\mathbf{sp}_{i,\mathcal{C}}(X) := \Delta^2 \cdot \prod_{j=1}^{t-\rho} \frac{(X - i_j)}{(m + i) - i_j} \cdot \prod_{j \in [\rho] \setminus \{i\}} \frac{(X - (m + j))}{(i - j)}$$

Note that $i_1, \dots, i_j \in [m]$ and are distinct. Therefore, $\prod_{j=1}^{t-\rho} (m + i) - i_j$ perfectly divides Δ and so does $\prod_{j \in [\rho] \setminus \{i\}} (i - j)$, which implies that the coefficients are all integers. Further, the degree of this polynomial is at most $t - 1$. Thus, $\mathbf{sp}_{i,\mathcal{C}}(X) \in \mathbb{Z}[X]_{t-1}$. This defines the resulting vector of sweeping polynomials $\mathbf{sp}_{\mathcal{C}}$. $\square$

Theorem 7. *Construction 7 is statistically private provided*

$$\ell_r \geq \ell_s + \lceil \log_2(\mathbf{sp}_{\max} \cdot (t - 1) \cdot \rho) \rceil + 1$$

Proof. Let $\mathbf{s}, \mathbf{s}' \in [0, 2^{\ell_s}]^\rho$ be two vectors of secrets. Then, $\tilde{\mathbf{s}} := \mathbf{s} \cdot \Delta$ and $\tilde{\mathbf{s}}' := \mathbf{s}' \cdot \Delta$. Let $\mathcal{C}$ denote an arbitrary subset of corrupted parties of size $|\mathcal{C}| = t - \rho$. Further, let us assume that $\tilde{\mathbf{s}}$ is shared using the polynomial $f(X)$ as defined below:

$$f(X) := q(X) \cdot \prod_{k=1}^{\rho} (X - \text{pos}_i) + \sum_{k=1}^{\rho} \tilde{s}_i \cdot L_k(X)$$

where $L_k(X) := \prod_{j \in [\rho] \setminus \{k\}} \frac{X - \text{pos}_j}{\text{pos}_k - \text{pos}_j} \cdot \Delta$. and $q(X)$ is a random polynomial of degree $t - \rho - 1$.

Now observe that the adversary see $|\mathcal{C}| = t - \rho$ shares corresponding to $f(i_j)$ for $i_j \in \mathcal{C}$. By Lagrange interpolation, this induces a one-to-one map from possible secrets to corresponding sharing polynomials. Specifically, we can use the vector of sweeping polynomials, as defined in Definition 7 to explicitly map any secret vector $\mathbf{s}^*$ to its sharing polynomial defined by $f(X) + \langle \mathbf{s}^* - \mathbf{s}, \mathbf{sp}_{\mathcal{C}}(X) \rangle$

In other words, the sharing polynomial to share $\mathbf{s}^*$ is defined by

$$f^*(X) = f(X) + \sum_{k=1}^{\rho} (s_k^* - s_k) \cdot \mathbf{sp}_{k,\mathcal{C}}(X)$$

One can verify the correctness. For example, to secret share s_1^* , at position $m + 1$, we get:

$$f^*(m+1) = f(m+1) + \sum_{k=1}^{\rho} (s_k^* - s_k) \cdot \mathbf{sp}_{k,\mathcal{C}}(X)$$

Now, observe that $f(m+1) = s_1 \cdot (\Delta^2)$. Meanwhile, $\mathbf{sp}_{1,\mathcal{C}}(m+1) = \Delta^2$ while $\mathbf{sp}_{j,\mathcal{C}}(m+1) = 0$ for $1 < j \leq \rho$. This simplifies to: $f^*(m+1) = s_1 \cdot \Delta^2 + (s_1^* - s_1) \cdot \Delta^2 = s_1^* \cdot \Delta^2$. However, while we have an efficient mapping, note that $f^*(X)$ could have coefficients that are not of the prescribed form, i.e., coefficients do not lie in the range $[0, 2^{\ell_r + \kappa_s}]$. We will call the event **good** if the coefficients lie in the range and **bad** even if one of the coefficients does not lie in the range.

Let us apply the above mapping to the secret $\mathbf{s}'$ and we have the resulting polynomial:

$$f'(X) = f(X) + \sum_{k=1}^{\rho} (s'_k - s_k) \cdot \mathbf{sp}_{k,\mathcal{C}}(X)$$

Now, observe that if $f'(X)$ was a good polynomial, then $f'(j) = f(j)$ for every $j \in \mathcal{C}$. It follows that if f' was **good**, then an adversary cannot distinguish whether the secret vector was $\mathbf{s}$ or $\mathbf{s}'$.

We will now upper bound the probability that f' was **bad** in at least one of the coefficients. We know that $|s'_k - s_k| \in [0, 2^{\ell_s}]$ for $k = 1, \dots, \rho$. Further all coefficients of $\mathbf{sp}_{k,\mathcal{C}}(X)$ are upper bounded by $\mathbf{sp}_{\max}$. Therefore, to any coefficient of $f(X)$, the maximum perturbation in value is: $2^{\ell_s} \cdot \mathbf{sp}_{\max} \cdot \rho$. Therefore, one requires that the original coefficients of f be sampled such that they lie in $[2^{\ell_s} \cdot \mathbf{sp}_{\max} \cdot \rho, 2^{\ell_r + \kappa_s} - 2^{\ell_s} \cdot \mathbf{sp}_{\max} \cdot \rho]$. In other words, the probability that one coefficient of f' is bad is:

$$\frac{2 \cdot 2^{\ell_s} \cdot \mathbf{sp}_{\max} \cdot \rho}{2^{\ell_r + \kappa_s}}$$

There are $t - 1$ such coefficients. This gives us that the probability is $\leq 2^{-\kappa_s}$ assuming that $\ell_r \geq \ell_s + \lceil \log_2(\mathbf{sp}_{\max} \cdot (t - 1) \cdot \rho) \rceil + 1$ $\square$

A.2 Pseudorandom Functions

Definition 8 (Pseudorandom Function (PRF)). A pseudorandom function family is defined by a tuple of PPT algorithms $\text{PRF} = (\text{Gen}, \text{Eval})$ with the following definitions:

- $\text{pp}_{\text{PRF}} \leftarrow \text{Gen}(1^\kappa)$: On input of the security parameter κ , the generation algorithm outputs the system parameters required to evaluate the function $F : \mathcal{K} \times \mathcal{X} \rightarrow \mathcal{Y}$ where $\mathcal{K}$ is the key space, $\mathcal{X}$ is the input space, and $\mathcal{Y}$ is the output space.
- $y \leftarrow \text{Eval}(k, x)$: On input of $x \in \mathcal{X}$ and a randomly chosen key $k \leftarrow \mathcal{K}$, the algorithm outputs $y \in \mathcal{Y}$ corresponding to the evaluation of $F(k, x)$.

We further require the following security property that: for all PPT adversaries $\mathcal{A}$, there exists a negligible function negl such that:

$$\Pr \left[b = b' \mid \begin{array}{c} b \leftarrow \{0, 1\}, k \leftarrow \mathcal{K} \\ \mathcal{O}_0(\cdot) := F(k, \cdot), \mathcal{O}_1(\cdot) := U(\mathcal{Y}) \\ b' \leftarrow \mathcal{A}^{\mathcal{O}_b(\cdot)} \end{array} \right] \leq \frac{1}{2} + \text{negl}(\kappa)$$

where $U(\mathcal{Y})$ outputs a randomly sampled element from $\mathcal{Y}$.

Definition 9 ((γ)-Key Homomorphic PRF). Let PRF be a pseudorandom function that realizes an efficiently computable function $F : \mathcal{K} \times \mathcal{X} \rightarrow \mathcal{Y}$ such that $(\mathcal{K}, \oplus)$ is a group. Then, we say that it is

- key homomorphic if: $(\mathcal{Y}, \otimes)$ is also a group and for every $k_1, k_2 \in \mathcal{K}$ and every $x \in \mathcal{X}$ we get: $\text{Eval}(k_1, x) \otimes \text{Eval}(k_2, x) = \text{Eval}(k_1 \oplus k_2, x)$.
- $\gamma = 1$ -almost key homomorphic if: $\mathcal{Y} = \mathbb{Z}_p$ if for every $k_1, k_2 \in \mathcal{K}$ and every $x \in \mathcal{X}$, there exists an error $e \in \{0, 1\}$ we get: $\text{Eval}(k_1, x) \otimes \text{Eval}(k_2, x) = \text{Eval}(k_1 \oplus k_2, x) + e$.

A.3 Distributed Key Homomorphic PRF

Definition 10 (Distributed Key Homomorphic PRF (DPRF)). A (t, m) -Distributed PRF is a tuple of PPT algorithms $\text{DPRF} := (\text{Gen}, \text{Share}, \text{Eval}, \text{P-Eval}, \text{Combine})$ with the following syntax:

- $\text{pp}_{\text{PRF}} \leftarrow \text{Gen}(1^\kappa, 1^t, 1^m)$: On input of the threshold t and number of parties m , and security parameter κ , the Gen algorithm produces the system parameter pp_{PRF} which is implicitly consumed by all the other algorithms.
- $k^{(1)}, \dots, k^{(m)} \leftarrow \text{Share}(k, t, r, m)$: On input of the number of parties m , corruption threshold t , reconstruction threshold r , and a key $k \leftarrow \mathcal{K}$, the share algorithm produces the key share for each party.
- $Y \leftarrow \text{Eval}(k, x)$: On input of the PRF key k and input x , the algorithm outputs y corresponding to some pseudorandom function $F : \mathcal{K} \times \mathcal{X} \rightarrow \mathcal{Y}$.
- $y_i \leftarrow \text{P-Eval}(k^{(i)}, x)$: On input of the PRF key share $k^{(i)}$, the partial evaluation algorithm outputs a partial evaluation y_i on input $x \in \mathcal{X}$.
- $Y' \leftarrow \text{Combine}(\{y_i\}_{i \in \mathcal{S}})$: On input of partial evaluations y_i corresponding to some subset of shares $\mathcal{S}$ such that $|\mathcal{S}| \geq t$, the algorithm outputs Y' .

We further require the following properties:

- **Correctness:** We require that the following holds for any $m, t, r, \kappa \in \mathbb{Z}$ with $t < r \leq m$ and any set $\mathcal{S} \subseteq [m]$ with $|\mathcal{S}| \geq r$, any input $x \in \mathcal{X}$:

$$\Pr \left[Y = Y' \mid \begin{array}{c} \text{pp}_{\text{PRF}} \leftarrow \text{Gen}(1^\kappa, 1^t, 1^r, 1^m), k \leftarrow \mathcal{K} \\ \{k^{(i)}\}_{i \in [m]} \leftarrow \text{Share}(k), \{y_i \leftarrow \text{P-Eval}(k^{(i)}, x)\}_{i \in \mathcal{S}} \\ Y' \leftarrow \text{Combine}(\{y_i\}_{i \in \mathcal{S}}), Y = \text{Eval}(k, x) \end{array} \right] = 1$$

- *Pseudorandomness with Static Corruptions:* We require that for any integers t, r, m with $t < r \leq m$, and for all PPT adversary $\mathcal{A}$, there exists a negligible function negl such that:

$$\Pr \left[\begin{array}{c} b = b' \\ |\mathbf{K} \cup \{j : (j, x^*) \in \mathbf{E}\}| \leq t \end{array} \middle| \begin{array}{l} \text{pp}_{\text{PRF}} \leftarrow \text{Gen}(1^\kappa, 1^t, 1^r, 1^m), k \leftarrow \mathcal{K} \\ (st, \mathbf{K}) \leftarrow \mathcal{A}(\text{pp}_{\text{PRF}}) \\ \{k^{(i)}\}_{i \in [m]} \leftarrow \text{Share}(k, t, r, m,) \\ (st, x^*) \leftarrow \mathcal{A}^{\mathcal{O}_{\text{Eval}}}(\{k^{(i)}\}_{i \in \mathbf{K}}) \\ b \leftarrow \{0, 1\}, Y_0 \leftarrow \text{Eval}(k, x^*) \\ Y_1 \leftarrow \mathcal{U}(\mathcal{Y}), b' \leftarrow \mathcal{A}^{\mathcal{O}_{\text{Eval}}}(st, Y_b) \end{array} \right] = 1$$

where:

$$\begin{array}{l} \mathcal{O}_{\text{Eval}}(i, x) \\ \mathbf{E} := \mathbf{E} \cup \{(i, x)\} \\ \text{return P-Eval}(k^{(i)}, x) \end{array}$$

- *Key Homomorphic:* We require that if $(\mathcal{K}, \oplus), (\mathcal{Y}, \otimes)$ are groups such that:

- $\forall x \in \mathcal{X}, \forall k_1, k_2 \in \mathcal{K}, \text{Eval}(k_1, x) \otimes \text{Eval}(k_2, x) = \text{Eval}(k_1 \oplus k_2, x)$, and
- $\forall x \in \mathcal{X}, \forall k_1, k_2 \in \mathcal{K}, \left\{ k_b^{(j)} \leftarrow \text{Share}(k_b) \right\}_{j \in [m], b \in \{1, 2\}}$,

$$\forall j \in [m], y_{1,2}^{(j)} := \left(\text{P-Eval}(k_1^{(j)}, x) \otimes \text{P-Eval}(k_2^{(j)}, x) \right), \text{ and } \forall \mathcal{S} \subseteq [m] \text{ with } |\mathcal{S}| \geq r, \text{ Combine} \\ \left(\left\{ y_{1,2}^{(j)} \right\}_{j \in \mathcal{S}} \right) = \text{Eval}(k_1 \oplus k_2, x)$$

A.4 Class Groups Framework

Class group-based cryptography is a cryptographic technique that originated in the late 1980s, with the idea that the class group of ideals of maximal orders of imaginary quadratic fields may be more secure than the multiplicative group of finite fields [22, 74]. The CL framework was first introduced by the work of Castagnos and Laguillaumie [31]. This framework operates on a group where there exists a subgroup with support for efficient discrete logarithm construction. Subsequent works [32, 28, 29, 97, 20] have refined the original framework. The framework has been used in various applications over the years [32, 28, 29, 101, 43, 51, 96, 20, 60]. Meanwhile, class group cryptography itself has been employed in numerous applications [99, 100, 67, 17, 30, 33, 62, 23, 6, 1, 41, 40, 7].

Broadly, the framework is defined by two functions - CLGen, CLSolve with the former outputting a tuple of public parameters. The elements of this framework are the following:

- *Input Parameters:* κ_c is the computational security parameter, κ_s is a statistical security parameter, a prime p such that $p > 2^{\kappa_c}$, and uniform randomness ρ that is used by the CLGen algorithm and is made public.
- *Groups:* $\widehat{\mathbb{G}}$ is a finite multiplicative abelian group, $\mathbb{G}$ is a cyclic subgroup of $\widehat{\mathbb{G}}$, $\mathbb{F}$ is a subgroup of $\mathbb{G}$, $H = \{x^p, x \in \mathbb{G}\}$
- *Orders:* $\mathbb{F}$ has order p , $\widehat{\mathbb{G}}$ has order $p \cdot \widehat{s}$, $\mathbb{G}$ has order $p \cdot s$ such that s divides $\widehat{s}$ and $\gcd(p, \widehat{s}) = 1, \gcd(p, s) = 1$, H has order s and therefore $\mathbb{G} = \mathbb{F} \times H$.
- *Generators:* f is the generator of $\mathbb{F}$, g is the generator of $\mathbb{G}$, and h is the generator of H with the property that $g = f \cdot h$

- *Upper Bound*: Only an upper bound $\bar{s}$ of $\hat{s}$ (and s) is provided.
- *Additional Properties*: Only encodings of $\hat{\mathbb{G}}$ can be recognized as valid encodings and $s, \hat{s}$ are unknown.
- *Distributions*: $\mathcal{D}$ (resp. $\mathcal{D}_p$) be a distribution over the set of integers such that the distribution $\{g^x : x \leftarrow^s \mathcal{D}\}$ (resp. $\{g_p^x : x \leftarrow^s \mathcal{D}_p\}$) is at most distance $2^{-\kappa_s}$ from the uniform distribution over $\mathbb{G}$ (resp. H).
- *Additional Group and its properties*: $\hat{\mathbb{G}}^p = \{x^p, x \in \hat{\mathbb{G}}\}$, $\hat{\mathbb{G}}$ factors as $\hat{\mathbb{G}}^p \times \mathbb{F}$.¹⁰ Let $\bar{\omega}$ be the group exponent of $\hat{\mathbb{G}}^p$. Then, the order of any $x \in \hat{\mathbb{G}}^p$ divides $\bar{\omega}$.¹¹

Remark 4. The motivations behind these additional distributions are as follows. One can efficiently recognize valid encodings of elements in $\hat{\mathbb{G}}$ but not $\mathbb{G}$. Therefore, a malicious adversary $\mathcal{A}$ can run our constructions by inputting elements belonging to $\hat{\mathbb{G}}^p$ (rather than in H). Unfortunately, this malicious behavior cannot be detected which allows $\mathcal{A}$ to obtain information on the sampled exponents modulo $\bar{\omega}$ (the group exponent of $\hat{\mathbb{G}}^p$). By requiring the statistical closeness of the induced distribution to uniform in the aforementioned groups allows flexibility in proofs. Note that the assumptions do not depend on the choice of these two distributions. Further, the order s of H and group exponent $\bar{\omega}$ of $\hat{\mathbb{G}}^p$ are unknown and the upper bound $\bar{s}$ is used to instantiate the aforementioned distribution. Specifically, looking ahead we will set $\mathcal{D}_H$ to be the uniform distribution over the set of integers $[B]$ where $B = 2^{\kappa_s} \cdot \bar{s}$. Using Lemma 8, we get that the distribution is less than $2^{-\kappa_s}$ away from uniform distribution in H . In our constructions we will set $\kappa_s = 40$. We will make this sampling more efficient for our later constructions. We refer the readers to Tucker [97, §3.1.3, §3.7] for more discussions about this instantiation. Finally, as stated we will also set $\hat{\mathcal{D}} = \mathcal{D}$ and $\hat{\mathcal{D}}_H = \mathcal{D}_H$.

We also have the following lemma from Castagnos, Imbert, and Laguillaumie [30] which defines how to sample from a discrete Gaussian distribution.

Lemma 8. *Let $\mathbb{G}$ be a cyclic group of order n , generated by g . Consider the random variable X sampled uniformly from $\mathbb{G}$; as such it satisfies $\Pr[X = h] = \frac{1}{n}$ for all $h \in \mathbb{G}$. Now consider the random variable Y with values in $\mathbb{G}$ as follows: draw y from the discrete Gaussian distribution $\mathcal{D}_{\mathbb{Z}, \sigma}$ with $\sigma \geq n \sqrt{\frac{\ln(2(1+1/\epsilon))}{\pi}}$ and set $Y := g^y$. Then, it holds that:*

$$\Delta(X, Y) \leq 2\epsilon$$

Remark 5. By definition, the distribution $\{g^x : x \leftarrow^s \mathcal{D}\}$ is statistically indistinguishable from $\{g^y : y \leftarrow^s \{0, \dots, p \cdot s - 1\}\}$. Therefore, it follows that $\{x \bmod p \cdot s : x \leftarrow^s \mathcal{D}\}$ is statistically indistinguishable from $\{x : x \leftarrow^s \{0, \dots, p \cdot s - 1\}\}$. Similarly, $\{x \bmod s : x \leftarrow^s \mathcal{D}_p\}$ is statistically indistinguishable from $\{x : x \leftarrow^s \{0, \dots, s - 1\}\}$. Furthermore, sampling a value x corresponding to $\mathcal{D}$ is statistically indistinguishable from the uniform distribution in $\{0, \dots, s - 1\}$ because s divides $p \cdot s$.

Definition 11 (Class Group Framework). *The framework is defined by two algorithms (CLGen, CLSolve) such that:*

- $\text{pp}_{\text{CL}} = (p, \kappa_c, \kappa_s, \bar{s}, f, h, \hat{\mathbb{G}}, \mathbb{F}, \mathcal{D}, \mathcal{D}_H, \hat{\mathcal{D}}, \hat{\mathcal{D}}_H, \rho) \leftarrow^s \text{CLGen}(1^{\kappa_c}, 1^{\kappa_s}, p; \rho)$
- *The DL problem is easy in $\mathbb{F}$, i.e., there exists a deterministic polynomial algorithm CLSolve that solves the discrete logarithm problem in $\mathbb{F}$:*

$$\Pr \left[x = x' \mid \begin{array}{l} \text{pp}_{\text{CL}} = \leftarrow^s \text{CLGen}(1^{\kappa_c}, 1^{\kappa_s}, p; \rho) \\ x \leftarrow^s \mathbb{Z}/p\mathbb{Z}, X = f^x; \\ x' \leftarrow \text{CLSolve}(\text{pp}_{\text{CL}}, X) \end{array} \right] = 1$$

¹⁰Recall that p and $\hat{s}$ are co-prime.

¹¹This follows from the property that the exponent of a finite Abelian group is the least common multiple of its elements.

Definition 12 (Hard Subgroup Membership Assumption (HSM_M) Assumption [32]). Let κ be a the security parameter with prime p such that $|p| \geq \kappa$. Let $(CLGen, CLSolve)$ be the group generator algorithms as defined in Definition 11, then the HSM_M assumption requires that that HSM_M problem be hard in $\mathbb{G}$ even with access to the Solve algorithm. More formally, let $\mathcal{D}$ (resp. $\mathcal{D}_p$) be a distribution over the set of integers such that the distribution $\{g^x : x \leftarrow \mathcal{D}\}$ (resp. $\{g_p^x : x \leftarrow \mathcal{D}_p\}$) is at most distance $2^{-\kappa}$ from the uniform distribution over $\mathbb{G}$ (resp. H). Then, we say that the HSM_M problem is hard if for all PPT adversaries $\mathcal{A}$, there exists a negligible function $\text{negl}(\kappa)$ such that:

$$\Pr \left[b = b' \mid \begin{array}{l} \text{pp}_{CL} \leftarrow \text{CLGen}(1^{\kappa_c}, 1^{\kappa_s}, p; \rho) \\ x \leftarrow \mathcal{D}, x' \leftarrow \mathcal{D}_p \\ b \leftarrow \{0, 1\}; Z_0 = g^x; Z_1 = g_p^{x'} \\ b' \leftarrow \mathcal{A}^{\text{Solve}(\text{pp}_{CL}, \cdot)}(\text{pp}_{CL}, Z_b) \end{array} \right] \leq \frac{1}{2} + \text{negl}(\kappa)$$

When dealing with groups of known order, one can sample elements in a group $\mathbb{G}$ easily by merely sampling exponents modulo the group order and then raising the generator of the group to that exponent. Unfortunately, note that here neither the order of $\mathbb{G}$ (i.e., ps) nor that of H (i.e. s) is known. Therefore, we instead use the knowledge of the upper-bound $\bar{s}$ of s to instantiate the distributions $\mathcal{D}$ and $\mathcal{D}_p$ respectively. This choice of choosing from the distributions $\mathcal{D}$ and $\mathcal{D}_p$ respectively allows for flexibility of various proofs.

B Constructions in CL Framework

Construction 8 (PRF in CL Framework). Let $(CLGen, CLSolve)$ be the class group framework as defined in Definition 11. Then, let $\text{pp}_{CL} \leftarrow \text{CLGen}(1^{\kappa_c}, 1^{\kappa_s})$. Further, let $H : \mathcal{X} \rightarrow \mathbb{H}$ be a hash function. Then, consider the following definition of $\mathcal{K} = \mathcal{D}_H$, $\mathcal{X} = \{0, 1\}^*$, $\mathcal{Y} = \mathbb{H}$, $F_{CL}(k, x) = H(x)^k$.

Theorem 9. *Construction 8 is a secure PRF where H is modeled as a random oracle under the HSM_M assumption.*

The proof is deferred to Section E.2

B.1 Distributed PRF in CL Framework

We build our construction of Distributed PRF from the Linear Secret Sharing Scheme $LSS := (\text{Share}, \text{GetCoeff}, \text{Reconstruct})$ with pp_{SS} denoting the public parameters of the LSS scheme. We specifically employ the Shamir Secret Sharing scheme over the Integers, as defined in [20].

Construction 9 (Distributed PRF in CL Framework). A (r, m) -Distributed PRF is a tuple of PPT algorithms $DPRF := (\text{Gen}, \text{Share}, \text{Eval}, \text{P-Eval}, \text{Combine})$ with the algorithms as defined in Figure 8. For simplicity, in the construction below we will set the corruption threshold $t = r - 1$. Though, the construction also holds for a lower t .

Theorem 10. *In the Random Oracle Model, if Construction 8 is a secure pseudorandom function if Integer Secret Sharing is statistically private, then Construction 9 is pseudorandom in the static corruptions setting.*

The proof of security and correctness can be found in Section E.3.

B.2 Construction of One-shot Private Aggregation without Leakage Simulation in CL Framework

Construction 10. We present the construction of One-shot Private Aggregation without Leakage Simulation, in the CL Framework in Figure 9.

Protocol Distributed PRF

<u>Gen($1^\kappa, 1^r, 1^m$)</u> Parse $\kappa = (\kappa_s, \kappa_c)$ Run $\text{pp}_{\text{CL}} \leftarrow \text{CLGen}(1^{\kappa_c}, 1^{\kappa_s})$ Set $\ell_s := \mathcal{M}$ Sample $\mathbb{H} : \mathcal{X} \rightarrow \mathbb{H}$ return $\text{pp}_{\text{PRF}} := (\text{pp}_{\text{CL}}, \text{LISS.pp}_{\text{SS}}, \mathbb{H})$ <u>Share(k, r, m)</u> Run $k^{(1)}, \dots, k^{(m)} \leftarrow \text{LISS.Share}(k, r, m)$ return $k^{(1)}, \dots, k^{(m)}$	<u>Eval(k, x)</u> Compute $Y = \mathbb{H}(x)^{\Delta^3 \cdot k}$ return Y <u>P-Eval($k^{(i)}, x$)</u> Compute $y_i = \mathbb{H}(x)^{\Delta \cdot k^{(i)}}$ return y_i <u>Combine($\{y_i\}_{i \in \mathcal{S}}$)</u> Run $\Lambda_{i \in \mathcal{S}} = \text{LISS.GetCoeff}(\mathcal{S})$ Compute $Y' = \prod_{i \in \mathcal{S}} y_i^{\Lambda_i}$ return Y'
---	---

Figure 8: Construction of Distributed PRF based on the LISS := (Share, GetCoeff, Reconstruct) scheme of [20], with pp_{SS} denoting the public parameters of the LISS scheme. Recall that the offset $\Delta := m!$ where m is the number of shares generated.

C Constructions of Leakage Resilient Key-Homomorphic Pseudorandom Functions

C.1 Construction from LWR Assumption

We also have the construction from Boneh *et al.* [18] of an *almost* Key Homomorphic PRF from LWR in the Random Oracle model which was later formally proved secure by Ernst and Koch [46] with $\gamma = 1$.

Construction 11 (Key Homomorphic PRF from LWR). Let $\mathbb{H} : \mathcal{X} \rightarrow \mathbb{Z}_q^\lambda$. Then, define the efficiently computable function $F_{\text{LWR}} : \mathbb{Z}_q^\lambda \times \mathcal{X} \rightarrow \mathbb{Z}_p$ as $\lfloor \langle \mathbb{H}(x), \mathbf{k} \rangle \rfloor_p$. F_{LWR} is an almost key homomorphic PRF with $\gamma = 1$.

Construction 12 (Length Extended Key-Homomorphic from LWR). Let F_{LWR} be the function as defined in Construction 11. Then, consider $F_{\text{LWR}}^L := (F_{\text{LWR}}(\mathbf{k}, (x, 1)), \dots, F_{\text{LWR}}(\mathbf{k}, (x, L)))$.

It is easy to see that Construction 12 is a secure pseudorandom function. An adversary that can break the security of Construction 12 can be used to break the security of Construction 11.

Theorem 11 (Leakage Resilience of Construction 12). *Let PRF be the PRF defined in Construction 12. Recall that $\text{PRF}.\mathcal{K} = \mathbb{Z}_q^\lambda$. Then, it is leakage resilient in the following sense:*

$$\{\text{PRF}(\mathbf{k}, x), (\mathbf{k} + r) \bmod q : \mathbf{k}, r \leftarrow \mathcal{K}\} \approx_c \{Y, (\mathbf{k} + r) \bmod q : Y \leftarrow \mathcal{Y}, \mathbf{k}, r \leftarrow \mathcal{K}\}$$

Proof. The proof proceeds through a sequence of hybrids.

Hybrid₀(κ): The left distribution is provided to the adversary. In other words, the adversary gets:

$$\{\text{PRF}(\mathbf{k}, x), (\mathbf{k} + r) \bmod q : \mathbf{k}, r \leftarrow \mathcal{K}\}$$

Hybrid₁(κ): In this hybrid, we replace $(\mathbf{k} + r) \bmod q$ with a uniformly random value $\mathbf{k}' \leftarrow \mathcal{K}$.

$$\{\text{PRF}(\mathbf{k}, x), \mathbf{k}' : \mathbf{k}, \mathbf{k}' \leftarrow \mathcal{K}\}$$

Note that $(\mathbf{k} + r) \bmod q$ and $\mathbf{k}'$ are identically distributed. Therefore, the Hybrid₀, Hybrid₁ are identically distributed.

Protocol OPA'

System Parameters Generation

Run $\text{pp} \leftarrow \text{CL.Gen}(1^\kappa, 1^t, 1^r, 1^m)$

Server Initiating Iteration ℓ

Select n clients $\mathcal{C}_\ell$.

Select $m = \lfloor \log_2 n \rfloor$ -sized committee

Broadcast to n clients the committee.

Data Encryption Phase by Client i in iteration ℓ

Input: $\mathbf{x}_{i,\ell} = (x_{i,\ell}^{(1)}, \dots, x_{i,\ell}^{(L)})$

Sample $k_{i,1}, \dots, k_{i,L} \leftarrow \text{DPRF}.\mathcal{K}$

for $in = 1, \dots, L$ **do**

$h_{i,\ell}^{(in)} = \text{DPRF.Eval}(k_{i,in}, \ell)$

Compute $\text{ct}_{i,\ell}^{(in)} = f^{x_{i,\ell}^{(in)}} \cdot h_{i,\ell}^{(in)}$

Compute $\{k_{i,in}^{(j)}\}_{j \in [m]} \leftarrow \text{DPRF.Share}(k_{i,in}, t, r, m)$

For $j \in [m]$, set $\{\text{aux}_{i,\ell}^{(j,in)}\} = \text{DPRF.P-Eval}(k_{i,in}^{(j)}, \ell)$

Send $\text{ct}_{i,\ell}^{(1)}, \dots, \text{ct}_{i,\ell}^{(L)}$ to the Server

Send $\text{aux}_{i,\ell}^{(j,1)}, \dots, \text{aux}_{i,\ell}^{(j,L)}$ to committee member $j \forall j \in [m], i \in \mathcal{C}$, via Server appropriately encrypted

Server Forwards in iteration ℓ

Let $\mathcal{C}$ be the clients who sent the prescribed messages.

Send encrypted $\text{aux}_{i,\ell}^{(j)}$ to committee member $j \forall j \in [m], i \in \mathcal{C}$

Data Combination Phase by Committee Member j in iteration ℓ

Input: $\{\text{aux}_{i,\ell}^{(j,1)}, \dots, \text{aux}_{i,\ell}^{(j,L)}\}_{i \in \mathcal{C}}$

for $in = 1, \dots, L$ **do**

$(\text{AUX}_\ell^{(j,in)}) \leftarrow \otimes_{i \in \mathcal{C}} (\text{aux}_{i,\ell}^{(j,in)})$

Send $\text{AUX}_\ell^{(j,1)}, \dots, \text{AUX}_\ell^{(j,L)}$ to Server

Data Aggregation Phase by Server in iteration ℓ

Input: $\left\{ \left\{ \text{AUX}_\ell^{(j,in)} \right\}_{j \in \mathcal{S}}, \left\{ \text{ct}_{i,\ell}^{(in)} \right\}_{i \in \mathcal{C}} \right\}_{in \in [L]}, |\mathcal{S}| \geq t.$

for $in = 1, \dots, L$ **do**

Run $\text{AUX}_\ell^{(in)} \leftarrow \text{DPRF.Combine}(\{\text{AUX}_\ell^{(j,in)}\}_{j \in \mathcal{S}})$

Run $X_\ell^{(in)} \leftarrow \text{CLSolve}(\text{pp}, (\text{AUX}_\ell^{(in)})^{-1} \cdot \prod_{i \in \mathcal{C}} \text{ct}_{i,\ell}^{(in)})$

return $\{X_\ell^{(in)}\}_{in \in [L]}$

Figure 9: In this figure, we present the modified steps for OPA'. For simplicity, we present only the semi-honest construction. For malicious security, we augment similarly with a second mask.

Hybrid₂(κ): In this hybrid, we will replace the PRF computation with a random value from the range.

$$\{Y, k' : Y \leftarrow \mathcal{Y}, k' \leftarrow \mathcal{K}\}$$

Under the security of the PRF, we get that Hybrid₁, Hybrid₂ are computationally indistinguishable.

Hybrid₃(κ): We replace k' with $(k + r) \bmod q$.

$$\{Y, (k + r) \bmod q : Y \leftarrow \mathcal{Y}, k, r \leftarrow \mathcal{K}\}$$

As argued before Hybrid₂, Hybrid₃ are identically distributed.

Note that Hybrid₃ is the right distribution from the theorem statement. This completes the proof. $\square$

C.2 Learning with Errors Assumption

Construction 13 (Key Homomorphic PRF from LWE). Let $H_A : \mathcal{X} \rightarrow \mathbb{Z}_q^{L \times \lambda}$. Then, define the efficiently computable function $F_{LWE} : (\mathbb{Z}_q^\lambda \times \chi^L) \times \mathcal{X} \rightarrow \mathbb{Z}_q^L$ as $F_{LWE}((\mathbf{k}, \mathbf{e}), x) := H_A(x)\mathbf{k} + \mathbf{e}$. F_{LWE} is an almost key homomorphic PRF.

Remark 6. We note that it has been shown that one can also sample $\mathbf{x}$ from the same distribution as $\mathbf{e}$. This is known as the short-secret LWE assumption and was employed in the encryption scheme of Lyubashevsky *et al.* [71]. An immediate consequence of this assumption is that one can set $\mathbf{A}$'s dimension m to be much smaller than what is required for the LWE assumption.

Similar to the LWR construction, we need to show that the PRF based on LWE assumption is also leakage resilient. Unfortunately, a similar proof technique does not work because the construction also suffers from leakage on the error vector which are usually Gaussian secrets. Instead, we rely on the Hint-LWE Assumption (Definition 4, as before).

D Construction from LWR

D.1 Distributed PRF from LWR

Let us revisit Construction 11. First, observe that the key space is from $\mathbb{Z}_q^p$ which implies that the order of $\mathcal{K}$ is known. Further, the computation occurs over a group whose structure and order is known. This is a departure from the construction based on the HSM_M assumption. Consequently, by assuming that both p and q are primes, one can avoid integer secret sharing but instead rely on traditional Shamir's Secret Sharing over a field, which we defined earlier (see Construction 4).

We saw earlier that Construction 11 was only almost key homomorphic, i.e.:

$$F(\mathbf{k}_1 + \mathbf{k}_2, x) = F(\mathbf{k}_1, x) + F(\mathbf{k}_2, x) + e$$

where $e \in \{0, 1\}$. It also follows that:

$$T \cdot F(\mathbf{k}_1, x) = F(T \cdot \mathbf{k}_0, x) - e_T$$

where $e_T \in \{0, \dots, T\}$. This becomes a cause for concern as, in the threshold construction using the Shamir Secret Sharing over the field as shown in Construction 4, one often recombinates by multiplying with a Lagrange coefficient λ_{i_j} . Unfortunately, multiplying the result by λ_{i_j} implies that the error term $e_{\lambda_{i_j}} \in \{0, \dots, i_j\}$. The requirement is that this error term should not become "too large". However, interpreting Lagrange coefficients as elements in $\mathbb{Z}_p$ results in the error term failing to be low-norm leading to error propagation. To mitigate this, we use techniques quite similar to Construction 6 by essentially clearing the denominator by multiplying with $\Delta := m!$. This is a technique made popular by the work of Shoup [92] and later used in several other works including in the context of lattice-based cryptography by Agrawal *et al.* [4] and later to construct a distributed key homomorphic PRF from any almost key homomorphic PRF by Boneh *et al.* [18].¹² Then, the combine algorithm will simply multiply all partial evaluations with Δ as well.

Construction 14 (Distributed Almost Key Homomorphic PRF from LWR). A (t, m) -Distributed PRF is a tuple of PPT algorithms $DPRF := (\text{Gen}, \text{Share}, \text{Eval}, \text{P-Eval}, \text{Combine})$ with the algorithms as defined in Figure 10.

Issues with the Construction from Boneh *et al.* [18, §7.1.1]. As remarked earlier, their generic construction suffers from issues stemming from their security reduction. Specifically, their security reduction proceeds similarly to the proof of Theorem 10 and requires $\mathcal{B}$ to answer honest evaluation queries for key

¹²However, their generic construction is incorrect, owing to issues in their security proof which is not entirely sketched out. We fix the issues in our construction.

Protocol Distributed PRF from Learning with Rounding Assumption

<u>Gen</u>($1^\kappa, 1^t, 1^m$) Parse $\kappa = (\rho)$ Run $\text{pp}_{\text{LWR}} = (\rho, q, p) \leftarrow \text{LWRGen}(1^\rho)$ Set $\ell_s := q$ Set u such that $\lfloor p/u \rfloor > (\Delta + 1)t\Delta$ Set v such that $\lfloor u/v \rfloor > \Delta t$ Sample $\mathbb{H} : \mathcal{X} \rightarrow \mathbb{Z}_q^\rho$ return $\text{pp}_{\text{PRF}} := (\text{pp}_{\text{LWR}}, \text{pp}_{\text{SS}}, \mathbb{H}, u)$ <u>Share</u>($\mathbf{k} \in \mathbb{Z}_q^\rho, t, m$) for $i = 1, \dots, \rho$ do $k_i^{(1)}, \dots, k_i^{(m)} \leftarrow \text{SS.SecretShare}(k_i, t, m)$ return $\{\mathbf{k}^{(j)} = (k_1^{(j)}, \dots, k_\rho^{(j)})\}_{j \in [m]}$	<u>Eval</u>($\mathbf{k}, x$) Compute $Y = \left\lfloor \Delta \left\lfloor \Delta \left\lfloor \langle \mathbb{H}(x), \mathbf{k} \rangle \right\rfloor_p \right\rfloor_u \right\rfloor_v$ return Y <u>P-Eval</u>($\mathbf{k}^{(i)}, x$) Compute $y_i = \left\lfloor \Delta \left\lfloor \langle \mathbb{H}(x), \mathbf{k}^{(i)} \rangle \right\rfloor_p \right\rfloor_u$ return y_i <u>Combine</u>($\{y_i\}_{i \in \mathcal{S}}$) Run $\lambda_{i \in \mathcal{S}} = \text{SS.Coeff}(\mathcal{S})$ Compute $Y' = \left\lfloor \sum_{i \in \mathcal{S}} \Delta \lambda_i \cdot y_i \right\rfloor_v$ return Y'
--	---

Figure 10: Construction of Distributed PRF based on the Secret Sharing scheme of Construction 4 where $\text{SS} = (\text{SecretShare}, \text{Coeff})$ with pp_{SS} denoting the public parameters of the secret sharing scheme.

indices for which it does not know the actual key share. Their explanation suggests that we again use the “clearing out the denominator” trick by multiplying with Δ . However, the issue is that the resulting response will be of the form $\Delta \cdot F(k_{i^*}, x)$ for i^* , unknown to $\mathcal{B}$. Consequently, one has to change the partial evaluation response to also include this offset to ensure the correctness of reduction. This would imply that the **Combine** algorithm will multiply with Δ again, which would thus result in the actual **Eval** algorithm having an offset of Δ^2 . Furthermore, the partial evaluation algorithm should also have to round down to the elements in $[0, u - 1]$ for the same reason that the **Combine** algorithm required this fix.

Correctness.

$$\begin{aligned}
\left\lfloor \Delta \left\lfloor \langle \mathbb{H}(x), \mathbf{k} \rangle \right\rfloor_p \right\rfloor_u &= \left\lfloor \Delta \left\lfloor \sum_{z=1}^{\rho} H(x, z) \cdot k_z \right\rfloor_p \right\rfloor_u \\
&= \left\lfloor \Delta \left\lfloor \sum_{z=1}^{\rho} H(x, z) \sum_{i_j \in \{i_1, \dots, i_t\}} \lambda_{i_j} s_z^{(i_j)} \right\rfloor_p \right\rfloor_u \\
&= \left\lfloor \Delta \left\lfloor \sum_{i_j \in \{i_1, \dots, i_t\}} \sum_{z=1}^{\rho} H(x, z) \cdot \lambda_{i_j} \cdot s_z^{(i_j)} \right\rfloor_p \right\rfloor_u \\
&= \left\lfloor \Delta \left\lfloor \sum_{i_j \in \{i_1, \dots, i_t\}} \langle \mathbb{H}(x), \lambda_{i_j} \mathbf{k}^{(i_j)} \rangle \right\rfloor_p \right\rfloor_u \\
&= \left\lfloor \Delta \left(e_t + \sum_{i_j \in \{i_1, \dots, i_t\}} \left\lfloor \lambda_{i_j} \langle \mathbb{H}(x), \mathbf{k}^{(i_j)} \rangle \right\rfloor_p \right) \right\rfloor_u \\
&= \left\lfloor \Delta \left(e_t + \sum_{i_j \in \{i_1, \dots, i_t\}} e_{\lambda_{i_j}} + \lambda_{i_j} \left\lfloor \langle \mathbb{H}(x), \mathbf{k}^{(i_j)} \rangle \right\rfloor_p \right) \right\rfloor_u \\
&= \left\lfloor \Delta \left(e_t + \sum_{i_j \in \{i_1, \dots, i_t\}} e_{\lambda_{i_j}} \right) + \sum_{i_j \in \{i_1, \dots, i_t\}} \Delta \cdot \lambda_{i_j} \left\lfloor \langle \mathbb{H}(x), \mathbf{k}^{(i_j)} \rangle \right\rfloor_p \right\rfloor_u \\
&= \left\lfloor \sum_{i_j \in \{i_1, \dots, i_t\}} \Delta \cdot \lambda_{i_j} \left\lfloor \langle \mathbb{H}(x), \mathbf{k}^{(i_j)} \rangle \right\rfloor_p \right\rfloor_u
\end{aligned}$$

The last step follows provided the error term is small. Recall that $e_t \in \{0, \dots, t\}$ and $e_{\lambda_{i_j}} \in \{0, \dots, \lambda_{i_j}\}$. Now observe that we multiply with Δ and λ_{i_j} has a maximum value Δ . Therefore, $\Delta \cdot e_{\lambda_{i_j}} < \Delta^2$. Therefore, the size of the error term is $\leq t \cdot \Delta + t \cdot \Delta^2$. Therefore, provided u is chosen such that $\lfloor p/u \rfloor > (\Delta + 1) \cdot t \cdot \Delta$, then the last step is correct. Now, we have:

$$\begin{aligned}
\left\lfloor \Delta \left\lfloor \langle H(x), \mathbf{k} \rangle \right\rfloor_p \right\rfloor_u &= \left\lfloor \sum_{i_j \in \{i_1, \dots, i_t\}} \Delta \cdot \lambda_{i_j} \left\lfloor \langle H(x), \mathbf{k}^{(i_j)} \rangle \right\rfloor_p \right\rfloor_u \\
\left\lfloor \Delta \left\lfloor \Delta \left\lfloor \langle H(x), \mathbf{k} \rangle \right\rfloor_p \right\rfloor_u \right\rfloor_v &= \left\lfloor \Delta \left\lfloor \sum_{i_j \in \{i_1, \dots, i_t\}} \Delta \cdot \lambda_{i_j} \left\lfloor \langle H(x), \mathbf{k}^{(i_j)} \rangle \right\rfloor_p \right\rfloor_u \right\rfloor_v \\
&= \left\lfloor \Delta \left(e_t + \sum_{i_j \in \{i_1, \dots, i_t\}} \lambda_{i_j} \left\lfloor \Delta \cdot \left\lfloor \langle H(x), \mathbf{k}^{(i_j)} \rangle \right\rfloor_p \right\rfloor_u \right) \right\rfloor_v \\
&= \left\lfloor \Delta \left(e_t + \sum_{i_j \in \{i_1, \dots, i_t\}} \lambda_{i_j} \text{P-Eval}(\mathbf{k}^{(i_j)}, x) \right) \right\rfloor_v \\
&= \left\lfloor \sum_{i_j \in \{i_1, \dots, i_t\}} \lambda_{i_j} \cdot \Delta \cdot \text{P-Eval}(\mathbf{k}^{(i_j)}, x) \right\rfloor_v \\
&= \text{Combine}(\{ \text{P-Eval}(\mathbf{k}^{(i_j)}, x)_{i_j \in \{i_1, \dots, i_t\}} \})
\end{aligned}$$

provided $\lfloor u/v \rfloor > t\Delta$.

Pseudorandomness. The proof of pseudorandomness follows the outline of the proof of Theorem 10 but with some important differences. First, we do not rely on integer secret sharing but rather plain secret sharing over the field. Therefore, the Lagrange coefficients correspond to λ_{i_j} . Or more formally, to respond to a partial evaluation query at point x_j with target key index i^* , the adversary $\mathcal{B}$ does the following:

- Use its oracle to get partial evaluation on x_j at i_t , which we call as $h_{j,t}$.
- Then, use Lagrange coefficients but with suitably multiplying with Δ to compute the correct distribution by rounding down to u . The choice of u guarantees that the response is correct.

For challenge query, it simply does two rounding down, first to u and then to v .

Verification of Almost Key Homomorphism. Let $\mathbf{k}_1, \mathbf{k}_2$ be two keys that are shared. Now, let the key shares received by some party i_j be $\mathbf{k}_1^{(i_j)}$ and $\mathbf{k}_2^{(i_j)}$. Then,

$$\begin{aligned}
\text{P-Eval}(\mathbf{k}_1^{(i_j)}, x) + \text{P-Eval}(\mathbf{k}_2^{(i_j)}, x) &= \left\lfloor \Delta \left\lfloor \langle H(x), \mathbf{k}_1^{(i_j)} \rangle \right\rfloor_p \right\rfloor_u + \left\lfloor \Delta \left\lfloor \langle H(x), \mathbf{k}_2^{(i_j)} \rangle \right\rfloor_p \right\rfloor_u \\
&= \left\lfloor \Delta \left\lfloor \langle H(x), \mathbf{k}_1^{(i_j)} \rangle \right\rfloor_p + \Delta \left\lfloor \langle H(x), \mathbf{k}_2^{(i_j)} \rangle \right\rfloor_p \right\rfloor_u - e_1 \\
&= \left\lfloor \Delta \left\lfloor \langle H(x), \mathbf{k}_1^{(i_j)} \rangle + \langle H(x), \mathbf{k}_2^{(i_j)} \rangle \right\rfloor_p \right\rfloor_u - 2e_1 \\
&= \text{P-Eval}(\mathbf{k}_1^{(i_j)} + \mathbf{k}_2^{(i_j)}, x) - 2e_1
\end{aligned}$$

It follows that for n such keys:

$$\sum_{i=1}^n \text{P-Eval}(\mathbf{k}_i^{(i_j)}, x) = \text{P-Eval}(\sum_{i=1}^n \mathbf{k}_i^{(i_j)}, x) - n \cdot e_1$$

This shows that the P-Eval is almost key-homomorphic. Consequently, one can verify that the whole Eval procedure is almost key homomorphic for the appropriate error function. To do this, recall that from the correctness of our algorithm:

$$\text{Share}(\mathbf{k}, m, t) = \left\{ \mathbf{k}^{(i)} \right\}_{i \in [m]}, \text{Combine}(\left\{ \text{Eval}(\mathbf{k}^{(i_j)}, x) \right\}_{i \in \{i_1, \dots, i_t\}}) = \text{Eval}(\mathbf{k}, x)$$

In other words, for $\mathbf{k}_1, \mathbf{k}_2 \in \mathcal{K}$, $\text{Share}(\mathbf{k}_1, m, t) = \left\{ \mathbf{k}_1^{(i)} \right\}_{i \in [m]}$ and $\text{Share}(\mathbf{k}_2, m, t) = \left\{ \mathbf{k}_2^{(i)} \right\}_{i \in [m]}$, we will have for $i \in [m]$, $\text{Eval}(\mathbf{k}_1^{(i)}, x), \text{Eval}(\mathbf{k}_2^{(i)}, x) = \text{Eval}(\mathbf{k}_1^{(i)} + \mathbf{k}_2^{(i)}, x) - 2 \cdot e_1$.

D.2 OPA' Construction based on LWR Assumption

We build OPA based on the LWR Assumption, building it based on the Key Homomorphic, Distributed PRF as presented in Construction 11. However, our construction is largely different from the template followed to build OPA from the HSM_M assumption. This is primarily because of the growth in error when combining partial evaluations. Specifically, will get that $\text{P-Eval}(\sum_{i=1}^n \mathbf{k}_i^{(j)}, x) = \sum_{i=1}^n \text{P-Eval}(\mathbf{k}_i^{(j)}, x) + e$ where $e \in \{0, \dots, n-1\}$ where n is the number of clients participating for that label. This would require us to round down to a new value u' such that $\lfloor u/u' \rfloor > n-1$. Therefore, while we still employ the underlying functions of the distributed, key-homomorphic PRF based on LWR, we have to open up the generic reduction. For simplicity, we detail the construction for $L = 1$. Then, these are the differences:

- As done fore Construction 2, the input is encoded as $x_i \cdot n + 1$.
- Specifically, the client's share to the committee will only be the first level of the evaluation, i.e., rounded down to p .
- Then, committee member j will then add the shares up, multiply with the offset, and then round down to u . We will show that provided $\lfloor p/u \rfloor > \Delta \cdot n$, this is consistent with $\text{P-Eval}(\sum_{i=1}^n \mathbf{k}_i^{(j)}, x)$.
- The decoding algorithm is also similar to the one from Construction 2.

Recall that DPRF correctness requires that $\lfloor p/u \rfloor > t \cdot \Delta + t \cdot \Delta^2$, and so one just needs $\lfloor p/u \rfloor > \max(t \cdot \Delta + t \cdot \Delta^2, n \cdot \Delta)$. Then, one can rely on the correctness of DPRF as shown below to argue that when the server runs DPRF.Combine , the output is $\text{Eval}(\sum_{i=1}^n \mathbf{k}_i, x)$.

Correctness. We showed how the output of the server's invocation of DPRF.Combine is $\text{Eval}(\sum_{i=1}^n \mathbf{k}_i, x)$. Now, let us look at the remaining steps:

$$\begin{aligned} X_\ell &= \sum_{i=1}^n \text{ct}_{i,\ell} - \text{AUX}_\ell \\ &= \sum_{i=1}^n (x_{i,\ell} * n + 1) + \text{Eval}(\mathbf{k}_i, \ell) - \text{Eval}(\sum_{i=1}^n \mathbf{k}_i, \ell) \bmod v \\ &= n \cdot \sum_{i=1}^n x_{i,\ell} + n + \sum_{i=1}^n \text{Eval}(\mathbf{k}_i, \ell) - \text{Eval}(\sum_{i=1}^n \mathbf{k}_i, \ell) \bmod v \\ &= n \cdot \sum_{i=1}^n x_{i,\ell} + n + \text{Eval}(\sum_{i=1}^n \mathbf{k}_i, \ell) - \text{Eval}(\sum_{i=1}^n \mathbf{k}_i, \ell) - e_{n-1} \bmod v \end{aligned}$$

$$\begin{aligned}
&= n \cdot \sum_{i=1}^n x_{i,\ell} + n - e_{n-1} \bmod v \\
&= n \cdot \sum_{i=1}^n x_{i,\ell} + n - e_{n-1}
\end{aligned}$$

For the last step to hold, we need that

$$0 \leq n \cdot \sum_{i=1}^n x_{i,\ell} + n - e_{n-1} < v$$

e_{n-1} the small value is 0 and the largest value is $n-1$ which requires that $\sum_{i=1}^n x_{i,\ell} < (v-n)/n$. Note that we already require $\lfloor p/u \rfloor > n\Delta$, $\lfloor u/v \rfloor > t\Delta \implies \lfloor p/v \rfloor > nt\Delta^2$. In other words, $\sum x_i < \frac{p}{n^2 t \Delta^2}$. Finally, $X'_\ell = n \cdot \sum_{i=1}^n x_{i,\ell} + n$ and that completes the remaining steps.

E Deferred Proofs

E.1 Simulation-Based Proofs of Security

E.2 Proof of Theorem 9

Theorem 9. *Construction 8 is a secure PRF where H is modeled as a random oracle under the HSM_M assumption.*

Proof. We denote the challenger by $\mathcal{B}$. Let S_j be the event that the adversary wins in Hybrid_j for each $j \in \{0, \dots, 2\}$. Let q_e (resp. q_h) denote the number of evaluation queries (resp. hash oracle queries) that the adversary makes. We use an analysis similar to the technique by Coron [37].

Hybrid₀(κ): Corresponds to the security game as defined for security of PRF. It follows that the advantage of the adversary is

$$\text{Adv}_0 = 2 \cdot |\Pr[S_0] - 1/2| = \text{Adv}_{\mathcal{A}}^{\text{PRF}}$$

Hybrid₁(κ): This game is identical to Hybrid_0 with the following difference. The challenger tosses biased coin δ_t for each random oracle query $H(t)$. The biasing of the coin is as follows: takes a value 1 with probability $\frac{1}{q_e+1}$ and 0 with probability $\frac{q_e}{q_e+1}$. Then, one can consider the following event E : that the adversary makes a query to the random oracle with x_i as an input where x_i was one of the evaluation inputs and for this choice we have that δ_t was flipped to 0.

If E happens, the challenger halts and declares failure. Then, we have that:

$$\Pr[\neg E] = \left(\frac{q_e}{q_e+1} \right)^{q_e} \geq \frac{1}{e(q_e+1)}$$

where e is the Napier's constant. Finally, we get that:

$$\Pr[S_1] = \Pr[S_0] \cdot \Pr[\neg E] \geq \frac{\Pr[S_0]}{e(q_e+1)}$$

Hybrid₂(κ): This game is similar to Hybrid_1 with the following difference: we modify the random oracle outputs.

- If $\delta_t = 0$, the challenger samples $w_t \leftarrow \mathcal{D}_H$ and sets $H(t) = h^{w_t}$
- If $\delta_t = 1$, the challenger samples $w_t \leftarrow \mathcal{D}_H$, $u_t \leftarrow \mathbb{Z}/M\mathbb{Z}$ and sets $H(t) = h^{w_t} \cdot f^{u_t}$

Note that, under the HSM assumption, an adversary cannot distinguish between the two hybrids. Therefore, we get:

$$|\Pr[S_2] - \Pr[S_1]| \leq \epsilon_{\text{HSM}_M}$$

where ϵ_{HSM_M} is the advantage that an adversary has in the HSM_M game. Note that Hybrid_2 corresponds to the case where the outputs are all random elements in $\mathbb{G}$. Therefore, the inputs are sufficiently masked and leak no information about the key. Therefore, $\Pr[S_2] = 0$. Then,

$$\text{Adv}_{\mathcal{A}}^{\text{PRF}} \leq (e \cdot (q_e + 1)) \cdot \epsilon_{\text{HSM}_M}$$

□

Remark 7. Note that the above scheme is simply an adaptation of the famous DDH-based construction of a key-homomorphic PRF that was shown to be secure by Naor *et al.* [76]. It is easy to verify that our construction is also key homomorphic as $H(x)^{(k_1+k_2)} = H(x)^{k_1} \cdot H(x)^{k_2}$.

E.3 Distributed Pseudorandom Function

Construction 9 (Distributed PRF in CL Framework). A (r, m) -Distributed PRF is a tuple of PPT algorithms $\text{DPRF} := (\text{Gen}, \text{Share}, \text{Eval}, \text{P-Eval}, \text{Combine})$ with the algorithms as defined in Figure 8. For simplicity, in the construction below we will set the corruption threshold $t = r - 1$. Though, the construction also holds for a lower t .

Correctness. For a polynomial $f \in \mathbb{Z}[X]$, every $f(i)$ leaks information about the secret $s \bmod i$ leading to a choice of polynomial f such that $f(0) = \Delta \cdot s$. For our use case, the secret is the PRF key k . Let us consider a set $\mathcal{S} = \{i_1, \dots, i_t\}$ of indices and corresponding evaluations of the polynomial f at $i_1, \dots, i_t$ giving us key shares: $k^{(i_1)}, \dots, k^{(i_t)}$. To begin with, one can compute the Lagrange coefficients corresponding to the set $\mathcal{S}$ as: $\forall i \in \mathcal{S}, \lambda_i(X) := \prod_{j \in \mathcal{S} \setminus \{i\}} \frac{x_j - X}{x_j - x_i}$. This implies that the resulting polynomial is $f(X) := \sum_{j=1}^t \lambda_{i_j}(X) \cdot k^{(i_j)}$.

However, $\lambda_i(X)$ requires one to perform a division $x_j - x_i$ which is undefined as $\mathbb{H}$ hashes to $\mathbb{G}$ whose order is unknown. To avoid this issue, a standard technique is to instead compute coefficient $\Lambda_i(X) := \Delta \cdot \lambda_i(x)$. Thereby, the resulting polynomial that is reconstructed is $f'(X) = \Delta \cdot f(X) = \sum_{j=1}^t \Lambda_{i_j}(X) \cdot k^{(i_j)}$. Consequently,

$$\begin{aligned} \mathbb{H}(x)^{\Delta^3 \cdot k} &= \mathbb{H}(x)^{\Delta \cdot f'(0)} = \mathbb{H}(x)^{\Delta \cdot \sum_{j=1}^t \Lambda_{i_j}(0) \cdot k^{(i_j)}} \\ &= \prod_{j=1}^t \left(\mathbb{H}(x)^{\Delta \cdot k^{(i_j)}} \right)^{\Lambda_{i_j}(0)} \\ &= \prod_{j=1}^t \left(\text{P-Eval}(k^{(i_j)}, x) \right)^{\Lambda_{i_j}(0)} \end{aligned}$$

Thus, our protocol is correct.

Pseudorandomness. Next, we consider the pseudorandomness property of our construction.

Theorem 10. *In the Random Oracle Model, if Construction 8 is a secure pseudorandom function if Integer Secret Sharing is statistically private, then Construction 9 is pseudorandom in the static corruptions setting.*

Boneh *et al.* [18] showed that from any Key Homomorphic PRF (which Construction 8), one can build a Distributed PRF. The proof of the following theorem follows the template of this scheme with certain important adaptations as our secret sharing scheme is over integers. The proof technique is to show that if there exists an adversary $\mathcal{A}$ that can break the DPRF security, one can then use it to build an adversary $\mathcal{B}$ to

break the pseudorandomness of our original PRF, as defined in Construction 8. The idea behind the proof is for $\mathcal{B}$, upon receiving choice of $t - 1$ corruptions as indices $i_1, \dots, i_{t-1}$, to then choose a random index i_t and implicitly set $k^{(i_t)}$ to be the PRF key chosen by its challenger. Therefore, the $\mathcal{B}$ now has knowledge of t indices, with which it can sample the Lagrange coefficients as before:

for $j = 1, \dots, t$ **do**

$$\Lambda_{i_j}(X) := \prod_{\zeta \in \{1, \dots, t\} \setminus j} \frac{i_\zeta - X}{i_\zeta - i_j} \cdot (\Delta)$$

Now, $\mathcal{B}$ with knowledge of the keys for indices $i_1, \dots, i_{t-1}$ along with access to Oracle needs to simulate valid responses to P-Eval queries for an unknown index. Call this index i^* . Then, we have:

$$\begin{aligned} \text{P-Eval}(k^{(i^*)}, x) &:= H(x)^{\Delta \cdot k^{(i^*)}} = H(x)^{\sum_{j=1}^t \Lambda_{i_j}(i^*) \cdot k^{(i_j)}} \\ &= H(x)^{\sum_{j=1}^{t-1} \Lambda_{i_j}(i^*) \cdot k^{(i_j)}} \cdot \left(H(x)^{k^{(i_t)}} \right)^{\Lambda_{i_t}(i^*)} \end{aligned}$$

The last term is simulated using $\mathcal{B}$'s own oracle access.

Proof. Let $\mathcal{A}$ be a PPT attacker against the pseudorandomness property of DPRF, having advantage ϵ .

$\mathcal{A}$ first chooses $t - 1$ indices $\mathbf{K} = \{i_1, \dots, i_{t-1}\}$ where each index is a subset of $\{1, \dots, m\}$. $\mathcal{A}$ receives the shares of the keys $k^{(i_1)} = f(i_1), \dots, k^{(i_{t-1})} = f(i_{t-1})$ (for unknown polynomial f of degree t such that $f(0) = k \cdot \Delta$ with $\Delta := \Delta$). Further, $\mathcal{A}$ has access to $\mathcal{O}_{\text{Eval}}(i, x)$ receiving P-Eval(k_i, x) in response. Additionally, $\mathcal{A}$ expects to have oracle access to the random oracle H .

Using this attacker $\mathcal{A}$, we now define a PPT attacker $\mathcal{B}$ which will break the pseudorandomness property of Construction 8. Note that $\mathcal{B}$ is given access to the oracle that either outputs the real evaluation of the PRF on key k^* or a random value. Additionally, $\mathcal{B}$ expects to have oracle access to the random oracle H .

- **Setup:** $\mathcal{B}$ does the following during Setup.
 - Receive set $S = \{i_1, \dots, i_{t-1}\}$ from $\mathcal{A}$.
 - Next $\mathcal{B}$ generates the key shares and public key as follows:
 - * Sample $k^{(i_1)}, \dots, k^{(i_{t-1})} \in \mathbb{Z}$.
 - * $\mathcal{B}$ picks an index i_t at random and implicitly sets the PRF key chosen by its challenger as $k^{(i_t)}$.
 - * Immediately, given the t indices, one can construct the secret sharing polynomial $f \in \mathbb{Z}[X]$ as described earlier, but instead recreating the polynomial $f'(X)$ using the coefficients $\Lambda_{i_j}(X)$ for $j = 1, \dots, t$ with $k^{(i_t)}$ being unknown to $\mathcal{B}$ and using its challenger to simulate a response.
 - * $\mathcal{B}$ gives $k^{(i_1)}, \dots, k^{(i_{t-1})}$ to $\mathcal{A}$.
- **Queries to H:** $\mathcal{B}$ merely responds to all queries from $\mathcal{A}$ to H by using its oracle access to H .
- **Queries to Partial Evaluation:** $\mathcal{B}$ receives as query input, some choice of key index specified by i^* and input x_j for $i = 1, \dots, Q$. In response $\mathcal{B}$ does the following:
 - Forward x_j to its challenger. In response it implicitly receives P-Eval($k^{(i_t)}, x_j$), but off by a factor of Δ in the exponent. Call this $h_{j,t}$.
 - Compute: $h_{j,i^*} = H(x_j)^{\sum_{i=1}^{t-1} \Lambda_{i_j}(i^*) \cdot k^{(i_j)}} \cdot (h_{j,t})^{\Lambda_{i_t}(i^*)}$ where $\mathcal{B}$ uses its own access to hash oracle to get $H(x_j)$.
 - It returns h_{j,i^*} to $\mathcal{A}$.
- **Challenge Query:** On receiving the challenge input x^* , $\mathcal{B}$ does the following:
 - Ensure that it is a valid input, i.e., there is no partial evaluation queries on x^* at any unknown index point.

- If not, $\mathcal{B}$ forwards to its challenger x^* . In response it implicitly receives $\text{P-Eval}(k^{(i_t)}, x^*)$, but off by a factor of Δ in the exponent. Call this h^* .
- It also uses its oracle access to $\mathbf{H}$ to receive $h = \mathbf{H}(x^*)$.
- It finally computes $y = \mathbf{H}(x_j)^{\Delta^2 \cdot \sum_{i=1}^{t-1} \Lambda_{i_j}(0) \cdot k^{(i_j)}} \cdot (h^*)^{\Delta^2 \cdot \Lambda_{i_t}(0)}$ and outputs y to $\mathcal{A}$

• **Finish:** It forwards $\mathcal{A}$'s guess as its own guess.

Analysis of the Reduction. Note that for the case when $b = 0$, $\mathcal{A}$ expects to receive $\mathbf{H}(x^*)^{\Delta^3 \cdot k}$ where k is defined at the point 0. So, we get:

$$\begin{aligned} \mathbf{H}(x^*)^{\Delta^3 \cdot k^{(0)}} &= \mathbf{H}(x^*)^{\Delta^2 \cdot f'(0)} = \mathbf{H}(x)^{\Delta^2 \sum_{j=1}^t \Lambda_{i_j}(0) \cdot k^{(i_j)}} \\ &= \mathbf{H}(x)^{\Delta^2 \sum_{i=1}^{t-1} \Lambda_{i_j}(0) \cdot k^{(i_j)}} \cdot \left(\mathbf{H}(x)^{k^{(i_t)}} \right)^{\Delta^2 \Lambda_{i_t}(0)} \end{aligned}$$

This shows that the returned value y is consistent when $b = 0$. Meanwhile, when $b = 1$, h^* is a random element in the group and then y is a truly random value which means that $\mathcal{B}$ has produced a valid random output for $\mathcal{A}$. Similarly, when $b = 0$, every response to partial evaluation is also done consistently by correctness of the underlying secret sharing scheme. Meanwhile, when $b = 1$, we can rely on the statistical privacy preserving guarantee of the underlying secret sharing scheme to argue that the difference that the adversary can notice is statistically negligible. This concludes the proof where $\mathcal{B}$ can only succeed with advantage ϵ . $\square$

F Heterogeneity and Poisoning Attacks

Secure Aggregation was a useful tool to realize FedAvg [75]. However, research [86] has shown that FedAvg does not yield good accuracy or model convergence when confronted with non-i.i.d client dataset distribution. Prior works such as DReS-FL [89] often require expensive cryptographic techniques to be resilient to heterogeneous datasets. For example, DReS-FL requires that each client secretly share its datasets with other clients before having each client train. In this section, we show how to combine secure aggregation with FedOpt algorithm [86], extending existing secure aggregation techniques to privacy-preserving federated learning that can handle heterogeneity in the dataset.

F.1 FedOpt

FedOpt [86] is a family of algorithms that abstracts (and generalizes FedAvg). It allows for a choice of optimizer other than Stochastic Gradient Descent (SGD) on the client side and a more resilient update rule on the server side. Indeed, the work of Reddi *et al.* [86], also presents instantiations of various server-side update rules. See Algorithm 1 for pseudocode where the text is in black. Here T is the number of iterations, and x_t is the global model at t . At the start of every iteration, each client i sets its model $x_{i,0}^t$ to be x_t . Meanwhile, K is the number of local iterations the client performs, with k being the iterating variable. **ClientOpt** is the algorithm employed by the client based on its local learning rate η_i to update the model $x_{i,k}^t$ to $x_{i,k+1}^t$ (line 8). Δ_i^t is the update between the global model at iteration t and the local model at the end of K local iterations, at iteration t . The former is denoted by x_t while the latter is $x_{i,K}^t$ (line 9). Finally, line 13 shows the server side optimization **ServerOpt** to update the current global model x_t based on the computed aggregate of clients Δ_t along with global learning rate η .

FedOpt and OPA. Observe that the input to **ServerOpt** is independent of the individual client updates and instead only takes as parameter Δ_t (the *average* of client updates Δ_i^t), the current model x_t , learning parameter η , and iteration count t (line 11). Therefore, with Secure Aggregation, the server can compute Δ_t , while preserving the honest client's updates Δ_i^t , and later rely on a suitable **ServerOpt** that is more resilient

Algorithm 1 FedOpt and OPA

```
1: Input:  $x_0$ , ClientOpt, ServerOpt
2: for  $t = 0$  to  $T - 1$  do
3:   Sample a subset  $S$  of clients and subset  $C$  of committee clients // We can use the approach of Flamingo
   to generate  $C$ , independent of server.
4:    $x_{i,0}^t = x_t$ 
5:   for each client  $i \in S$  in parallel do
6:     for  $k = 0$  to  $K - 1$  do
7:       Compute an unbiased estimate  $g_{i,k}^t$  of  $\nabla F_i(x_{i,k}^t)$ 
8:        $x_{i,k+1}^t = \text{ClientOpt}(x_{i,k}^t, g_{i,k}^t, \eta_i, t)$ 
9:        $\Delta_i^t = x_{i,K}^t - x_t$ 
10:      Use  $\text{OPA}_{\text{LWR}}$  to send masked  $\Delta_i^t$  to server and route the encrypted seed share to  $C$ .
11:      Reconstruct the seed sum and aggregate the masked updates.
12:      Recover  $\Delta_t = \frac{1}{|S|} \sum_{i \in S} \Delta_i^t$ 
13:       $x_{t+1} = \text{ServerOpt}(x_t, -\Delta_t, \eta, t)$ 
```

to heterogeneity. We concretely formalize the pseudocode in Algorithm 1, with the additional steps needed marked in blue.

F.2 Byzantine-Robust Stochastic Aggregation (bRSA)

bRSA [65] is a class of stochastic sub-gradient methods for distributed learning resilient to Byzantine workers (i.e., clients sending arbitrary inputs). It mitigates the effects of incorrect messages due to poisoning behaviors, communication failures, or uneven data distribution by incorporating a regularization term in the objective function. At each iteration t , clients compute parameter updates based on local data, prior local models, and global parameters.

At each iteration k , client i computes parameter updates based on local data (ξ_i^k) , prior local models (x_i^k) , and global parameters (w^k) . The client and server updates are:

$$\begin{aligned} \text{Client: } x_i^{k+1} &= x_i^k - \eta^k \left(\nabla F(x_i^k, \xi_i^k) + \lambda \text{sign}(x_i^k - w^k) \right) \\ \text{Server: } w^{k+1} &= w^k - \eta^k \left(\nabla f_0(w^k) + \lambda \sum_{i \in [n]} \text{sign}(w^k - x_i^k) \right) \end{aligned}$$

where η is the learning rate, ξ is a local dataset sample, $F(\cdot, \cdot)$ is the loss function, $f_{\ell_2}(\cdot)$ is the robust regularization term, λ weights the robustness term, sign is element-wise, and $[n]$ is the client set.

However, unlike FedOpt, this approach aggregates not the model gradient but rather a function of the current worldwide model and the gradient update sent by the client. Specifically, the server computes $\text{sign}(w^k, x_i^k)$, which can be viewed as a function of how far away the client's gradient (x_i^k) is from the current global model (w^k) . These are then added before proceeding with additional server-side optimization.

bRSA and OPA. As pointed out by Franzese *et al.* [48], the only information needed by the server to aggregate is $\text{sign}(\Delta_i^t - x_t)$. The clients, rather than providing Δ_i^t , computes locally $\text{sign}(\Delta_i^t - x_t)$ and provides this as an input to the server. Note that this is simply a vector with elements in $\{-1, 1\}$. Let this be vector $\mathbf{u}_i \in \{-1, 1\}^L$ where L is the size of the model.

We can optimize further by sending a binary vector instead (say $\mathbf{v}_i$) with the property that $\mathbf{v}_i[j] = 0$ iff $\mathbf{u}_i[j] = 0$ for $j = 1, \dots, L$. Then, while the server requires $\sum \mathbf{u}_i$, this is equivalent to $2 \cdot \sum_{i=1}^n \mathbf{v}_i - n$. Sending $\mathbf{v}_i$ lends itself to efficient zero-knowledge proof to show that a masked input is a binary vector.

Protocol Malicious Security with Abort

Setup: Let $\mathcal{H} : \{0, 1\}^* \rightarrow \mathbb{F}^{m-d-2}$ where $d = r - 1$ be a hash function modeled as a random oracle. Let $\mathcal{H}' : \{0, 1\}^* \rightarrow \mathbb{F}$ be the hash function used to generate the challenge. Let G be a group generated by g where the Discrete Logarithm and DDH problems are hard, and G is of prime order q , the same as the order of the field used for Shamir Secret Sharing.

Client i : Performs the following steps:

1. Commit to $\mathbf{sd}_{i,\ell}^{(0)} = \mathbf{sd}_{i,\ell}, \mathbf{sd}_{i,\ell}^{(1)}, \dots, \mathbf{sd}_{i,\ell}^{(m)}$ as $C_i^{(j)} := g^{\mathbf{sd}_{i,\ell}^{(j)}}$.
2. Generate the coefficients of the polynomial of degree $m - d - 2$ using the Fiat-Shamir transform:
 $m_0, \dots, m_{m-d-2} \leftarrow \mathcal{H}(C_i^{(0)}, \dots, C_i^{(m)})$.
3. Compute $v_0, \dots, v_m$ as $v_i := \prod_{j \in \{0, \dots, m\} \setminus i} (i - j)^{-1}$.
4. Compute $\mathbf{w} := (v_0 \cdot m^*(0), \dots, v_m \cdot m^*(m))$.
5. Generate $\mathbf{t} := (t_0, \dots, t_m) \leftarrow \mathbb{F}$.
6. Commit to $t_0, \dots, t_m$ as $C_t^{(j)} := g^{t_j}$.
7. Compute $r := \langle \mathbf{t}, \mathbf{w} \rangle$.
8. Compute $c := \mathcal{H}'(C_i^{(0)}, \dots, C_i^{(m)}, C_t^{(0)}, \dots, C_t^{(m)}, \mathbf{w}, r)$.
9. Compute $z_0, \dots, z_m$ where $z_i := t_i + c \cdot \mathbf{sd}_{i,\ell}^{(i)}$.
10. Set $\pi_i := (\{C_i^{(j)}\}, r, \mathbf{z} = (z_0, \dots, z_m), c)$.

Server: Upon receiving π_i from client i , performs the following:

1. Parse $\pi_i := (\{C_i^{(j)}\}, r, \mathbf{z} = (z_0, \dots, z_m), c)$.
2. Compute $\mathbf{w}$ (as done by the client) and check if $\langle \mathbf{w}, \mathbf{z} \rangle = r$.
3. For each $j = 0, \dots, m$, compute $C_t^{(j)} = g^{z_j} \cdot (C_i^{(j)})^{-c}$.
4. Compute $c' = \mathcal{H}'(C_i^{(0)}, \dots, C_i^{(m)}, C_t^{(0)}, \dots, C_t^{(m)}, \mathbf{w}, r)$.
5. Accept input from client i if $c = c'$, else client i is dropped.
6. Send $C_i^{(j)}$ and the encrypted shares for committee member j to committee member j .

Committee Member j : Upon receiving data from the server:

1. Decrypt and recover the share $\mathbf{sd}_{i,\ell}^{(j)}$.
2. Verify that the recovered share matches the commitment forwarded by the server.
3. If verification fails, complain to the server.

Server: 1. If any complaint is received, protocol is aborted.

2. Server checks if $\prod_{i \in \mathcal{C}} C_i^{(0)} \stackrel{?}{=} g^{\sum_{i \in \mathcal{C}} \mathbf{sd}_{i,\ell}}$ where $\sum_{i \in \mathcal{C}} \mathbf{sd}_{i,\ell}$ is obtained by reconstruction from the shares.

Figure 11: Malicious Security with Abort

G Stronger Security Definition

Hitherto, we have only considered the indistinguishability of information from the perspective of the server. However, one can consider the requirement to hold for even corrupt committee members. Specifically, the client's input remains hidden if their entire committee collude (or at least t of them). It is easy to observe

that Figure 9 does not satisfy the stronger security definition. If we had a single committee member, the auxiliary information (available to the committee member) masks the input and, therefore, can be unmasked. Thus, to accommodate security against collusion of all committee members, we modify OPA syntax and construction to include a key from the server to keep client privacy. Informally, we do the following:

- First, the server or the aggregator also has a secret key, denoted by k_0 .
- Second, for each label ℓ , the server first publishes a “public key”, as a function of the following algorithm $\text{aux}_{0,\ell} \leftarrow \text{PublicKeyGen}(k_0, \ell)$
- Third, the encrypt procedure takes into account this auxiliary information, i.e., $\left(\text{ct}_{i,\ell}, \left\{ \text{aux}_{i,\ell}^{(j)} \right\}_{j \in [m]} \right) \leftarrow^s \text{Enc}(\text{pp}, k_i, x_i, \text{aux}_{0,\ell}, \mathbf{t}, \mathbf{m}, \ell)$. In other words, the server publishes the public key, and then the client can encrypt it to a label.
- Fourth, the decrypt procedure takes k_0 as input too.

The committee indistinguishability game proceeds in phases.

- **Setup Phase:** The challenger begins by running the setup algorithm to generate the system parameters. The adversary is then provided with the system parameters pp and is asked to output an adversarial choice of n , which is the number of users that will be registered. In response, the challenger runs the KeyGen algorithm $n + 1$ times, each for the n users and once for the server’s secret key. This phase ends with the adversary being provided with the server’s secret key denoted by k_0 .
- **Learning Phase:** The adversary issues queries to the various oracles defined by $\mathcal{O}_{\text{Corr}}, \mathcal{O}_{\text{Enc}}$ to learn any information it could. $\mathcal{O}_{\text{Corr}}$ proceeds where the adversary can corrupt any user and receive its key. These corruptions are tracked. Meanwhile, $\mathcal{O}_{\text{Enc}}$ allows the adversary to issue any arbitrary encryption queries on behalf of any of the users, with the restriction that it can only do so once per user per label. In response, it receives both the ciphertext encrypting the input and *all* the auxiliary information.

This phase ends with the adversary committing to a target label τ .

- **Challenge Phase:** In this phase, the challenger begins by identifying eligible users $\mathcal{U}$ who are honest, which is defined by $[n] \setminus \mathbf{K}$. Without loss of generality, we assume there have been no queries to $\mathcal{O}_{\text{Enc}}$ with τ as the label. Should there be such queries, those users i such that $(i, \tau, \cdot) \in \mathbf{E}$ are also removed from the set $\mathcal{U}$, and these inputs are later used to compute the challenge. Upon receiving $\mathcal{U}$, the adversary commits to two sets: $\mathcal{H} \subseteq \mathcal{U}$ is the set of honest users that the adversary is targeting, and $\mathcal{S}$ that is the set of committee members for whom the adversary receives $\left\{ \text{aux}_{i,\tau}^{(j)} \right\}_{i \in \mathcal{H}, j \in \mathcal{S}}$ provided $|\mathcal{S}| \leq \mathbf{t} - 1$. Further, the adversary also provides inputs two choices of inputs for user in $\mathcal{H}$ denoted by $\{x_{i,0}, x_{i,1}\}_{i \in \mathcal{H}}$ and inputs $\{x_i\}_{i \in [n] \setminus \mathcal{H}}$ for the remaining users.
- Finally, the adversary is provided with individual encryptions and auxiliary information for all committee members.
- **Guessing Phase:** The adversary outputs a guess b' and wins if $b' = b$, provided trivial attacks do not happen.

Definition 13 (C-IND-CPA Security). *We say that a $(\mathbf{t}, \mathbf{m}, \mathbf{M})$ One-shot Private Aggregation Scheme OPA with label space $\mathcal{L}$ is Server-Indistinguishable under Chosen Plaintext Attack (S-IND-CPA) if for any PPT*

adversary $\mathcal{A}$, there exists a negligible function negl such that:

$$\Pr \left[\begin{array}{c} b = b' \\ \text{pp} \leftarrow \text{Setup}(1^\kappa); b \leftarrow \{0, 1\} \\ (st, n) \leftarrow \mathcal{A}(\text{pp}), \{k_i \leftarrow \text{KeyGen}()\}_{i \in [n] \cup \{0\}} \\ (st, \tau) \leftarrow \mathcal{A}^{\mathcal{O}_{\text{Corr}}, \mathcal{O}_{\text{Enc}}}(st, k_0), \mathcal{U} := [n] \setminus K \\ (\mathcal{H}, \mathcal{S}, \{x_{i,0}, x_{i,1}\}_{i \in \mathcal{H}}, \{x_i\}_{i \in [n] \setminus \mathcal{H}}) \leftarrow \mathcal{A}(st, \mathcal{U}) \\ \left\{ \text{ct}_{i,\tau}, \left\{ \text{aux}_{i,\tau}^{(j)} \right\}_{j \in [m]} \leftarrow \text{Enc}(\text{pp}, r_i, x_{i,b}) \right\}_{i \in \mathcal{H}} \\ \left\{ \text{ct}_{i,\tau}, \left\{ \text{aux}_{i,\tau}^{(j)} \right\}_{j \in [m]} \leftarrow \text{Enc}(\text{pp}, r_i, x_i) \right\}_{i \in [n] \setminus \mathcal{H}} \\ b' \leftarrow \mathcal{A}(st, \left\{ \text{ct}_{i,\tau}, \text{aux}_{i,\tau}^{(j)} \right\}_{i \in [n], j \in [m]}) \end{array} \right] \leq \frac{1}{2} + \text{negl}(\kappa)$$

G.1 Updated Committee Indistinguishable Construction

These are the changes to OPA construction based on the HSM_M assumption to adapt it to the stronger security definition:

- PublicKeyGen(k_0, ℓ)
 Compute $\text{pk}_{0,\ell} \leftarrow \text{DPRF.Eval}(k_0, \ell)$
return $\text{pk}_{0,\ell}$
- Modify the encryption procedure as follows:

Enc(pp, $k_i, x_i, \text{pk}_{0,\ell}, t, m, \ell$)

Parse $k_i = k_i$
 Compute $h_{i,\ell} = \text{DPRF.Eval}(k_i, \ell)$
 Compute $\text{ct}_{i,\ell} = f^{x_i} \cdot \text{pk}_{0,\ell}^{k_i}$
 Compute $(k_i^{(j)})_{j \in [m]} \leftarrow \text{DPRF.Share}(k_i, t, m)$
for $j = 1, \dots, m$ **do**
 $\text{aux}_{i,\ell}^{(j)} = \text{DPRF.Eval}(k_i^{(j)}, \ell)$
return $\text{ct}_{i,\ell}, \left\{ \text{aux}_{i,\ell}^{(j)} \right\}_{j \in [m]}$

In other words, $\text{aux}_{i,\ell}^{(j)}$ can be viewed as the j -th partial evaluation of the key (k_i) where k_i was key shared using Secret Sharing scheme, while $\text{ct}_{i,\ell}$ was masked by the DPRF evaluation on the key $k_i \cdot k_0$

Now, the committee member multiplies all the auxiliary information. As a result, $\text{AUX}_\ell^{(j)}$ is simply a partial evaluation of the following key share $\sum_{i=1}^n k_i^{(j)}$. Therefore, the server computes $\text{DPRF.Eval}(\cdot, (\sum_{i=1}^n k_i))$. Now, let us look at the decryption procedure:

Aggregate($\text{AUX}_\ell, k_0, \{\text{ct}_{i,\ell}\}_{i \in \mathcal{C}}$)

Compute $M = \text{AUX}_\ell^{-k_0} \cdot (\prod_{i \in \mathcal{C}} \text{ct}_{i,\ell})$
 Compute $X_\ell \leftarrow \text{CLSolve}(\text{pp}, M)$
return $X_\ell \bmod M$

The security of this construction follows from the intuition that the adversary gets all of the auxiliary information, from which it can only construct a Diffie-Hellman key on the fly, from which it cannot compute any masking information to unmask the inputs.